code dont f4il me now...

Shahjalal University of Science and Technology

1 Scripts & Builds 1
- 1.1 sublime . . . 1
- 1.2 vs code . . . 1

2 Number Theory 1
- 2.1 Phi sieve, Phi brute . . . 1
- 2.2 Mobius Inversion . . . 1
- 2.3 Segmented Sieve . . . 2
- 2.4 Extended GCD . . . 2
- 2.5 CRT . . . 2
- 2.6 Diophantine Equation . . . 2
- 2.7 Miller Rabin & Pollar Rho . . . 2
- 2.8 Mod Square Root . . . 2
- 2.9 Discrete Logarithm . . . 2
- 2.10 Floor Sum . . . 2
- 2.11 Ballot Number . . . 3
- 2.12 Multiplicative Order . . . 3
- 2.13 Power Tower . . . 3
- 2.14 Dirichlet Convolution . . . 3

3 Data Structure 3
- 3.1 Fenwick Tree . . . 3
- 3.2 Fenwick Tree 2D Index Compression . . . 3
- 3.3 Fenwick 2D . . . 3
- 3.4 Segment Tree . . . 3
- 3.5 Lazy Propagation . . . 4
- 3.6 Persistent Segment Tree . . . 4
- 3.7 Persistent Lazy Propagation . . . 4
- 3.8 Segment Tree 2D . . . 4
- 3.9 Segment Tree 2D (Index compression) . . . 5
- 3.10 Merse Sort Tree with Update . . . 5
- 3.11 Treap . . . 5
- 3.12 Persistent DSU . . . 6
- 3.13 MOs with updates . . . 6
- 3.14 MOs on Trees . . . 6
- 3.15 Hilbert Curve . . . 6
- 3.16 Dynamic Connectivity . . . 6
- 3.17 Xor Basis . . . 6
- 3.18 Link Cut Tree . . . 6
- 3.19 Wavelet Tree . . . 7
- 3.20 Sparse Table (RMQ) . . . 7

4 Dynamic Programming 7
- 4.1 WQS Binary Search . . . 7
- 4.2 Divide & Conquer . . . 7
- 4.3 SOS DP . . . 7
- 4.4 CHT . . . 7
- 4.5 Dynamic CHT . . . 8

5 Numerical 8
- 5.1 FFT . . . 8
- 5.2 FFT With mod M . . . 8
- 5.3 NTT . . . 8
- 5.4 FWHT . . . 8
- 5.5 GCD & LCM Convolution . . . 8
- 5.6 Berlekamp Messey . . . 9

6 Combinatorics 9
- 6.1 Mod Template . . . 9
- 6.2 Fast Multipoint Evaluation . . . 9
- 6.3 Stirling 1st with Fixed n . . . 9
- 6.4 Stirling 1st with Fixed k . . . 9

7 Linear Algebra 9
- 7.1 The Matrix Class . . . 9
- 7.2 Gaussian Elimination . . . 10
- 7.3 Determinant . . . 10
- 7.4 Matrix Inverse . . . 10

8 Graph 10
- 8.1 Bellmanford . . . 10
- 8.2 Articulation Point & Bridge . . . 10
- 8.3 Block Cut Tree . . . 10
- 8.4 LCA in O(1) sparse table . . . 11
- 8.5 SCC . . . 11
- 8.6 Kosaraju . . . 11
- 8.7 HLD . . . 11
- 8.8 Arpa's Trick . . . 11

9 String 11
- 9.1 Trie . . . 11
- 9.2 Bit Trie . . . 12
- 9.3 Z Algorithm . . . 12
- 9.4 Manacher . . . 12
- 9.5 KMP . . . 12
- 9.6 Hashing . . . 12
- 9.7 Suffix Array . . . 12
- 9.8 Suffix Automaton . . . 12
- 9.9 Aho Corasick . . . 13
- 9.10 Palindromic Tree . . . 13

10 Flow and Matching 13
- 10.1 Dinic . . . 13
- 10.2 Min Cost Max Flow . . . 13
- 10.3 Bipartite Matching - Hopcroft Karp . . . 13

11 Geometry 14
- 11.1 Geometry Primitives . . . 14
- 11.2 Point 2D . . . 14
- 11.3 Polar Sort . . . 14
- 11.4 Line Intersection . . . 14
- 11.5 Line Equation . . . 14
- 11.6 Segment Distance . . . 14
- 11.7 Segment Intersection . . . 14
- 11.8 Side Of . . . 14
- 11.9 On Segment . . . 14
- 11.10 Linear Transformation . . . 14
- 11.11 Circle Intersection . . . 14
- 11.12 Circle Polygon Intersection . . . 14
- 11.13 Circle Line Intersection . . . 15
- 11.14 Circle Segment Intersection . . . 15
- 11.15 Circle Tangents . . . 15
- 11.16 Minimum Enclosing Circle . . . 15
- 11.17 Inside Polygon . . . 15
- 11.18 Polygon Area . . . 15
- 11.19 Polygon Center . . . 15
- 11.20 Polygon Cut . . . 15
- 11.21 Polygon Union . . . 15
- 11.22 Extreme Vertex . . . 15
- 11.23 Manhattan MST . . . 15
- 11.24 Convex Hull . . . 16
- 11.25 Hull Diameter . . . 16
- 11.26 Point Inside Hull . . . 16
- 11.27 Line Hull Intersection . . . 16
- 11.28 Half Plane Intersection . . . 16
- 11.29 Minkowski Sum & Dif . . . 16
- 11.30 Maximum Inscribed Circle . . . 16
- 11.31 Rotating Calipers . . . 16
- 11.32 Closest Pair . . . 16

12 Geometry 3D 17
- 12.1 Point 3D . . . 17
- 12.2 Plane . . . 17
- 12.3 Line 3D . . . 17
- 12.4 Plane Line . . . 17
- 12.5 Convex Hull 3d . . . 17
- 12.6 Sphere . . . 18
- 12.7 TetraHedron Volume . . . 18

13 Miscellaneous 18
- 13.1 Interval Container . . . 18
- 13.2 Efficient Mask Traversal . . . 18
- 13.3 Generationg Permutations . . . 18
- 13.4 Iterating over Submasks . . . 18
- 13.5 Random . . . 18
- 13.6 pbds . . . 18
- 13.7 Bitset . . . 19
- 13.8 Primes . . . 19
- 13.9 Digit Range Sum . . . 19
- 13.10 Custom Hash . . . 19
- 13.11 Pragma . . . 19

14 Formulae 19
- 14.1 Combinatorics . . . 19
- 14.2 Math . . . 20
- 14.3 Number Theory . . . 21
- 14.4 Geometry . . . 22
- 14.5 Miscellaneous . . . 23

1 Scripts & Builds

1.1 sublime

```
{
  "shell_cmd": "ulimit -f 10240 -v 1048576 -s
     1048576; g++ -Wall -std=c++20 -O2 -
     DLOCAL $file -o $file_base_name && /usr/
     bin/time -f \"%es | %MKB\" timeout 5 ./
     $file_base_name < input.in > output.out
     2> error.log",
  "file_regex": "^(..[^:]*):([0-9]+):?([0-9]+)
     ?:? (.*)$",
  "working_dir": "${file_path}",
  "selector": "source.cpp",
  // sanitizer flags: -g -fsanitize=address,
     undefined -fno-omit-frame-pointer
}
```

1.2 vs code

```
{
  "version": "2.0.0",
  "tasks": [{
    "label": "cp",
    "type": "shell",
    "command": "g++ -Wall -std=c++20 -O2 -
     DLOCAL \"${file}\" -o \"${fileDirname}/${
     fileBasenameNoExtension}\" && \"${
     fileDirname}/${fileBasenameNoExtension
     }\" < input.in > output.out 2> error.log
     ",
    "group": "build",
    "presentation": { "reveal": "silent" },
    "problemMatcher": ["$gcc"]
  }]
}
```

2 Number Theory

2.1 Phi sieve, Phi brute

```
int phi[MAXN];
void sieve() {
  for (int i = 0; i < MAXN; ++i) phi[i] = i;
  for (int i = 2; i < MAXN; ++i)
    if (phi[i] == i)
      for (int j = i; j < MAXN; j += i) phi[j]
       -= phi[j] / i;
}
int phi(int n) {
  int res = n;
  for (int i = 2; i * i <= n; i++) {
    if (n % i == 0) {
      while (n % i == 0) n /= i;
      res -= res / i;
    }
  }
  if (n > 1) res -= res / n;
  return res;
}
```

2.2 Mobius Inversion

```
int mob[N];
// Precomputes Mobius function mu(i) for all 1
     <= i < N.
void mobius() {
  mob[1] = 1;
  for (int i = 2; i < N; i++){
    mob[i]--;
    for (int j = i + i; j < N; j += i) mob[j]
     -= mob[i];
  }
}
bool vis[N];
vector<int> d[N];
int mul[N];
// Inserts (k=1) or removes (k=-1) x by
     updating divisor frequencies.
void add(int x, int k) {
  for (auto y: d[x]) mul[y] += k;
}
// Returns the count of active set elements
     coprime to x.
int query(int x) {
  int ans = 0;
  for (auto y: d[x]) ans += mul[y] * mob[y];
  return ans;
}
```

2.3 Segmented Sieve

```
// Generate all primes from l to r using
     segmented sieve in
// O((r - l) log (r) + sqrt(r)) // needs sieve
     (limit)
vector<int64_t> segmented_sieve(int64_t l,
     int64_t r) {
  if (l == 1) l++;
  int limit = sqrtl(r);
  while (limit * limit <= r) limit++;
  while (limit * limit > r) limit--;
  auto primes = sieve(limit);
  vector<bool> is_prime(r - l + 1, true);
  for (int64_t p : primes) {
    int64_t start = max(p * p, (l + p - 1) / p
       * p);
    for (int64_t j = start; j <= r; j += p) {
      is_prime[j - l] = false;
    }
  }
  vector<int64_t> vec;
  for (int64_t i = l; i <= r; ++i) {
    if (is_prime[i - l]) vec.push_back(i);
  }
  return vec;
}
```

2.4 Extended GCD

Description: Finds solutions (x, y) in $ax + by = gcd(a, b)$. (Check formulae for more details)

```
ll extgcd(ll a, ll b, ll &x, ll &y) {
  if (!b) { x = 1; y = 0; return a; }
  ll x1, y1, g = extgcd(b, a % b, x1, y1);
  x = y1; y = x1 - (a / b) * y1;
  return g;
}
```

2.5 CRT

Description: Computes the unique solution x to a system of congruences $x \equiv a_i \pmod{m_i}$ using the Chinese Remainder Theorem. Assumes all moduli are pairwise coprime and the answer fits in `ll`.
Usage: Store congruences as `{remainder, modulus}` pairs in `mods`. Call `crt(mods)` to obtain the smallest non-negative solution modulo the product of all moduli. Requires `extended_gcd()` and `modinv()`.
Time: $O(k \log M)$

```
bool merge(ll a1, ll m1, ll a2, ll m2, ll &res
     , ll &mod) {
  ll x, y, g = extgcd(m1, m2, x, y);
  if ((a2 - a1) % g != 0) return false;
  ll m2g = m2 / g;
  ll t = (ll)((__int128)((a2 - a1) / g % m2g)
     * (x % m2g) % m2g);
  if (t < 0) t += m2g;
  __int128 lcm_mod = (__int128)(m1 / g) * m2;
  __int128 tmp = (__int128)a1 + (__int128)m1 *
       t;
  res = (ll)(tmp % lcm_mod);
  if (res < 0) res += (ll)lcm_mod;
  mod = (ll)lcm_mod;
  return true;
}
```

2.6 Diophantine Equation

Description: Solves $ax + by = c$ where $gcd(a, b)|c$. Find any solution returns $x \in [minx, maxx]$ and $y \in [miny, maxy]$. If $solves = true$, the function returns all solution pairs (x, y). (Check formulae for more details)

```
// Solves a*x + b*y = c
bool find_any_solution(ll a,ll b,ll c,ll &x0,
     ll &y0,ll &g){
    if(a==0&&b==0){
        if(c) return false;
        x0=y0=g=0; return true;
    }
    g=extended_gcd(abs(a),abs(b),x0,y0);
    if(c%g!=0) return false;
    x0*=c/g; y0*=c/g;
    if(a<0) x0*=-1;
    if(b<0) y0*=-1;
    return true;
}
// Shift solution by cnt
void shift_solution(ll &x,ll &y,ll a,ll b,ll
     cnt){
    x+=cnt*b; y-=cnt*a;
}
pair<ll, vector<pair<ll, ll>>>
     find_all_solutions(ll a,ll b,ll c,
                        ll minx,ll maxx,ll miny,
     ll maxy, bool solves){
    ll x,y,g;
    if(!find_any_solution(a,b,c,x,y,g)) return
      {0, {}};
    if(a==0&&b==0){
        assert(c==0);
        ll cnt = 1LL*(maxx-minx+1)*(maxy-miny
     +1);
        vector<pair<ll, ll>> res;
        if(solves){
            for(ll cur_x=minx; cur_x<=maxx; ++
     cur_x){
                for(ll cur_y=miny; cur_y<=maxy
     ; ++cur_y){
                    res.push_back({cur_x,
     cur_y});
                }
            }
        }
        return {cnt, res};
    }
    if(a==0){
        ll cnt = (maxx-minx+1)*(miny<=c/b&&c/b
     <=maxy);
        vector<pair<ll, ll>> res;
        if(solves && cnt > 0){
            ll cur_y = c/b;
            for(ll cur_x=minx; cur_x<=maxx; ++
     cur_x){
                res.push_back({cur_x, cur_y});
            }
        }
        return {cnt, res};
    }
    if(b==0){
        ll cnt = (maxy-miny+1)*(minx<=c/a&&c/a
     <=maxx);
        vector<pair<ll, ll>> res;
        if(solves && cnt > 0){
            ll cur_x = c/a;
            for(ll cur_y=miny; cur_y<=maxy; ++
     cur_y){
                res.push_back({cur_x, cur_y});
            }
        }
        return {cnt, res};
    }
    ll orig_a = a, orig_b = b;
    a/=g; b/=g;
    ll sign_a=a>0?1:-1, sign_b=b>0?1:-1;
    shift_solution(x,y,a,b,(minx-x)/b);
    if(x<minx) shift_solution(x,y,a,b,sign_b);
    if(x>maxx) return {0, {}};
    ll lx1=x;
    shift_solution(x,y,a,b,(maxx-x)/b);
    if(x>maxx) shift_solution(x,y,a,b,-sign_b)
     ;
    ll rx1=x;
    shift_solution(x,y,a,b,-(miny-y)/a);
    if(y<miny) shift_solution(x,y,a,b,-sign_a)
     ;
    if(y>maxy) return {0, {}};
    ll lx2=x;
    shift_solution(x,y,a,b,-(maxy-y)/a);
    if(y>maxy) shift_solution(x,y,a,b,sign_a);
    ll rx2=x;
    if(lx2>rx2) swap(lx2,rx2);
    ll lx=max(lx1,lx2), rx=min(rx1,rx2);
    if(lx>rx) return {0, {}};
    ll cnt = (rx-lx)/abs(b)+1;
    vector<pair<ll, ll>> res;
    if(solves){
        for(ll cur_x=lx; cur_x<=rx; cur_x+=abs
     (b)){
            res.push_back({cur_x, (c-orig_a*
     cur_x)/orig_b});
        }
    }
    return {cnt, res};
}
```

2.7 Miller Rabin & Pollar Rho

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -¿ {11, 19, 11}). Time: $O(n^{1/4})$, less for numbers with small factors.

```
bool isPrime(ull n) {
  if (n < 2 || n % 6 % 4 != 1) return (n | 1)
     == 3;
  ull A[] = {2, 325, 9375, 28178, 450775,
     9780504, 1795265022},
      s = __builtin_ctzll(n - 1), d = n >> s;
  for (ull a : A) {
    ull p = modpow(a % n, d, n), i = s;
    while (p != 1 && p != n - 1 && a % n && --
     i) p = mul(p, p, n);
    if (p != n - 1 && i != s) return 0;
  }
  return 1;
}
ull pollard(ull n) {
  ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
  auto f = [&](ull x) {
    ull res = mul(x, x, n);
    return res + i >= n ? res + i - n : res +
      i;
  };
  while (t++ % 40 || __gcd(prd, n) == 1) {
    if (x == y) x = ++i, y = f(x);
    if ((q = mul(prd, max(x, y) - min(x, y), n
     ))) prd = q;
    x = f(x), y = f(f(y));
  }
  return __gcd(prd, n);
}
vector<ull> factor(ull n) {
  if (n == 1) return {};
  if (isPrime(n)) return {n};
  ull x = pollard(n);
  auto l = factor(x), r = factor(n / x);
  l.insert(l.end(), all(r));
  return l;
}
```

2.8 Mod Square Root

Description: Tonelli-Shanks algorithm for modular square roots. Finds x s.t. $x^2 = a \pmod{p}$ ($-x$ gives the other solution). **Time:** $O(\log^2 p)$ worst case, $O(\log p)$ for most p

```
ll sqrt(ll a, ll p) {
  a %= p; if (a < 0) a += p;
  if (a == 0) return 0;
  assert(modpow(a, (p-1)/2, p) == 1); // else
     no solution
  if (p % 4 == 3) return modpow(a, (p+1)/4, p)
     ;
  // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works
     if p % 8 == 5
  ll s = p - 1, n = 2;
  int r = 0, m;
  while (s % 2 == 0)
    ++r, s /= 2;
  /// find a non-square mod p
  while (modpow(n, (p - 1) / 2, p) != p - 1)
     ++n;
  ll x = modpow(a, (s + 1) / 2, p);
  ll b = modpow(a, s, p), g = modpow(n, s, p);
  for (;; r = m) {
    ll t = b;
    for (m = 0; m < r && t != 1; ++m)
      t = t * t % p;
    if (m == 0) return x;
    ll gs = modpow(g, 1LL << (r - m - 1), p);
    g = gs * gs % p;
    x = x * gs % p;
    b = b * g % p;
  }
}
```

2.9 Discrete Logarithm

Description: Solves $a^x \equiv b \pmod{m}$ for the smallest $x > 0$, using the Baby-Step Giant-Step algorithm. Supports non-coprime a and m by explicitly checking small exponents $x \leq \sqrt{m}$ before applying step shifts. Can also compute the multiplicative order of a via `modLog(a, 1, m)`.
Usage:
`modLog(a, b, m) // returns x > 0 or -1`

```
ll modLog(ll a, ll b, ll m) {
  ll n = (ll) sqrt(m) + 1, e = 1, f = 1, j =
     1;
  unordered_map<ll, ll> A;
  while (j <= n && (e = f = e * a % m) != b %
     m)
    A[e * b % m] = j++;
  if (e == b % m) return j;
  if (__gcd(m, e) == __gcd(m, b))
    rep(i,2,n+2) if (A.count(e = e * f % m))
      return n * i - A[e];
  return -1;
}
```

2.10 Floor Sum

Description: Calculates $\sum_{i=0}^{n-1} \left\lfloor \frac{a \cdot i + b}{m} \right\rfloor$ in $O(\log m)$ time Handles negative a, b and values $\geq m$ by automatically normalizing parameters modulo m. **Usage:** `fsum(n, m, a, b)`. **Time:** $O(\log m)$

```
ll fsum(ll n, ll m, ll a, ll b) {
  ll r = 0;
  if (a < 0) { ll a2 = (a % m + m) % m; r -= n
     * (n - 1) / 2 * ((a2 - a) / m); a = a2;
     }
  if (b < 0) { ll b2 = (b % m + m) % m; r -= n
     * ((b2 - b) / m); b = b2; }
  if (a >= m) { r += (n - 1) * n / 2 * (a / m)
     ; a %= m; }
  if (b >= m) { r += n * (b / m); b %= m; }
  ll y = (a * n + b) / m;
```

```
  if (!y) return r;
  ll x = y * m - b;
  return r + (n - (x + a - 1) / a) * y + fsum(
      y, a, m, (a - x % a) % a);
}
```

2.11 Ballot Number

```
// Returns the number of paths from (0,0) to (
     a,b) staying at or below y = x + k
long long ballot_with_offset(int a, int b, int
     k) {
  if (b > a + k) return 0; // Already invalid
  long long total = nCr(a + b, a);
  long long invalid = nCr(a + b, a + k + 1);
  return (total - invalid + mod) % mod;
}
// Strict Ballot Number: A > B at all non-
     empty prefixes
long long ballot_strict(int a, int b) {
  if (a <= b) return 0;
  return ballot_with_offset(a - 1, b,0),;
}
```

2.12 Multiplicative Order

```
ll multiplicative_order(ll a, ll mod) {
  if (__gcd(a, mod) != 1) return -1;
  ll m = totient(mod), p = m;
  ll ans = 2e18;
  if (power(a, p, mod) == 1) ans = p;
  vector<ll> fac;
  for (ll i = 2 ; i * i <= m; i++) {
    if (m % i == 0) {
      while(m % i == 0) m /= i, fac.push_back(
      i);
    }
  }
  if (m > 1) fac.push_back(m);
  for (auto x: fac) {
    if (power(a, p / x, mod) == 1) p /= x, ans
      = p;
  }
  assert(ans != 2e18);
  return ans;
}
```

2.13 Power Tower

```
map<ll, ll> mp;
ll phi(ll n) {
  if (mp.count(n)) return mp[n];
  ll ans = n, m = n;
  for (ll i = 2; i * i <= m; i++) {
    if (m % i == 0) {
      while (m % i == 0) m /= i;
      ans = ans / i * (i - 1);
    }
  }
  if (m > 1) ans = ans / m * (m - 1);
  return mp[n] = ans;
}
inline ll MOD(ll x, ll m) {
  if (x < m) return x;
  return x % m + m;
}
int a[N];
// if x >= log2(m), then a^x = a^(MOD(x, phi(m
     ))) % m
// Usage: ll ans = powerTower(l, r, m) % m; //
      computes (a[l] ^ a[l+1] ^ ... ^ a[r]) %
     m
ll powerTower(ll l, ll r, ll m) {
  if (l == r) return MOD(a[l], m);
  if (m == 1) return 1;
  return power(a[l], powerTower(l + 1, r, phi(
      m)), m);
}
```

2.14 Dirichlet Convolution

```
/**
Prefix sum of multiplicative functions :
  p_f : the prefix sum of f(x) (1 <= x <= T).
  p_g : the prefix sum of g(x) (0 <= x <= N).
  p_c : the prefix sum of f * g(x) (0 <= x <=
     N).
  T : the threshold, generally should be n ^
     (2 / 3). for n = 1e9, T = 1e6
Magic Functions for different f(x)
For f(x) = phi(x): Id(x) = phi * I(x) i.e. p_c
      = prefix sum of Id(x), p_g = prefix sum
     of I(x). Here Id(n) = n, I(n) = 1
For f(x) = mu(x): e(x) = mu * I(x). Here e(x)
     = x == 1 ? 1 : 0
Complexity: O(n^(2/3))
**/
namespace Dirichlet {
  //solution for f(x) = phi(x)
  const int T = 1e7 + 9;
  long long phi[T];
  gp_hash_table<long long, mint> mp;
  mint dp[T], inv;
  int sz, spf[T], prime[T];
  void init() {
    memset(spf, 0, sizeof spf);
    phi[1] = 1; sz = 0;
    for (int i = 2; i < T; i++) {
      if (spf[i] == 0) phi[i] = i - 1, spf[i]
      = i, prime[sz++] = i;
      for (int j = 0; j < sz && i * prime[j] <
      T && prime[j] <= spf[i]; j++) {
        spf[i * prime[j]] = prime[j];
        if (i % prime[j] == 0) phi[i * prime[j
    ]] = phi[i] * prime[j];
        else phi[i * prime[j]] = phi[i] * (
      prime[j] - 1);
      }
    }
    dp[0] = 0;
    for(int i = 1; i < T; i++) dp[i] = dp[i -
     1] + phi[i] % mod;
    inv = 1; // g(1)
  }
  mint p_c(long long n) {
    if (n % 2 == 0) return n / 2 % mod * ((n +
      1) % mod) % mod;
    return (n + 1) / 2 % mod * (n % mod) % mod
     ;
  }
  mint p_g(long long n) {
    return n % mod;
  }
  mint solve (long long x) {
    if (x < T) return dp[x];
    if (mp.find(x) != mp.end()) return mp[x];
    mint ans = 0;
    for (long long i = 2, last; i <= x; i =
     last + 1) {
      last = x / (x / i);
      ans += solve (x / i) * (p_g(last) - p_g(
     i - 1));
    }
    ans = p_c(x) - ans;
    ans /= inv;
    return mp[x] = ans;
  }
}
```

3 Data Structure

3.1 Fenwick Tree

Description: Use for point update + range query or range update + point query. **1-indexed**.
Time: $O(\log N)$

```
struct FT {
  int n; vector<ll> s;
  FT(int n) : n(n + 1), s(n + 1) {}
  void update(int pos, ll dif) {
    for (; pos < n; pos += pos & -pos) s[pos]
     += dif;
  }
  ll query(int pos) {
    ll res = 0;
    for (; pos > 0; pos -= pos & -pos) res +=
     s[pos];
    return res;
  }
  ll query(int l, int r) {
    return query(r) - query(l - 1);
  }
  void update(int l, int r, ll dif) {
    update(l, dif);
    if (r + 1 < n) update(r + 1, -dif);
  }
  int lower_bound(ll sum) {
    if (sum <= 0) return 0;
    int pos = 0;
    for (int pw = 1 << 25; pw; pw >>= 1) {
      if (pos + pw < n && s[pos+pw] < sum) {
        pos |= pw, sum -= s[pos];
      }
    }
    return pos + 1;
  }
};
```

3.2 Fenwick Tree 2D Index Compression

Description: Computes sums $a[i,j]$ for all $i \le I, j \le J$, and increases single elements $a[i,j]$. Requires that the elements to be updated are known in advance (call `fakeUpdate()` before `init()`).
Time: $O(\log^2 N)$. (Use persistent segment trees for $O(\log N)$)

```
struct FT2 {
  int n; vector<FT> ft;
  vector<vector<int>> ys;
  FT2(int limx) : n(limx + 1), ys(limx + 1) {}
  void fakeUpdate(int x, int y) {
    for (; x < n; x += x & -x) {
      ys[x].push_back(y);
    }
  }
  void init() {
    ft.assign(ys.size(), FT(0));
    for (int i = 0; i < n; ++i) {
      sort(ys[i].begin(), ys[i].end());
      ys[i].erase(unique(ys[i].begin(), ys[i].
     end()), ys[i].end());
      ft[i] = FT(ys[i].size());
    }
  }
  int ind(int x, int y) {
    return upper_bound(ys[x].begin(), ys[x].
      end(), y) - ys[x].begin();
  }
  void update(int x, int y, ll dif) {
    for (; x < n; x += x & -x)
      ft[x].update(ind(x, y), dif);
  }
  ll query(int x, int y) {
    ll sum = 0;
    for (; x > 0; x -= x & -x) {
      sum += ft[x].query(ind(x, y));
    }
    return sum;
  }
  ll query(int x1, int y1, int x2, int y2) {
    return query(x2,y2) - query(x1-1, y2) -
        query(x2, y1-1) + query(x1-1, y1-1);
  }
};
```

3.3 Fenwick 2D

```
struct FT2D {
  int n, m; vector<vector<ll>> s;
  FT2D(int n, int m) : n(n + 1), m(m + 1), s(n
      + 1, vector<ll>(m + 1)) {}
  FT2D(int n) : FT2D(n, n) {}
  void update(int x, int y, ll dif) {
    for (int i = x; i < n; i += i & -i)
      for (int j = y; j < m; j += j & -j) s[i
     ][j] += dif;
  }
  ll query(int x, int y) {
    ll res = 0;
    for (int i = x; i > 0; i -= i & -i)
      for (int j = y; j > 0; j -= j & -j) res
      += s[i][j];
    return res;
  }
  ll query(int x1, int y1, int x2, int y2) {
    return query(x2, y2) - query(x1 - 1, y2) -
      query(x2, y1 - 1) + query(x1 - 1, y1 -
     1);
  }
};
```

3.4 Segment Tree

```
template<typename T>
class segTree{
public:
   int sz;
   T neutral;
   vector<T> t;
   segTree(){}
   segTree(int l, T tm){
      sz=l;
      neutral=tm;
      if(__builtin_popcount(sz)!=1){
         int x=31-__builtin_clz(sz);
         sz=(1LL<<(x+1));
      }
      t.assign(((sz<<1)|1),neutral);
   }
   // need to update
   T combine(T& left, T& right){
      T tm=neutral;
      // tm=(left+right);
      return tm;
   }
   void update(int root, int tl, int tr, int
     pos, T x){
      if(tl==tr){
         t[root]=combine(t[root],x);
         return;
      }
      int mid=(tl+tr)/2;
      if(pos<=mid)    update((root<<1),tl,mid,
     pos,x);
      else            update(((root<<1)|1),mid
     +1,tr,pos,x);
      t[root]=combine(t[(root<<1)],t[((root
     <<1)|1)]);
```

```
    }
    T query(int root,  int tl, int tr, int l,
      int r){
       if(l>tr || r<tl)    return neutral;
       if(l<=tl && r>=tr)  return t[root];
       int mid=(tl+tr)/2;
       auto q1=query((root<<1),tl,mid,l,r);
       auto q2=query(((root<<1)|1),mid+1,tr,l,r
       );
       return combine(q1,q2);
    }
    void update(int pos, T x){
       update(1,0,sz-1,pos,x);
    }
    T query(int l, int r){
       return query(1,0,sz-1,l,r);
    }
};
```

3.5 Lazy Propagation

```
template<typename T>
class segTree{
public:
   int sz;
   vector<T> t;
   vector<T> lazy;
   segTree(){}
   segTree(int l){
      sz = l;
      if (__builtin_popcount(sz) != 1){
         int x = 31 - __builtin_clz(sz);
         sz = (1LL << (x + 1));
      }
      t.assign(2 * sz + 1, 0);
      lazy.assign(2 * sz + 1, 0);
   }
   void apply(int root, int tl, int tr){
      if (!lazy[root]) return;
      t[root] += lazy[root] * (tr - tl + 1);
      if (tl != tr){
         lazy[root * 2] += lazy[root];
         lazy[root * 2 + 1] += lazy[root];
      }
      lazy[root] = 0;
   }
   T combine(const T& left, const T& right){
      T tm = left + right;
      return tm;
   }
   void update(int root, int tl, int tr, int l
      , int r, T x){
      apply(root, tl, tr);
      if (l > tr || r < tl) return;
      if (l <= tl && r >= tr){
         lazy[root] += x;
         apply(root, tl, tr);
         return;
      }
      int mid = (tl + tr) / 2;
      update(root * 2, tl, mid, l, r, x);
      update(root * 2 + 1, mid + 1, tr, l, r,
     x);
      t[root] = combine(t[root * 2], t[root *
     2 + 1]);
   }
   T query(int root, int tl, int tr, int l,
      int r){
      apply(root, tl, tr);
      if (l > tr || r < tl) return 0;
      if (l <= tl && r >= tr) return t[root];
      int mid = (tl + tr) / 2;
      auto q1 = query(root * 2, tl, mid, l, r)
     ;
      auto q2 = query(root * 2 + 1, mid + 1,
     tr, l, r);
      return combine(q1, q2);
   }
   void update(int l, int r, T x){
      update(1, 0, sz - 1, l, r, x);
   }
   T query(int l, int r){
      return query(1, 0, sz - 1, l, r);
   }
};
```

3.6 Persistent Segment Tree

```
struct PST {
#define lc t[cur].l
#define rc t[cur].r
  struct node {
    int l = 0, r = 0, val = 0;
  } t[40 * N];
  int T = 0;
  int combine(int a, int b) { return a + b; }
     // change
  void pull(int cur) { t[cur].val = combine(t[
     lc].val, t[rc].val); }
  int build(int b, int e) {
    int cur = ++T;
    if (b == e) return cur;
    int mid = b + e >> 1;
    lc = build(b, mid);
    rc = build(mid + 1, e);
    pull(cur);
    return cur;
  }
  // Build tree from initial vector
  int build(int b, int e, const vector<int>& a
     ) {
    int cur = ++T;
    if (b == e) {
      t[cur].val = a[b];
      return cur;
    }
    int mid = b + e >> 1;
    lc = build(b, mid, a);
    rc = build(mid + 1, e, a);
    pull(cur);
    return cur;
  }
  int upd(int pre, int b, int e, int i, int v)
      {
    int cur = ++T;
    t[cur] = t[pre];
    if (b == e) {
      t[cur].val = v; // += for adding
      return cur;
    }
    int mid = b + e >> 1;
    if (i <= mid) lc = upd(t[pre].l, b, mid, i
     , v);
    else rc = upd(t[pre].r, mid + 1, e, i, v);
    pull(cur);
    return cur;
  }
  // Batch update: applies a sorted vector of
     updates {index, delta} in range [ql, qr]
  int upd_batch(int pre, int b, int e, const
     vector<pair<int, int>>& up, int ql, int
     qr) {
    if (ql > qr) return pre;
    int cur = ++T;
    t[cur] = t[pre];
    if (b == e) {
      for (int i = ql; i <= qr; ++i) t[cur].
     val += up[i].second;
      return cur;
    }
    int mid = b + e >> 1;
    int sp = upper_bound(up.begin() + ql, up.
     begin() + qr + 1, mid,
      [](int m, const pair<int, int>& p) {
     return m < p.first; }) - up.begin();
    lc = upd_batch(t[pre].l, b, mid, up, ql,
     sp - 1);
    rc = upd_batch(t[pre].r, mid + 1, e, up,
     sp, qr);
    pull(cur);
    return cur;
  }
  // Range query on a single version
  int query(int cur, int b, int e, int l, int
     r) {
    if (!cur || l > e || r < b) return 0; //
     Identity value (0 for sum, INF for min)
    if (l <= b && e <= r) return t[cur].val;
    int mid = b + e >> 1;
    return combine(query(lc, b, mid, l, r),
     query(rc, mid + 1, e, l, r));
  }
  // Query k-th smallest element between
     versions pre and cur
  int query_kth(int pre, int cur, int b, int e
     , int k) {
    if (b == e) return b;
    int cnt = t[lc].val - t[t[pre].l].val;
    int mid = b + e >> 1;
    if (cnt >= k) return query_kth(t[pre].l,
     lc, b, mid, k);
    else return query_kth(t[pre].r, rc, mid +
     1, e, k - cnt);
  }
#undef lc
#undef rc
};
```

3.7 Persistent Lazy Propagation

```
struct PST {
#define lc t[cur].l
#define rc t[cur].r
  struct node {
    int l = 0, r = 0;
    ll val = 0, lazy = 0;
  } t[250 * N];
  int T = 0;
  inline ll combine(ll a, ll b) {
    return a + b;
  }
  inline void pull(int cur) {
    t[cur].val = combine(t[lc].val, t[rc].val)
     ;
  }
  int build(int b, int e) {
    int cur = ++T;
    if(b == e) return cur;
    int mid = b + e >> 1;
    lc = build(b, mid);
    rc = build(mid + 1, e);
    pull(cur);
    return cur;
  }
  void apply(int cur, int b, int e, ll v) {
    t[cur].lazy += v;
    t[cur].val += (ll)(e - b + 1) * v;
  }
  void push(int cur, int b, int e) {
    if(!t[cur].lazy) return;
    if(b != e) {
      int mid = b + e >> 1;
      int nlc = ++T; t[nlc] = t[lc];
      int nrc = ++T; t[nrc] = t[rc];
      lc = nlc; rc = nrc;
      apply(lc, b, mid, t[cur].lazy);
      apply(rc, mid + 1, e, t[cur].lazy);
    }
    t[cur].lazy = 0;
  }
  int upd(int pre, int b, int e, int i, int j,
      ll v) {
    int cur = ++T;
    t[cur] = t[pre];
    if(i <= b && e <= j) {
      apply(cur, b, e, v);
      return cur;
    }
    push(cur, b, e);
    int mid = b + e >> 1;
    if(i <= mid) lc = upd(lc, b, mid, i, j, v)
     ;
    if(j > mid) rc = upd(rc, mid + 1, e, i, j,
      v);
    pull(cur);
    return cur;
  }
  // Propagating query
  ll query(int cur, int b, int e, int i, int j
     ) {
    if(!cur || b > j || e < i) return 0;
    if(i <= b && e <= j) return t[cur].val;
    push(cur, b, e);
    int mid = b + e >> 1;
    return combine(query(lc, b, mid, i, j),
     query(rc, mid + 1, e, i, j));
  }
  // Non-propagating query
  ll query(int cur, int b, int e, int i, int j
     , ll add = 0) {
    if(!cur || b > j || e < i) return 0;
    if(i <= b && e <= j) return t[cur].val + (
     ll)(e - b + 1) * add;
    int mid = (b + e) >> 1;
    add += t[cur].lazy;
    return combine(query(lc, b, mid, i, j, add
     ), query(rc, mid + 1, e, i, j, add));
  }
#undef lc
#undef rc
};
```

3.8 Segment Tree 2D

```
// USAGE
ST2D sg(NX, NY); // dimensions 1..NX, 1..NY
sg.upd   (1, 1, NX, x, y, v);
sg.query (1, 1, NX, x1, x2, y1, y2);
struct ST2D {
  #define lc (nd << 1)
  #define rc ((nd << 1) | 1)
  #define lcy (ndy << 1)
  #define rcy ((ndy << 1) | 1)
  int NX, NY;
  vector<vector<long long>> t;
  ST2D(int NX, int NY) : NX(NX), NY(NY), t(4 *
      NX, vector<long long>(4 * NY, 0)) {}
  inline long long combine(long long a, long
     long b) { return max(a, b); }
  inline void pull_y(int nd, int ndy) {
    t[nd][ndy] = combine(t[nd][lcy], t[nd][rcy
     ]);
  }
  inline void pull_x(int nd, int ndy) {
    t[nd][ndy] = combine(t[lc][ndy], t[rc][ndy
     ]);
  }
  void upd_y(int nd, int ndy, int by, int ey,
```

```
    int j, long long v, bool leaf_x) {
  if (by == ey) {
    if (leaf_x) t[nd][ndy] = v;
    else        t[nd][ndy] = combine(t[lc][
      ndy], t[rc][ndy]);
    return;
  }
  int mid = (by + ey) >> 1;
  if (j <= mid) upd_y(nd, lcy, by, mid, j, v
    , leaf_x);
  else          upd_y(nd, rcy, mid+1, ey, j,
     v, leaf_x);
  pull_y(nd, ndy);
}
long long query_y(int nd, int ndy, int by,
    int ey, int y1, int y2) {
  if (y1 > ey || by > y2) return 0;
  if (y1 <= by && ey <= y2) return t[nd][ndy
    ];
  int mid = (by + ey) >> 1;
  return combine(query_y(nd, lcy, by, mid,
    y1, y2),
                 query_y(nd, rcy, mid+1, ey,
     y1, y2));
}
void upd(int nd, int bx, int ex, int i, int
    j, long long v) {
  if (bx == ex) {
    upd_y(nd, 1, 1, NY, j, v, true);
    return;
  }
  int mid = (bx + ex) >> 1;
  if (i <= mid) upd(lc, bx, mid, i, j, v);
  else          upd(rc, mid+1, ex, i, j, v);
  upd_y(nd, 1, 1, NY, j, v, false);  // pull
     x into inner nodes
}
long long query(int nd, int bx, int ex, int
    x1, int x2, int y1, int y2) {
  if (x1 > ex || bx > x2) return 0;
  if (x1 <= bx && ex <= x2) return query_y(
    nd, 1, 1, NY, y1, y2);
  int mid = (bx + ex) >> 1;
  return combine(query(lc, bx, mid, x1, x2,
    y1, y2),
                 query(rc, mid+1, ex, x1, x2
    , y1, y2));
}
};
```

3.9 Segment Tree 2D (Index compression)

```
// USAGE
ST2D seg(n);
// Phase 1: register all (x, y) points that
     will ever be updated
for (auto& [x, y, v] : updates)
    seg.collect(1, 1, n, x, y);
// Phase 2: compress and allocate inner trees
seg.build(1, 1, n);
// Updates and queries
seg.upd   (1, 1, n, x, y, v);
seg.query (1, 1, n, x1, x2, y1, y2);
////////////////////////////////////////
void compress(vector<int>& a) {
    sort(all(a));
    a.erase(unique(all(a)), a.end());
}
int ind(vi& a, int val) {
    return lower_bound(all(a), val) - a.begin
      ();
}
struct ST2D {
  #define lc (nd << 1)
  #define rc ((nd << 1) | 1)
  int n;
  vector<vi> ys;       // compressed y-coords
     per x-node
  vector<vi> t;        // inner seg tree per x
     -node
  ST2D(int n) : n(n), ys(4 * n), t(4 * n) {}
  void collect(int nd, int b, int e, int x,
     int y) {
    ys[nd].pb(y);
    if (b == e) return;
    int mid = (b + e) >> 1;
    if (x <= mid) collect(lc, b, mid, x, y);
    else          collect(rc, mid + 1, e, x, y
      );
  }
  void build(int nd, int b, int e) {
    compress(ys[nd]);
    t[nd].assign(ys[nd].sz() * 2, 0);
    if (b == e) return;
    int mid = (b + e) >> 1;
    build(lc, b, mid);
    build(rc, mid + 1, e);
  }
  inline long long combine(long long a, long
     long b) { return max(a, b); }
  void upd_y(int nd, int y, long long v) {
    int m = ys[nd].sz();
    int i = ind(ys[nd], y) + m;
    t[nd][i] = combine(t[nd][i], v);
    for (i >>= 1; i >= 1; i >>= 1)
      t[nd][i] = combine(t[nd][i*2], t[nd][i
      *2+1]);
  }
  long long query_y(int nd, int y1, int y2) {
    if (y1 > y2) return 0;
    int m = ys[nd].sz();
    int l = ind(ys[nd], y1) + m;
    int r = (int)(upper_bound(all(ys[nd]), y2)
      - ys[nd].begin() - 1) + m;
    ll mx = 0;
    while (l <= r) {
      if (l%2 == 1) mx = combine(mx, t[nd][l
      ++]);
      if (r%2 == 0) mx = combine(mx, t[nd][r
      --]);
      l >>= 1; r >>= 1;
    }
    return mx;
  }
  void upd(int nd, int b, int e, int x, int y,
      long long v) {
    upd_y(nd, y, v);
    if (b == e) return;
    int mid = (b + e) >> 1;
    if (x <= mid) upd(lc, b, mid, x, y, v);
    else          upd(rc, mid + 1, e, x, y, v)
      ;
  }
  long long query(int nd, int b, int e, int x1
      , int x2, int y1, int y2) {
    if (x1 > x2 || y1 > y2) return 0;  //
      empty range (e.g. a[i].f == 1)
    if (x1 > e  || b > x2)  return 0;
    if (x1 <= b && e <= x2)  return query_y(nd
      , y1, y2);
    int mid = (b + e) >> 1;
    return combine(query(lc, b, mid, x1, x2,
      y1, y2),
                   query(rc, mid+1, e, x1, x2,
       y1, y2));
}
};
```

3.10 Merse Sort Tree with Update

```
// 0 based indexing
template <typename T>
struct MST {
  int n;
  vector<ordered_set<pair<T, int>>> tree;
  vector<T> a;
  MST(vector<T>& v) : a(v) {
    n = v.size();
    tree.resize(4 * n);
    build(0, 0, n - 1, a);
  }
  void merge(int u, int l, int r) {
    tree[u] = tree[l];
    for (auto x : tree[r]) tree[u].insert(x);
  }
  void build(int u, int l, int r, vector<T>& a
      ) {
    if (l == r) {
      tree[u].insert({a[l], l});
      return;
    }
    int mid = (l + r) / 2;
    build(2 * u + 1, l, mid, a);
    build(2 * u + 2, mid + 1, r, a);
    merge(u, 2 * u + 1, 2 * u + 2);
  }
  // Returns number of elements smaller than x
       in range l, r
  int query(int l, int r, T x) { return get(0,
      0, n - 1, l, r, x); }
  void update(int i, T x) {
    _update(0, 0, n - 1, i, x);
    a[i] = x;
  }
  int get(int u, int tl, int tr, int l, int r,
      T x) {
    if (tl >= l && tr <= r) return tree[u].
      order_of_key({x, 0});
    if (tl > r || tr < l) return 0;
    int mid = (tl + tr) / 2;
    return get(2 * u + 1, tl, mid, l, r, x) +
           get(2 * u + 2, mid + 1, tr, l, r, x
      );
  }
  void _update(int u, int l, int r, int i, T x
      ) {
    tree[u].erase({a[i], i});
    tree[u].insert({x, i});
    if (l == r) return;
    int mid = (l + r) / 2;
    if (i <= mid)
      _update(2 * u + 1, l, mid, i, x);
    else
      _update(2 * u + 2, mid + 1, r, i, x);
  }
};
```

3.11 Treap

```
const int id = 0, lazy_id = 0;
struct Node {
    int val, sum, lazy = 0;
    int pri, sz = 1;
    Node *l, *r;
    Node(int v, int p) : val(v), sum(v), pri(p
      ) { l = r = nullptr; }
};
mt19937 rnd{(uint32_t)chrono::steady_clock::
     now().time_since_epoch().count()};
int sz(Node* n) { return n ? n->sz : 0; }
int sum(Node* n) { return n ? n->sum : id; }
int combine(int x, int y) { return x + y; }
void apply(Node* n, int v) { // change here
    if (!n) return;
    n->val += v; n->lazy += v; n->sum += v * n
      ->sz;
}
void push(Node* n) {
    if (n->lazy != lazy_id) { apply(n->l, n->
      lazy); apply(n->r, n->lazy); n->lazy =
      0; }
}
void pull(Node* n) {
    n->sz = sz(n->l) + sz(n->r) + 1;
    n->sum = combine(sum(n->l), sum(n->r)) + n
      ->val;
}
pair<Node*, Node*> split(Node* n, int k) { //
     splits into 1..k | k+1..n
    if (!n) return {nullptr, nullptr};
    push(n);
    int sl = sz(n->l);  // n->val for key
    if (sl < k) {
      auto [a, b] = split(n->r, k - sl - 1);
      n->r = a; pull(n); return {n, b};
    } // else for split_by_key
    auto [a, b] = split(n->l, k);
    n->l = b; pull(n); return {a, n};
}
Node* merge(Node* a, Node* b) {
    if (!a) return b;
    if (!b) return a;
    if (a->pri > b->pri) { push(a); a->r =
      merge(a->r, b); pull(a); return a; }
    push(b); b->l = merge(a, b->l); pull(b);
      return b;
}
// Merges two OVERLAPPING sorted treaps O(K
     log(N/K))
Node* meld(Node* u, Node* v) {
    if (!u || !v) return u ? u : v;
    if (u->pri < v->pri) swap(u, v);
    auto [l, r] = split_by_key(v, u->val);
    u->l = meld(u->l, l);
    u->r = meld(u->r, r);
    pull(u);
    return u;
}
void clear(Node* n) {
    if (!n) return;
    clear(n->l); clear(n->r); delete n;
}
Node* insert(Node* root, int val, int pos = 0)
     {
    Node* n = new Node(val, rnd());
    pos = pos ? pos - 1 : sz(root);
    auto [a, b] = split(root, pos);
    root = merge(merge(a, n), b);
    return root;
}
Node* erase(Node* root, int pos) { // 1-based
    auto [a, b] = split(root, pos);
    auto [c, d] = split(a, pos - 1);
    root = merge(c, b); if (d) delete d;
    return root;
}
int get(int i, Node* n) { // 1-based index
    if(sz(n) < i) return -1;
    push(n);
    int id = sz(n->l) + 1;  //
    if (id == i) return n->val;
    return id < i ? get(i - id, n->r) : get(i,
      n->l);
}
int search(int val, Node* n) {  // in sorted
    if(!n) return 0;
```

```
    push(n);
    int id = sz(n->l) + 1;
    int v = n->val;
    // cout << id << ' ' << v << '\n';
    if(v < val) return id+search(val, n->r);
    else return search(val, n->l);
}
```

3.12 Persistent DSU

```
struct DSU {
  vector<vector<pair<int, int>>> par;
  int time = 0; //initial time
  DSU(int n) : par(n + 1, {{-1, 0}}) {}
  bool merge(int u, int v) {
    ++time;
    if ((u = root(u, time)) == (v = root(v,
     time))) return 0;
    if (par[u].back().first > par[v].back().
     first) swap(u, v);
    par[u].push_back({par[u].back().first +
     par[v].back().first, time});
    par[v].push_back({u, time}); //par[u] = u
    return 1;
  }
  bool same(int u, int v, int t) {
    return root(u, t) == root(v, t);
  }
  int root(int u, int t) { //root of u at time
      t
    if (par[u].back().first >= 0 && par[u].
     back().second <= t) return root(par[u].
     back().first, t);
    return u;
  }
  int size(int u, int t) { //size of the
     component of u at time t
    u = root(u, t);
    int l = 0, r = (int) par[u].size() - 1,
     ans = 0;
    while (l <= r) {
      int mid = l + r >> 1;
      if (par[u][mid].second <= t) ans = mid,
     l = mid + 1;
      else r = mid - 1;
    }
    return -par[u][ans].first;
  }
};
```

3.13 MOs with updates

```
struct mos_update {
  ll a[100005];
  ll ans;
  mos_update() { /* ... */ }
  void add(int ind, int end, int L, int R) {
     /* ... */ }
  void del(int ind, int end, int L, int R) {
     /* ... */ }
  void apply(int pos, int val, int L, int R) {
    if (L <= pos && pos <= R) {
      del(pos, -1, L, R); a[pos] = val; add(
     pos, -1, L, R);
    } else {
      a[pos] = val;
    }
  }
  vector<ll> mo(vector<vector<int>>& Q, vector
     <vector<int>>& U, int n) {
    int L = 1, R = 0, T = 0;
    ans = 0;
    int blk = max(1, (int)pow(Q.size(), 0.666)
     );
    vector<int> s(Q.size());
    vector<ll> res(Q.size());
    iota(s.begin(), s.end(), 0);
    sort(s.begin(), s.end(), [&](int i, int j)
      {
      int li = Q[i][0] / blk, lj = Q[j][0] /
     blk;
      if (li != lj) return li < lj;
      int ri = Q[i][1] / blk, rj = Q[j][1] /
     blk;
      if (ri != rj) return (li & 1) ? ri < rj
     : ri > rj;
      return (ri & 1) ? Q[i][2] < Q[j][2] : Q[
     i][2] > Q[j][2];
    });
    for (int qi : s) {
      int ql = Q[qi][0], qr = Q[qi][1], qt = Q
     [qi][2];
      while (T < qt) { apply(U[T][0], U[T][2],
      L, R); T++; }
      while (T > qt) { --T; apply(U[T][0], U[T
     ][1], L, R); }
      while (L > ql) { --L; add(L, 0, L, R); }
      while (R < qr) { ++R; add(R, 1, L, R); }
      while (L < ql) { del(L, 0, L, R); L++; }
      while (R > qr) { del(R, 1, L, R); R--; }
      res[qi] = ans;
    }
    return res;
  }
};
```

3.14 MOs on Trees

```
struct MO_Tree{
   int n, q, B, res;
   struct query { int l, r, id, lca; };
   vector<query> Q;
   vector<int> ans, cnt;
   vector<bool> vis;
   MO_Tree(int n, int q) : n(n), q(q) {
      res = 0;
      B = sqrt(2 * n) + 1;
      ans.resize(q + 1);
      cnt.resize(n + 10);
      vis.assign(n + 1, false);
   }
   void add_query(int u, int v, int id, int
     lca, const vector<int>& st, const vector
     <int>& en) {
      if (st[u] > st[v]) swap(u, v);
      if (lca == u) {
         Q.push_back({st[u], st[v], id, -1});
      } else {
         Q.push_back({en[u], st[v], id, lca});
      }
   }
   void toggle(int node, const vector<int>& a)
      {
      if (vis[node]) {
      } else {
      }
      vis[node] = !vis[node];
   }
   void get_ans(const vector<int>& a, const
     vector<int>& tour) {
      sort(Q.begin(), Q.end(), [&](auto& x,
     auto& y) {
         if (x.l / B == y.l / B) {
            return ((x.l / B) & 1) ? x.r > y.r
      : x.r < y.r;
         }
         return x.l / B < y.l / B;
      });
      int L = 0, R = -1;
      for (auto& q : Q) {
         while (L > q.l) toggle(tour[--L], a);
         while (R < q.r) toggle(tour[++R], a);
         while (L < q.l) toggle(tour[L++], a);
         while (R > q.r) toggle(tour[R--], a);
         if (q.lca != -1) toggle(q.lca, a);
         ans[q.id] = res;
         if (q.lca != -1) toggle(q.lca, a);
      }
   }
};
```

3.15 Hilbert Curve

```
int K = 17; // adjust
inline uint64_t hilbertOrder(int x, int y) {
  uint64_t r = 0;
  for (int s = 1 << (K - 1); s; s >>= 1) {
    bool a = x & s, b = y & s;
    r = r << 2 | (a ? b ? 2 : 1 : b * 3);
    if (!a) {
      if (b) x = (1 << K) - 1 - x, y = (1 << K
     ) - 1 - y;
      swap(x, y);
    }
  }
  return r;
}
```

3.16 Dynamic Connectivity

```
struct ST {
#define lc (n << 1)
#define rc ((n << 1) | 1)
  vector<query> t[maxN*4];
  ST(int n) {}
  void upd(int n, int b, int e, int ql, int qr
     , query& v) {
    if(ql <= b && e <= qr) {
       t[n].push_back(v);
       return;
    }
    int mid = (b + e) >> 1;
    if (ql <= mid) upd(lc, b, mid, ql, qr, v);
    if(qr > mid) upd(rc, mid + 1, e, ql, qr, v
     );
  }
  // can be generalized to any rollback
     structure
  void dfs(int n, int b, int e, int root, int
     eg) {
    int sz = t[n].size(), init = pst.T, croot
     = root;
    for(auto u : t[n]) croot = pst.upd(croot,
     1, eg, u.a, u.b, u.h);
    if(b == e) {  // query ans
       if(!(ask[b][0] == 0 && ask[b][1] == 0)
     )
          ans[b] = pst.query(croot, 1, eg,
     ask[b][0], ask[b][1]);
    } else {
       int mid = (b+e) >> 1;
       dfs(lc, b, mid, croot, eg); dfs(rc,
     mid+1, e, croot, eg);
    }
    pst.T = init; // rollback
  }
  #undef lc
  #undef rc
};
```

3.17 Xor Basis

```
const int LOG_K = 60;  // Adjust as needed
struct XorBasis {
  vector<int64_t> basis;
  int64_t N = 0, tmp = 0;
  void add(int64_t x) {
    N++; tmp |= x;
    for (auto& i : basis) x = min(x, x ^ i);
    if (!x) return;
    for (auto& i : basis) if ((i ^ x) < i) i
     ^= x;
    basis.push_back(x);
    sort(basis.begin(), basis.end());
  }
  int64_t size() { return basis.size(); }
  void clear() { N = 0; tmp = 0; basis.clear()
     ; }
  bool minxor(int64_t x) {
    for (auto& i : basis) x = min(x, x ^ i);
     // for maxxor, return max
    return !x;  // for possible(x), return x;
  }
  int64_t cntxor(int64_t x) {
    if (!possible(x)) return 0;
    return 1LL << (N - size());
  }
  int64_t sumOfAll() { return tmp * (1LL << (N
     - 1)); }
  int64_t kth(int64_t k) {
    int64_t sz = size();
    if (k > (1LL << sz)) return -1;
    k--;
    int64_t ans = 0;
    for (int64_t i = 0; i < sz; i++) if (k >>
     i & 1) ans ^= basis[i];
    return ans;
  }
};
```

3.18 Link Cut Tree

```
struct node {
  int p = 0, c[2] = {0, 0}, sz = 0, ssz = 0,
     vsz = 0, mn_id = 0;
  bool flip = false;
  long long val = INF, sum = 0, lazy = 0,
     subsum = 0, vsum = 0, mn = INF;
  node() {}
  node(int id, long long x) : val(x), sum(x),
     subsum(x), mn(x), mn_id(id), sz(1), ssz
     (1) {}
};
struct LCT {
  vector<node> t;
  LCT(int n, int m = 0) : t(n + m + 1) {
    for (int i = 1; i <= n + m; i++) t[i] =
     node(i, INF);
    t[0] = node();
    t[0].sz = t[0].ssz = t[0].vsz = t[0].sum =
      t[0].subsum = t[0].vsum = 0;
  }
  bool is_root(int x) { return t[t[x].p].c[0]
     != x && t[t[x].p].c[1] != x; }
  int dir(int x) { return t[t[x].p].c[1] == x;
     }
  void flip(int x) {
    if (x) swap(t[x].c[0], t[x].c[1]), t[x].
     flip ^= 1;
  }
  void add(int x, long long v) {
    if (!x) return;
    t[x].val += v; t[x].mn += v; t[x].lazy +=
     v;
    t[x].sum += v * t[x].sz; t[x].subsum += v
     * t[x].ssz; t[x].vsum += v * t[x].vsz;
  }
  void push(int x) {
    if (!x) return;
```

```
    if (t[x].flip) flip(t[x].c[0]), flip(t[x].
      c[1]), t[x].flip = false;
    if (t[x].lazy) add(t[x].c[0], t[x].lazy),
      add(t[x].c[1], t[x].lazy), t[x].lazy =
      0;
  }
  void pull(int x) {
    if (!x) return;
    int l = t[x].c[0], r = t[x].c[1];
    t[x].sz = t[l].sz + t[r].sz + 1;
    t[x].ssz = t[l].ssz + t[r].ssz + t[x].vsz
      + 1;
    t[x].sum = t[l].sum + t[r].sum + t[x].val;
    t[x].subsum = t[l].subsum + t[r].subsum +
      t[x].vsum + t[x].val;
    t[x].mn = t[x].val; t[x].mn_id = x;
    if (t[l].mn < t[x].mn) t[x].mn = t[l].mn,
      t[x].mn_id = t[l].mn_id;
    if (t[r].mn < t[x].mn) t[x].mn = t[r].mn,
      t[x].mn_id = t[r].mn_id;
  }
  void push_all(int x) {
    if (!is_root(x)) push_all(t[x].p);
    push(x);
  }
  void rotate(int x) {
    int y = t[x].p, z = t[y].p, dx = dir(x),
      dy = dir(y), w = t[x].c[!dx];
    if (!is_root(y)) t[z].c[dy] = x;
    t[x].p = z; t[x].c[!dx] = y; t[y].p = x; t
      [y].c[dx] = w;
    if (w) t[w].p = y;
    pull(y); pull(x);
  }
  void splay(int x) {
    push_all(x);
    while (!is_root(x)) {
      int y = t[x].p;
      if (!is_root(y)) rotate(dir(x) == dir(y)
        ? y : x);
      rotate(x);
    }
  }
  int access(int u) {
    int last = 0;
    for (int v = 0; u; v = u, u = t[u].p) {
      splay(u);
      int r = t[u].c[1];
      t[u].vsz += t[r].ssz - t[v].ssz;
      t[u].vsum += t[r].subsum - t[v].subsum;
      t[u].c[1] = v; pull(u); last = u;
    }
    return last;
  }
  void make_root(int u) { access(u); splay(u);
      flip(u); }
  int find_root(int u) {
    access(u); splay(u); push(u);
    while (t[u].c[0]) u = t[u].c[0], push(u);
    splay(u);
    return u;
  }
  bool connected(int u, int v) { return
      find_root(u) == find_root(v); }
  void link(int u, int v) {
    make_root(u);
    if (find_root(v) == u) return;
    access(v); splay(v);
    t[u].p = v; t[v].vsz += t[u].ssz; t[v].
      vsum += t[u].subsum; pull(v);
  }
  void cut(int u, int v) {
    make_root(u); access(v); splay(v);
    if (t[v].c[0] == u && !t[u].c[1]) t[v].c
      [0] = t[u].p = 0, pull(v);
  }
  // --- Operations ---
  void link_edge(int e, int u, int v, long
      long w) { set_val(e, w); link(u, e);
      link(e, v); }
  void cut_edge(int e, int u, int v) { cut(u,
      e); cut(e, v); }
  void set_val(int u, long long x) { access(u)
      ; splay(u); t[u].val = x; pull(u); }
  void add_val(int u, long long x) { access(u)
      ; splay(u); t[u].val += x; pull(u); }
  void add_path(int u, int v, long long x) {
      make_root(u); access(v); splay(v); add(v
      , x); }
  pair<long long, int> query_min(int u, int v)
       { make_root(u); access(v); splay(v);
      return {t[v].mn, t[v].mn_id}; }
  long long query_path_sum(int u, int v) {
      make_root(u); access(v); splay(v);
      return t[v].sum; }
  long long query_subtree_sum(int u, int root
      = 1) { make_root(root); access(u); splay
      (u); return t[u].val + t[u].vsum; }
};
```

3.19 Wavelet Tree

```
struct wavelet_tree {
  int lo, hi;
  wavelet_tree *l = nullptr, *r = nullptr;
  vector<int> b, c;
  wavelet_tree() : lo(0), hi(0) {}
  void init(int *from, int *to, int x, int y)
      {
    lo = x; hi = y;
    if (from >= to) return;
    int mid = lo + (hi - lo) / 2;
    auto f = [mid](int val) { return val <=
      mid; };
    b.reserve(to - from + 1); c.reserve(to -
      from + 1);
    b.push_back(0); c.push_back(0);
    for (auto it = from; it != to; ++it) {
      b.push_back(b.back() + f(*it));
      c.push_back(c.back() + *it);
    }
    if (lo == hi) return;
    auto pivot = stable_partition(from, to, f)
      ;
    (l = new wavelet_tree())->init(from, pivot
      , lo, mid);
    (r = new wavelet_tree())->init(pivot, to,
      mid + 1, hi);
  }
  int kth(int l, int r, int k) {
    if (l > r) return 0;
    if (lo == hi) return lo;
    int inLeft = b[r] - b[l - 1], lb = b[l -
      1], rb = b[r];
    if (k <= inLeft) return this->l->kth(lb +
      1, rb, k);
    return this->r->kth(l - lb, r - rb, k -
      inLeft);
  }
  int LTE(int l, int r, int k) {
    if (l > r || k < lo) return 0;
    if (hi <= k) return r - l + 1;
    int lb = b[l - 1], rb = b[r];
    return l->LTE(lb + 1, rb, k) + r->LTE(l -
      lb, r - rb, k);
  }
  int count(int l, int r, int k) {
    if (l > r || k < lo || k > hi) return 0;
    if (lo == hi) return r - l + 1;
    int lb = b[l - 1], rb = b[r], mid = lo + (
      hi - lo) / 2;
    if (k <= mid) return l->count(lb + 1, rb,
      k);
    return r->count(l - lb, r - rb, k);
  }
  int sum(int l, int r, int k) {
    if (l > r || k < lo) return 0;
    if (hi <= k) return c[r] - c[l - 1];
    int lb = b[l - 1], rb = b[r];
    return l->sum(lb + 1, rb, k) + r->sum(l -
      lb, r - rb, k);
  }
  ~wavelet_tree() { delete l; delete r; }
};
```

3.20 Sparse Table (RMQ)

```
template<typename T>
struct SparseTable {
  const int N = 1e5 + 9;
  vector<vector<T>> t;
  vector<T> a;
  T combine(T x, T y) { return min(x, y); }
  void build(int n) {
    t.assign(n + 1, vector<T>(18));
    for(int i = 1; i <= n; ++i) t[i][0] = a[i
      ];
    for(int k = 1; k < 18; ++k)
      for(int i = 1; i + (1 << k) - 1 <= n; ++
      i)
        t[i][k] = combine(t[i][k-1], t[i + (1
      << (k-1))][k-1]);
  }
  T query(int l, int r) {
    int k = 31 - __builtin_clz(r - l + 1);
    return combine(t[l][k], t[r - (1 << k) +
      1][k]);
  }
};
```

4 Dynamic Programming

4.1 WQS Binary Search

```
//choose at most k disjoint subarrays from
    array 'a' (-1e9 <= a_i <= 1e9) such that
    the sum of elements included in a
    subarray is maximized.
/* returns the maximum sum along with the
    number of subarrays used, if creating a
    subarray penalizes the sum by "lmb" and
    there is no limit to the number of
    subarrays you can create*/
auto solve_lambda = [&](ll lmb) {
 pair<ll, ll> dp[n][2];
 dp[0][0] = {0, 0}; dp[0][1] = {a[0] - lmb,
     1};
 for (int i = 1; i < n; i++) {
  dp[i][0] = max(dp[i - 1][0], dp[i - 1][1]);
  dp[i][1] = max(make_pair(dp[i - 1][0].first
     + a[i] - lmb, dp[i - 1][0].second + 1),
    make_pair(dp[i - 1][1].first + a[i], dp[i
     - 1][1].second));}
 return max(dp[n - 1][0], dp[n - 1][1]);};
ll lo = 0; ll hi = 1e18;
while (lo < hi) {
 ll mid = (lo + hi + 1) / 2;
 solve_lambda(mid).second >= k ? lo = mid : hi
      = mid - 1;}
cout << solve_lambda(lo).first + lo * k <<
     endl;
```

4.2 Divide & Conquer

```
int n, k;
int li = 1, ri = 0, ansrng = 0; // current
    global answer
void update(int ind, bool type, bool inv) {/*
    ...*/}
void shift(int ql, int qr) {
  while(li > ql) { --li; update(li, 0, 1); }
  while(ri < qr) { ++ri; update(ri, 0, 0); }
  while(li < ql) { update(li, 1, 0); li++; }
  while(ri > qr) { update(ri, 1, 1); ri--; }
}
void divconq(int K, int L, int R, int l, int r
    ) {
  if(L > R) return;
  int M = (L+R) >> 1, best = l;
  for(int i = l; i <= min(M, r); i++) {
    shift(i, M);
    int cand = dp[K-1][i-1]+ansrng;
    if(cand <= dp[K][M]) { dp[K][M] = cand;
      best = i; }
  }
  divconq(K, L, M-1, l, best); divconq(K, M+1,
     R, best, r);
}
```

4.3 SOS DP

```
const int B = 20;
int a[1 << B], f[1 << B], g[1 << B];
for (int i = 1; i <= n; i++) {
  cin >> a[i];
  f[a[i]]++;
  g[a[i]]++;
}
// sum over subsets
for (int i = 0; i < B; i++) {
  for (int mask = 0; mask < (1 << B); mask++)
      {
    if ((mask & (1 << i)) != 0) {
      f[mask] += f[mask ^ (1 << i)];
    }
  }
}
// sum over supersets
for (int i = 0; i < B; i++) {
  for (int mask = (1 << B) - 1; mask >= 0;
      mask--) {
    if ((mask & (1 << i)) == 0) g[mask] += g[
      mask ^ (1 << i)];
  }
}
```

4.4 CHT

```
struct CHT { // min query, upper hull (left to
     right)
  struct Line { ll m, c; ll f(ll x) { return m
      * x + c; } };
  deque<Line> hull;
  bool bad(Line l1, Line l2, Line l3) {
    return (__int128)(l3.c - l1.c) * (l1.m -
      l2.m)
       <= (__int128)(l2.c - l1.c) * (l1.m -
      l3.m);
  }
  void add(ll m, ll c) { // lines must be
      added with decreasing slope
    Line l = {m, c};
    if (!hull.empty() && hull.back().m == m) {
      if (c >= hull.back().c) return;
      hull.pop_back();
    }
    while (hull.size() > 1 && bad(hull[hull.
      size()-2], hull.back(), l))
```

```
      hull.pop_back();
    hull.push_back(l);
  }
  ll query(ll x) { // O(log n), arbitrary x
    int l = 0, r = hull.size() - 1;
    while (l < r) {
      int mid = (l + r) / 2;
      if (hull[mid].f(x) > hull[mid+1].f(x)) l
       = mid + 1;
      else r = mid;
    }
    return hull[l].f(x);
  }
  ll query_monotone(ll x) { // O(1) amortized,
      x must be increasing
    while (hull.size() > 1 && hull[0].f(x) >
     hull[1].f(x))
      hull.pop_front();
    return hull[0].f(x);
  }
};
```

4.5 Dynamic CHT

```
struct Line {
  mutable int64_t m, b, p;
  bool operator<(const Line& o) const { return
      m < o.m; }
  bool operator<(int64_t x) const { return p <
      x; }
};
struct CHT : multiset<Line, less<>> {
  bool isMax;
  CHT(bool _isMax = true) { isMax = _isMax; }
  static int64_t div(int64_t a, int64_t b) {
    return a / b - ((a ^ b) < 0 && a % b);
  }
  bool isect(iterator x, iterator y) {
    if (y == end()) {
      x->p = LLONG_MAX;
      return false;
    }
    if (x->m == y->m) {
      x->p = (x->b > y->b ? LLONG_MAX :
     LLONG_MIN);
    } else {
      x->p = div(y->b - x->b, x->m - y->m);
    }
    return x->p >= y->p;
  }
  pair<int64_t, int64_t> convert(int64_t m,
     int64_t b) {
    if (isMax) return {m, b};
    return {-m, -b};
  }
  void add(int64_t m, int64_t b) {
    tie(m, b) = convert(m, b);
    auto z = insert({m, b, 0});
    auto y = z++;
    auto x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y)) {
      isect(x, y = erase(y));
    }
    while ((y = x) != begin() && (--x)->p >= y
     ->p) {
      isect(x, erase(y));
    }
  }
  static int64_t eval(const Line& l, int64_t x
      ) {
    return (int64_t)((__int128)l.m * x + l.b);
  }
  int64_t query(int64_t x) {
    assert(!empty());
    auto l = *lower_bound(x);
    int64_t ans = eval(l, x);
    if (!isMax) ans = -ans;
    return ans;
  }
};
```

5 Numerical

5.1 FFT

Description: fft(a) computes $\hat{f}(k) = \sum_x a[x]\exp(2\pi i \cdot kx/N)$ for all k. N must be a power of 2. Useful for convolution: conv(a, b) = c, where $c[x] = \sum a[i]b[x-i]$. For convolution of complex numbers or more than two vectors: FFT, multiply pointwise, divide by n, reverse(start+1, end), FFT back. Rounding is safe if $(\sum a_i^2 + \sum b_i^2)\log_2 N < 9 \cdot 10^{14}$ (in practice 10^{16}; higher for random inputs). Otherwise, use NTT/FFTMod. Time: $O(N \log N)$ with $N = |A|+|B|$ ($\tilde{1}$s for $N = 2^{22}$)

```
using C = complex<double>;
using vl = vector<long long>;
void fft(vector<C>& a) {
  int n = a.size(), L = 31 - __builtin_clz(n);
  static vector<complex<long double>> R(2, 1);
  static vector<C> rt(2, 1);
  static int kBuilt = 2;
  for (; kBuilt < n; kBuilt *= 2) {
    R.resize(2 * kBuilt); rt.resize(2 * kBuilt
     );
    auto x = polar(1.0L, acos(-1.0L) / kBuilt)
     ;
    for (int i = kBuilt; i < 2 * kBuilt; i++)
      rt[i] = R[i] = (i & 1) ? R[i / 2] * x :
     R[i / 2];
  }
  vector<int> rev(n);
  for (int i = 0; i < n; i++) rev[i] = (rev[i
     / 2] | (i & 1) << L) / 2;
  for (int i = 0; i < n; i++) if (i < rev[i])
     swap(a[i], a[rev[i]]);
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k)
      for (int j = 0; j < k; j++) {
        auto x = (double*)&rt[j + k], y = (
     double*)&a[i + j + k];
        C z(x[0] * y[0] - x[1] * y[1], x[0] *
     y[1] + x[1] * y[0]);
        a[i + j + k] = a[i + j] - z;
        a[i + j] += z;
      }
}
vl conv(const vl& a, const vl& b) {
  if (a.empty() || b.empty()) return {};
  vl res(a.size() + b.size() - 1);
  int L = 32 - __builtin_clz((int)res.size()),
      n = 1 << L;
  vector<C> in(n), out(n);
  copy(a.begin(), a.end(), in.begin());
  for (int i = 0; i < (int)b.size(); i++) in[i
     ].imag(b[i]);
  fft(in);
  for (C& x : in) x *= x;
  for (int i = 0; i < n; i++) out[i] = in[-i &
      (n - 1)] - conj(in[i]);
  fft(out);
  for (int i = 0; i < (int)res.size(); i++)
    res[i] = (long long)round(imag(out[i]) /
     (4 * n));
  return res;
}
```

5.2 FFT With mod M

Description: Higher precision FFT, can be used for convolutions modulo arbitrary integers as long as $N \log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$ (in practice 10^{16} or higher). Inputs must be in $[0, \text{mod})$.
Time: $O(N \log N)$, where $N = |A| + |B|$ (twice as slow as NTT or FFT)

```
template <long long M>
vl convMod(const vl& a, const vl& b) {
  if (a.empty() || b.empty()) return {};
  vl res(a.size() + b.size() - 1);
  int B = 32 - __builtin_clz((int)res.size()),
      n = 1 << B;
  int cut = (int)sqrt((double)M);
  vector<C> fa(n), fb(n), outs(n), outl(n);
  for (int i = 0; i < (int)a.size(); i++) fa[i
     ] = C((int)a[i] / cut, (int)a[i] % cut);
  for (int i = 0; i < (int)b.size(); i++) fb[i
     ] = C((int)b[i] / cut, (int)b[i] % cut);
  fft(fa); fft(fb);
  for (int i = 0; i < n; i++) {
    int j = -i & (n - 1);
    outl[j] = (fa[i] + conj(fa[j])) * fb[i] /
     (2.0 * n);
    outs[j] = (fa[i] - conj(fa[j])) * fb[i] /
     (2.0 * n) / C(0, 1);
  }
  fft(outl); fft(outs);
  for (int i = 0; i < (int)res.size(); i++) {
    long long av = llround(real(outl[i]));
    long long bv = llround(imag(outl[i])) +
     llround(real(outs[i]));
    long long cv = llround(imag(outs[i]));
    res[i] = ((av % M * cut + bv) % M * cut +
     cv) % M;
  }
  return res;
}
```

5.3 NTT

Description: ntt(a) computes $\hat{f}(k) = \sum_x a[x]g^{xk}$ for all k, where $g = \text{root}^{(mod-1)/N}$. N must be a power of 2. Useful for convolution modulo specific nice primes. For manual convolution: NTT the inputs, multiply pointwise, divide by n, reverse(start+1, end), NTT back. Inputs must be in $[0, mod)$.
Time: $O(N \log N)$

```
const int N = 1 << 18;
const int mod = 998244353;
const int root = 3;
int lim, rev[N], w[N], wn[N], inv_lim;
void reduce(int &x) { x = (x + mod) % mod; }
int POW(int x, int y, int ans = 1) {
  for (; y; y >>= 1, x = (long long) x * x %
      mod) if (y & 1) ans = (long long) ans *
      x % mod;
  return ans;
}
void precompute(int len) {
  lim = wn[0] = 1; int s = -1;
  while (lim < len) lim <<= 1, ++s;
  for (int i = 0; i < lim; ++i) rev[i] = rev[i
      >> 1] >> 1 | (i & 1) << s;
  const int g = POW(root, (mod - 1) / lim);
  inv_lim = POW(lim, mod - 2);
  for (int i = 1; i < lim; ++i) wn[i] = (long
     long) wn[i - 1] * g % mod;
}
void ntt(vector<int> &a, int typ) {
  for (int i = 0; i < lim; ++i) if (i < rev[i
     ]) swap(a[i], a[rev[i]]);
  for (int i = 1; i < lim; i <<= 1) {
    for (int j = 0, t = lim / i / 2; j < i; ++
     j) w[j] = wn[j * t];
    for (int j = 0; j < lim; j += i << 1)
      for (int k = 0; k < i; ++k) {
        const int x = a[k + j], y = (long long
     ) a[k + j + i] * w[k] % mod;
        reduce(a[k + j] += y - mod), reduce(a[
     k + j + i] = x - y);
      }
  }
  if (!typ) {
    reverse(a.begin() + 1, a.begin() + lim);
    for (int i = 0; i < lim; ++i) a[i] = (long
      long) a[i] * inv_lim % mod;
  }
}
vector<int> multiply(vector<int> &f, vector<
    int> &g) {
  int n=(int)f.size() + (int)g.size() - 1;
  precompute(n);
  vector<int> a = f, b = g;
  a.resize(lim); b.resize(lim);
  ntt(a, 1), ntt(b, 1);
  for (int i = 0; i < lim; ++i) a[i] = (long
     long) a[i] * b[i] % mod;
  ntt(a, 0);
  return a;
}
```

5.4 FWHT

Description: Transform to a basis with fast convolutions of the form $c[z] = \sum_{z=x\oplus y} a[x] \cdot b[y]$, where $\oplus$ is one of AND, OR, XOR. The size of a must be a power of two. Time: $O(N \log N)$

```
void FST(vector<int>& a, bool inv) {
    for (int n = a.size(), step = 1; step < n;
      step *= 2) {
      for (int i = 0; i < n; i += 2 * step)
          for(int j = i; j < i+step; j++) {
          int &u = a[j], &v = a[j + step];
    tie(u, v) =
              inv ? pair<int,int>(v - u, u)
    : pair<int,int>(v, u + v); // AND
              // inv ? pair<int,int>(v, u -
    v) : pair<int,int>(u + v, u); // OR ///
    include-line
              // pair<int,int>(u + v, u - v)
    ;                   // XOR /// include-
    line
        }
    }
    // if (inv) for (int& x : a) x /= a.size()
     ; // XOR only /// include-line
}
vector<int> conv(vector<int> a, vector<int> b)
     {
    FST(a, 0); FST(b, 0);
    for(int i = 0; i < a.size(); i++) a[i] *=
     b[i];
    FST(a, 1); return a;
}
```

5.5 GCD & LCM Convolution

```
void FMT(vector<int> &a, bool inv) {
  int n = (int)a.size() - 1;
  vector<int> primes;
  vector<bool> is_p(n + 1, true);
  for (int i = 2; i <= n; i++) {
    if (is_p[i]) {
      primes.push_back(i);
      for (int j = 2 * i; j <= n; j += i) is_p
      [j] = false;
```

```
    }
  }
  for (int p : primes) {
    if (!inv) for (int i = n / p; i >= 1; i--)
      a[i] += a[i * p];  // GCD
    else for (int i = 1; i <= n / p; i++) a[i]
      -= a[i * p];
    // if (!inv) for (int i = 1; i <= n / p; i
      ++) a[i * p] += a[i]; /// LCM
    // else      for (int i = n / p; i >= 1; i
      --) a[i * p] -= a[i]; /// LCM
  }
}
vector<int> conv(vector<int> a, vector<int> b)
     {
  FMT(a, 0);
  FMT(b, 0);
  for (int i = 1; i < (int)a.size(); i++) a[i]
      *= b[i];
  FMT(a, 1);
  return a;
}
```

5.6 Berlekamp Messey

```
using mint = modint<mod>;
// Finds the minimum linear recurrence
     transition vector
vector<mint> BerlekampMassey(vector<mint> S) {
  int n = (int)S.size(), L = 0, m = 0;
  vector<mint> C(n), B(n), T;
  C[0] = B[0] = 1;
  mint b = 1;
  for(int i = 0; i < n; i++) {
    ++m; mint d = S[i];
    for(int j = 1; j <= L; j++) d += C[j] * S[
      i - j];
    if (d == 0) continue;
    T = C; mint coef = d * b.inv();
    for(int j = m; j < n; j++) C[j] -= coef *
      B[j - m];
    if (2 * L > i) continue;
    L = i + 1 - L; B = T; b = d; m = 0;
  }
  C.resize(L + 1); C.erase(C.begin());
  for(auto &x: C) x *= -1;
  return C;
}
vector<mint> combine(int n, vector<mint> &a,
     vector<mint> &b, vector<mint> &tr) {
  vector<mint> res(n * 2 + 1, 0);
  for (int i = 0; i < n + 1; i++) {
    for (int j = 0; j < n + 1; j++) res[i + j]
      += a[i] * b[j];
  }
  for (int i = 2 * n; i > n; --i) {
    for (int j = 0; j < n; j++) res[i - 1 - j]
      += res[i] * tr[j];
  }
  res.resize(n + 1);
  return res;
}
// S contains initial terms, k is 0-indexed
mint LinearRecurrence(vector<mint> &S, vector<
     mint> &tr, long long k) {
  int n = S.size(); assert(n == (int)tr.size()
     );
  if (n == 0) return 0;
  if (k < n) return S[k];
  vector<mint> pol(n + 1), e(pol);
  pol[0] = e[1] = 1;
  for (++k; k; k /= 2) {
    if (k % 2) pol = combine(n, pol, e, tr);
    e = combine(n, e, e, tr);
  }
  mint res = 0;
  for (int i = 0; i < n; i++) res += pol[i +
     1] * S[i];
  return res;
}
```

6 Combinatorics

6.1 Mod Template

```
template <unsigned P>
struct modint {
  unsigned v;
  modint() = default;
  template <class T>
  modint(T x) {
    x %= (signed)P, v = x < 0 ? x + P : x;
  }
  modint operator+() const { return *this; }
  modint operator-() const { return modint(0)
     - *this; }
  friend modint power(modint a, long long b) {
    modint res = 1;
    for (; b; b >>= 1, a *= a) if (b & 1) res
      *= a;
    return res;
  }
  modint inv() const { return assert(v), power
     (*this, P - 2); }
  modint& operator+=(const modint& q) {
    (v += q.v) < P || (v -= P);
    return *this;
  }
  modint& operator-=(const modint& q) {
    (v -= q.v) < P || (v += P);
    return *this;
  }
  modint& operator*=(const modint& q) {
    v = 1ull * v * q.v % P;
    return *this;
  }
  modint& operator/=(const modint& q) {
    v = 1ull * v * q.inv().v % P;
    return *this;
  }
  friend modint operator+(modint p, const
     modint& q) { return p += q; }
  friend modint operator-(modint p, const
     modint& q) { return p -= q; }
  friend modint operator*(modint p, const
     modint& q) { return p *= q; }
  friend modint operator/(modint p, const
     modint& q) { return p /= q; }
  friend bool operator==(const modint& p,
     const modint& q) { return p.v == q.v; }
  friend bool operator!=(const modint& p,
     const modint& q) { return p.v != q.v; }
};
```

6.2 Fast Multipoint Evaluation

```
int fact[N], ifact[N];
vector<int> shift(vector<int> &f, int c) { //f
     (x + c)
  int n=(int)f.size();
  precompute(n + n - 1);
  vector<int> a = f; a.resize(lim);
  for (int i = 0; i < n; ++i) a[i] = (long
     long) a[i] * fact[i] % mod;
  reverse(a.begin(), a.begin()+n);
  vector<int> b; b.resize(lim); b[0] = 1;
  for (int i = 1; i < n; ++i) b[i] = (long
     long) b[i - 1] * c % mod;
  for (int i = 0; i < n; ++i) b[i] = (long
     long) b[i] * ifact[i] % mod;
  ntt(a, 1), ntt(b, 1);
  for (int i = 0; i < lim; ++i) a[i] = (long
     long) a[i] * b[i] % mod;
  ntt(a, 0), reverse(a.begin(), a.begin() + n)
     ;
  vector<int> g; g.resize(n);
  for (int i = 0; i < n; ++i) g[i] = (long
     long) a[i] * ifact[i] % mod;
  return g;
}
```

6.3 Stirling 1st with Fixed n

```
// (x+1)*(x+2)*(x+3)...(x+n)
// O(n log n) only for ntt friendly primes
// otherwise use divide and conquer in O(n log
     ^2 n)
vector<int> range_mul(int n) {
  if (n == 0) return vector<int>({1});
  if (n & 1) {
    vector<int> f = range_mul(n - 1);
    f.push_back(0);
    for (int i = (int)f.size()-1; i; --i) f[i]
      = (f[i - 1] + (long long) n * f[i]) %
      mod;
    f[0] = (long long) f[0] * n % mod;
    return f;
  }
  else {
    int n_ = n >> 1;
    vector<int> f = range_mul(n_);
    vector<int> tmp = shift(f, n_);
    f.resize(n_ + 1);
    tmp.resize(n_ + 1);
    return multiply(f, tmp);
  }
}
// returns stirling1st(n, i) for 0 <= i <= n
vector<int> stirling(int n) {
  if (n == 0) return {1};
  vector<int> ans = range_mul(n - 1);
  ans.resize(n + 1);
  for (int i = n - 1; i >= 0; i--) {
    ans[i + 1] = ans[i];
  }
  ans[0] = 0;
  return ans;
}
```

6.4 Stirling 1st with Fixed k

```
// returns stirling1st(i, k) for 0 <= i < n
// O(n log^2 n)
vector<int> stirling(int n, int k) {
  assert(k <= n);
  vector<int> a(n); // sum c(i) / i! x^i, c(i)
     = (i - 1)!
  a[0] = 0;
  for (int i = 1; i < n; i++) {
    a[i] = POW(i, mod - 2);
  }
  auto ans = power(a, k, n);
  ans.resize(n);
  int f = 1;
  for (int i = 1; i <= k; i++) {
    f = 1LL * f * i % mod;
  }
  int cur = 1;
  // sum ans(i) / i! = 1 / k! * a^k
  for (int i = 0; i < n; i++) {
    ans[i] = 1LL * ans[i] * cur % mod;
    cur = 1LL * cur * (i + 1) % mod;
    ans[i] = 1LL * ans[i] * POW(f, mod - 2) %
      mod;
  }
  return ans;
}
```

7 Linear Algebra

7.1 The Matrix Class

```
struct Mat {
  int n, m;
  vector<vector<int>> a;
  Mat() { }
  Mat(int _n, int _m) {n = _n; m = _m; a.
     assign(n, vector<int>(m, 0)); }
  Mat(vector< vector<int> > v) { n = v.size();
      m = n ? v[0].size() : 0; a = v; }
  inline void make_unit() {
    assert(n == m);
    for (int i = 0; i < n; i++)  {
      for (int j = 0; j < n; j++) a[i][j] = i
     == j;
    }
  }
  inline Mat operator + (const Mat &b) {
    assert(n == b.n && m == b.m);
    Mat ans = Mat(n, m);
    for(int i = 0; i < n; i++) {
      for(int j = 0; j < m; j++) {
        ans.a[i][j] = (a[i][j] + b.a[i][j]) %
     mod;
      }
    }
    return ans;
  }
  inline Mat operator - (const Mat &b) {
    assert(n == b.n && m == b.m);
    Mat ans = Mat(n, m);
    for(int i = 0; i < n; i++) {
      for(int j = 0; j < m; j++) {
        ans.a[i][j] = (a[i][j] - b.a[i][j] +
     mod) % mod;
      }
    }
    return ans;
  }
  inline Mat operator * (const Mat &b) {
    assert(m == b.n);
    Mat ans = Mat(n, b.m);
    for(int i = 0; i < n; i++) {
      for(int j = 0; j < b.m; j++) {
        for(int k = 0; k < m; k++) {
          ans.a[i][j] = (ans.a[i][j] + 1LL * a
     [i][k] * b.a[k][j] % mod) % mod;
        }
      }
    }
    return ans;
  }
  inline Mat pow(long long k) {
    assert(n == m);
    Mat ans(n, n), t = a; ans.make_unit();
    while (k) {
      if (k & 1) ans = ans * t;
      t = t * t;
      k >>= 1;
    }
    return ans;
  }
  inline Mat& operator += (const Mat& b) {
     return *this = (*this) + b; }
  inline Mat& operator -= (const Mat& b) {
     return *this = (*this) - b; }
  inline Mat& operator *= (const Mat& b) {
```

```
    return *this = (*this) * b; }
  inline bool operator == (const Mat& b) {
    return a == b.a; }
  inline bool operator != (const Mat& b) {
    return a != b.a; }
};
```

7.2 Gaussian Elimination

Description: Input : Mat a of size n x (m+1), where the last column is the RHS (b vector) i.e. a represents the augmented matrix [A — b] Output: Fills 'ans' with solution vector x such that A*x = b (mod p) Returns -1 → no solution exists Returns 0 → unique solution Returns k → infinitely many solutions with k free variables

Usage:
```
Mat aug(n, m + 1); fill aug.a[i][j] with coefficients, aug.a[i][m] with RHS vector<int> sol; int result = Gauss(aug, sol); if (result == -1) // no solution else if (result == 0) // unique solution in sol else // result = number of free variables
```

```
int Gauss(Mat a, vector<int> &ans) {
  int n = a.n, m = a.m - 1;
  vector<int> pos(m, -1);
  int free_var = 0;
  const long long MODSQ = (long long)mod * mod
      ;
  int rank = 0;
  for (int col = 0, row = 0; col < m && row <
      n; col++) {
    int mx = row;
    for (int k = row; k < n; k++) if (a.a[k][
      col] > a.a[mx][col]) mx = k;
    if (a.a[mx][col] == 0) continue;
    for (int j = col; j <= m; j++) swap(a.a[mx
      ][j], a.a[row][j]);
    pos[col] = row;
    int inv = power(a.a[row][col], mod - 2);
    for (int i = 0; i < n && inv; i++) {
      if (i != row && a.a[i][col]) {
        int x = (1LL * a.a[i][col] * inv) %
      mod;
        for (int j = col; j <= m && x; j++) {
          if (a.a[row][j]) a.a[i][j] = (MODSQ
      + a.a[i][j] - 1LL * a.a[row][j] * x) %
      mod;
        }
      }
    }
    row++; ++rank;
  }
  ans.assign(m, 0);
  for (int i = 0; i < m; i++) {
    if (pos[i] == -1) free_var++;
    else ans[i] = (1LL * a.a[pos[i]][m] *
      power(a.a[pos[i]][i], mod - 2)) % mod;
  }
  for (int i = 0; i < n; i++) {
    long long val = 0;
    for (int j = 0; j < m; j++) val = (val + 1
      LL * ans[j] * a.a[i][j]) % mod;
    if (val != a.a[i][m]) return -1;
  }
  return free_var;
}
```

7.3 Determinant

Description: Determinant of a matrix.

Usage:
```
Mat a(n, n); // fill a.a[i][j] int d = Det(a); // d = det(A) mod p
```

```
int Det(Mat a) {
  assert(a.n == a.m);
  int n = a.n;
  const long long MODSQ = (long long)mod * mod
      ;
  int det = 1;
  for (int col = 0, row = 0; col < n && row <
      n; col++) {
    int mx = row;
    for (int k = row; k < n; k++) if (a.a[k][
      col] > a.a[mx][col]) mx = k;
    if (a.a[mx][col] == 0) { det = 0; continue
      ; }
    for (int j = col; j < n; j++) swap(a.a[mx
      ][j], a.a[row][j]);
    if (row != mx) det = (det == 0 ? 0 : mod -
       det);
    det = 1LL * det * a.a[row][col] % mod;
    int inv = power(a.a[row][col], mod - 2);
    for (int i = 0; i < n && inv; i++) {
      if (i != row && a.a[i][col]) {
        int x = (1LL * a.a[i][col] * inv) %
      mod;
        for (int j = col; j < n && x; j++) {
          if (a.a[row][j]) a.a[i][j] = (MODSQ
      + a.a[i][j] - 1LL * a.a[row][j] * x) %
      mod;
        }
      }
    }
    row++;
  }
  return det;
}
```

7.4 Matrix Inverse

Description: Inverse of a matrix

Usage:
```
Mat a(n, n), inv; // fill a.a[i][j] if (Inv(a, inv)) { /* use inv */ } else { /* singular */ }
```

```
bool Inv(Mat a, Mat &ans) {
  assert(a.n == a.m);
  int n = a.n;
  vector<int> col(n);
  Mat tmp(n, n); tmp.make_unit();
  for (int i = 0; i < n; i++) col[i] = i;
  for (int i = 0; i < n; i++) {
    int r = i, c = i;
    for (int j = i; j < n; j++)
      for (int k = i; k < n; k++)
        if (a.a[j][k]) { r = j; c = k; goto
      found; }
    return false;
    found:
    a.a[i].swap(a.a[r]); tmp.a[i].swap(tmp.a[r
      ]);
    for (int j = 0; j < n; j++) { swap(a.a[j][
      i], a.a[j][c]); swap(tmp.a[j][i], tmp.a[
      j][c]); }
    swap(col[i], col[c]);
    int v = power(a.a[i][i], mod - 2);
    for (int j = i + 1; j < n; j++) {
      int f = 1LL * a.a[j][i] * v % mod;
      a.a[j][i] = 0;
      for (int k = i + 1; k < n; k++) {
        a.a[j][k] -= 1LL * f * a.a[i][k] % mod
      ;
        if (a.a[j][k] < 0) a.a[j][k] += mod;
      }
      for (int k = 0; k < n; k++) {
        tmp.a[j][k] -= 1LL * f * tmp.a[i][k] %
       mod;
        if (tmp.a[j][k] < 0) tmp.a[j][k] +=
      mod;
      }
    }
    for (int j = i + 1; j < n; j++) a.a[i][j]
      = 1LL * a.a[i][j] * v % mod;
    for (int j = 0; j < n; j++) tmp.a[i][j] =
      1LL * tmp.a[i][j] * v % mod;
    a.a[i][i] = 1;
  }
  for (int i = n - 1; i > 0; --i) {
    for (int j = 0; j < i; j++) {
      int v = a.a[j][i];
      for (int k = 0; k < n; k++) {
        tmp.a[j][k] -= 1LL * v * tmp.a[i][k] %
       mod;
        if (tmp.a[j][k] < 0) tmp.a[j][k] +=
      mod;
      }
    }
  }
  ans = Mat(n, n);
  for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
      ans.a[col[i]][col[j]] = tmp.a[i][j];
  return true;
}
```

8 Graph

8.1 Bellmanford

Description: Calculates shortest paths from s in a graph that might have negative edge weights. Unreachable nodes get dist = inf; nodes reachable through negative-weight cycles get dist = -inf. Assumes $V^2 \max|w| < \infty$ to prevent overflow.
Time: $O(VE)$

```
const ll inf = LLONG_MAX;
struct Ed { int a, b, w; ll s() { return a < b
      ? a : -a; }};
struct Node { ll dist = inf; int prev = -1; };
void bellmanFord(vector<Node>& nodes, vector<
    Ed>& eds, int s) {
  nodes[s].dist = 0;
  sort(all(eds), [](Ed a, Ed b) { return a.s()
      < b.s(); });
  int lim = nodes.size() / 2 + 2; // /3+100
      with shuffled vertices
  for(int i = 0; i < lim; i++) for (Ed ed :
      eds) {
    Node cur = nodes[ed.a], &dest = nodes[ed.b
      ];
    if (abs(cur.dist) == inf) continue;
    ll d = cur.dist + ed.w;
    if (d < dest.dist) {
      dest.prev = ed.a;
      dest.dist = (i < lim-1 ? d : -inf);
    }
  }
  for(int i = 0; i < lim; i++) for (Ed e : eds
      ) {
    if (nodes[e.a].dist == -inf)
      nodes[e.b].dist = -inf;
  }
}
```

8.2 Articulation Point & Bridge

```
struct TECC { // 0 indexed
  int n, k;
  vector<vector<pair<int, int>>> g, t;
  vector<bool> used;
  vector<int> comp, ord, low;
  struct edge { int u, v, w; };
  vector<edge> br;
  void dfs(int x, int prv, int &c) {
    used[x] = 1; ord[x] = c++; low[x] = n;
    bool mul = 0;
    for (auto [y, w] : g[x]) {
      if (used[y]) {
        if (y != prv || mul) low[x] = min(low[
      x], ord[y]);
        else mul = 1;
        continue;
      }
      dfs(y, x, c);
      low[x] = min(low[x], low[y]);
    }
  }
  void dfs2(int x, int num) {
    comp[x] = num;
    for (auto [y, w] : g[x]) {
      if (comp[y] != -1) continue;
      if (ord[x] < low[y]) {
        br.push_back({x, y, w});
        k++; dfs2(y, k);
      } else dfs2(y, num);
    }
  }
  TECC(const vector<vector<pair<int, int>>> &g
      ) : g(g), n(g.size()), used(n), comp(n,
      -1), ord(n), low(n), k(0) {
    int c = 0;
    for (int i = 0; i < n; i++) {
      if (used[i]) continue;
      dfs(i, -1, c); dfs2(i, k); k++;
    }
  }
  void build_tree() {
    t.resize(k);
    for (auto e : br) {
      int x = comp[e.u], y = comp[e.v];
      t[x].push_back({y, e.w}); t[y].push_back
      ({x, e.w});
    }
  }
};
```

8.3 Block Cut Tree

```
int T, low[N], dis[N], art[N], sz;
vector<int> g[N], bcc[N], st;
void dfs(int u, int pre = 0) {
  low[u] = dis[u] = ++T;
  st.push_back(u);
  for (auto v : g[u]) {
    if (!dis[v]) {
      dfs(v, u);
      low[u] = min(low[u], low[v]);
      if (low[v] >= dis[u]) {
        sz++; int x;
        do { x = st.back(); st.pop_back(); bcc
      [x].push_back(sz); } while (x ^ v);
        bcc[u].push_back(sz);
      }
    } else if (v != pre) low[u] = min(low[u],
      dis[v]);
  }
}
int dep[N], par[N][20], cnt[N], id[N];
vector<int> bt[N];
void dfs1(int u, int pre = 0) {
  dep[u] = dep[pre] + 1; cnt[u] = cnt[pre] +
      art[u]; par[u][0] = pre;
  for (int k = 1; k <= 18; k++) par[u][k] =
      par[par[u][k - 1]][k - 1];
  for (auto v : bt[u]) if (v != pre) dfs1(v, u
      );
}
int lca(int u, int v) {
  if (dep[u] < dep[v]) swap(u, v);
  for (int k = 18; k >= 0; k--) if (dep[par[u
      ][k]] >= dep[v]) u = par[u][k];
```

```
  if (u == v) return u;
  for (int k = 18; k >= 0; k--) if (par[u][k]
      != par[v][k]) u = par[u][k], v = par[v][
      k];
  return par[u][0];
}
int dist(int u, int v) {
  int lc = lca(u, v);
  return cnt[u] + cnt[v] - 2 * cnt[lc] + art[
      lc];
}
```

8.4 LCA in O(1) sparse table

```
struct LCA {
  int t = 0, k = 0;
  vector<int> tin, tout;
  vector<vector<int>> st;
  vector<vector<int>>& tree;
  LCA(vector<vector<int>>& tree) : tree(tree)
      {
    int n = tree.size();
    tin.assign(n, 0); tout.assign(n, 0);
    k = __lg(2 * n - 1);
    st.assign(n, vector<int>(k + 1));
    dfs(1, 1);
  }
  void dfs(int u, int p) {
    tin[u] = t++; st[u][0] = p;
    for (int i = 1; i <= k; i++) st[u][i] = st
      [st[u][i-1]][i-1];
    for (auto v : tree[u]) if (v != p) dfs(v,
      u);
    tout[u] = t++;
  }
  bool is_anc(int a, int b) { return tin[a] <
      tin[b] && tout[a] > tout[b]; }
  int get(int a, int b) {
    if (a == b) return a;
    if (is_anc(a, b)) return a;
    if (is_anc(b, a)) return b;
    for (int i = k; i >= 0; i--) if (!is_anc(
      st[a][i], b)) a = st[a][i];
    return st[a][0];
  }
  int get_anc(int u, int n) { // n-th ancestor
    for (int i = 0; n > 0; i++)
      if (n & (1 << i)) u = st[u][i], n ^= (1
      << i);
    return u;
  }
};
```

8.5 SCC

Description: Finds strongly connected components in a directed graph. If vertices u, v belong to the same component, u and v is reachable from each other. get_sccs(graph) finds all components. sccs contains the components in reverse topological order. comp[i] holds the component index of node i (a component only has edges to components with lower index). ncomps contains the number of components.
Time: $O(E+V)$

```
vector<int> val, comp, z;
int Time, ncomps;
vector<vector<int>> sccs;
int dfs(int j, vector<vector<int>>& g) {
  int low = val[j] = ++Time, x; z.push_back(j)
      ;
  for (int e : g[j]) if (comp[e] < 0) low =
      min(low, val[e] ?: dfs(e, g));
  if (low == val[j]) {
    sccs.push_back({});
    do {
      x = z.back(); z.pop_back();
      comp[x] = ncomps; sccs.back().push_back(
      x);
    } while (x != j);
    ncomps++;
  }
  return val[j] = low;
}
void get_sccs(vector<vector<int>>& g) {
  int n = g.size();
  val.assign(n, 0); comp.assign(n, -1);
  z.clear(); sccs.clear(); Time = ncomps = 0;
  for (int i = 0; i < n; i++) if (comp[i] < 0)
      dfs(i, g);
}
```

8.6 Kosaraju

```
class Kosaraju{
  public:
  int n;
  vector<vector<int32_t>> g, rg;
  vector<int32_t> comp,order,sz; vector<bool>
      vis;
  void dfs1(int u) {
    vis[u] = true;
    for (int v : g[u]) {
      if(!vis[v]) dfs1(v);
    }
    order.push_back(u);
  }
  void dfs2(int u, int c) {
    comp[u] = c; sz[comp[u]]++;
    for (auto& v : rg[u]) {
      if(comp[v] == -1) dfs2(v, c);
    }
  }
  Kosaraju(int n) : n(n) {
    g.assign(n + 1, {});
    rg.assign(n + 1, {});
    sz.assign(n + 1, 0);
  }
  inline void add_edge(int u, int v) {
    g[u].push_back(v);
    rg[v].push_back(u);
  }
  int build() {
    vis.assign(n + 1, false);
    order.clear();
    for (int i = 1; i <= n; i++) {
      if (!vis[i]) dfs1(i);
    }
    comp.assign(n + 1, -1);
    reverse(order.begin(), order.end());
    int scc = 0;
    for (int u : order) {
      if (comp[u] == -1) {
        dfs2(u, ++scc);
      }
    }
    return scc;
  }
  vector<vector<int32_t>> build_dag(int scc) {
    vector<vector<int32_t>> dag(scc + 1);
    for (int u = 1; u <= n; u++) {
      for (int v : g[u]) {
        if (comp[u] != comp[v]){
          dag[comp[u]].push_back(comp[v]);
        }
      }
    }
    return dag;
  }
};
```

8.7 HLD

```
vector<int> heavy, chain, label, par, sz, dep;
int timer;
void dfsz(int u, int p, vector<vector<int>>&
    tree) {
  sz[u] = 1;
  par[u] = p;
  dep[u] = dep[p] + 1;
  int mx = -1;
  for (int v : tree[u]) {
    if (v == p) continue;
    dfsz(v, u, tree);
    sz[u] += sz[v];
    if (sz[v] > mx) mx = sz[v], heavy[u] = v;
  }
}
void dfshld(int u, int p, vector<vector<int>>&
    tree, int head) {
  label[u] = timer++;
  chain[u] = head;
  if (heavy[u] != -1) dfshld(heavy[u], u, tree
      , head);
  for (int v : tree[u]) {
    if (v == p || v == heavy[u]) continue;
    dfshld(v, u, tree, v);
  }
}
void inithld(vector<vector<int>>& tree) {
  int n = tree.size();
  heavy = vector<int>(n, -1);
  chain = vector<int>(n);
  label = vector<int>(n);
  sz = vector<int>(n);
  par = vector<int>(n);
  dep = vector<int>(n);
  timer = 1;
  dfsz(1, 0, tree);
  dfshld(1, 0, tree, 1);
}
// For path update, you can use the query
    function itself, change
// query with update
ll query(int u, int v, /*, fenwick/segtree/
    anything& ds*/) {
  ll ret = 0;  // init value
  for (; chain[u] != chain[v]; v = par[chain[v
      ]]) {
    if (dep[chain[u]] > dep[chain[v]]) swap(u,
      v);
    // ret ?= ds.query(label[chain[u]], label[
      u]);
  }
  if (dep[u] > dep[v]) swap(u, v);
  // lca is u
  // For EDGES, do label[u]+1
  // ret ?= ds.query(label[u], label[v]);
  return ret;
}
ll querySubtree(int u, /*, fenwick/segtree/
    anything& ds */) {
  // For EDGES, do label[u]+1
  return  // ds.query(label[u], label[u]+sz[u
      ]-1)
}
// Point Updating u : ds.update(label[u]...)
```

8.8 Arpa's Trick

```
const int N = 1e5 + 9;
vector<int> g[N];
int ans[N], col[N], sz[N], cnt[N];
bool big[N];
void dfs(int u, int p) {
    sz[u] = 1;
    for (auto v : g[u]) {
        if (v == p) continue;
        dfs(v, u);
        sz[u] += sz[v];
    }
}
void add(int u, int p, int x) {
    cnt[col[u]] += x;
    for (auto v : g[u]) {
        if (v == p || big[v] == 1) continue;
        add(v, u, x);
    }
}
void dsu(int u, int p, bool keep) {
    int bigchild = -1, mx = -1;
    for (auto v : g[u]) {
        if (v == p) continue;
        if (sz[v] > mx) mx = sz[v], bigchild =
      v;
    }
    for (auto v : g[u]) {
        if (v == p || v == bigchild) continue;
        dsu(v, u, 0);
    }
    if (bigchild != -1) dsu(bigchild, u, 1),
      big[bigchild] = 1;
    add(u, p, 1);
    ans[u] = cnt[u];
    if (bigchild != -1) big[bigchild] = 0;
    if (keep == 0) add(u, p, -1);
}
```

9 String

9.1 Trie

```
#define MAXN 4000005 // > total length of all
    strings combined
int trie[MAXN][26], tsz[MAXN], nodes = 0;
bool end[MAXN];
void insert(const string& s, char fst = 'a') {
  int cur = 0;
  for (char c : s) {
    int ch = c - fst;
    if (!trie[cur][ch]) trie[cur][ch] = ++
      nodes;
    tsz[cur]++;
    cur = trie[cur][ch];
  }
  tsz[cur]++;
  end[cur] = true;
}
bool search(const string& s, char fst = 'a') {
  int cur = 0;
  for (char c : s) {
    int ch = c - fst;
    if (!trie[cur][ch] || !tsz[cur]) return
      false;
    cur = trie[cur][ch];
  }
  return end[cur] && tsz[cur] > 0;
}
bool erase(const string& s, char fst = 'a') {
  if (!search(s, fst)) return false;
  int cur = 0;
  for (char c : s) {
    int ch = c - fst;
    tsz[cur]--;
    cur = trie[cur][ch];
  }
  tsz[cur]--;
  if (!tsz[cur]) end[cur] = false;
  return true;
```

```
}
void dfs(int u = 0, int depth = 0) {
  // custom logic
  for (int i = 0; i < 26; ++i) {
    if (trie[u][i]) {
      dfs(trie[u][i], depth + 1);
    }
  }
}
void deleteall(int root = 0) {
  for (int i = 0; i < 26; ++i) {
    if (trie[root][i]) {
      deleteall(trie[root][i]);
      trie[root][i] = 0;
    }
  }
  tsz[root] = 0;
  end[root] = false;
}
void cleanup() {
  nodes = 0;
  deleteall(0);
}
```

9.2 Bit Trie

```
#define MAXN 4000005 // > 31 * N
int trie[MAXN][2], tsz[MAXN], nodes = 0, bitsz
    = 30;
void insert(int x) {
  int cur = 0;
  for (int i = bitsz; i >= 0; --i) {
    int ch = (x >> i) & 1;
    if (!trie[cur][ch]) trie[cur][ch] = ++
     nodes;
    tsz[cur]++;
    cur = trie[cur][ch];
  }
  tsz[cur]++;
}
void erase(int x) {
  int cur = 0;
  for (int i = bitsz; i >= 0; --i) {
    int ch = (x >> i) & 1;
    if (!tsz[cur] || !trie[cur][ch]) return;
    tsz[cur]--;
    cur = trie[cur][ch];
  }
  tsz[cur]--;
}
int get_kth(int x, int k = 1) {
  int cur = 0, ans = 0;
  for (int i = bitsz; i >= 0; --i) {
    int ch = (x >> i) & 1;
    ch = 1 - ch;  // remove for kth_min()
    if (!trie[cur][ch]) ch = 1 - ch;
    else if (tsz[trie[cur][ch]] < k)
      k -= tsz[trie[cur][ch]], ch = 1 - ch;
    cur = trie[cur][ch];
    ans |= ch << i;
  }
  return ans ^ x;
}
void dfs(int u = 0, int depth = 0) {
  // custom logic
  if (trie[u][0]) dfs(trie[u][0], depth + 1);
  if (trie[u][1]) dfs(trie[u][1], depth + 1);
}
void deleteall(int root) {
  if (trie[root][0]) deleteall(trie[root][0]);
  if (trie[root][1]) deleteall(trie[root][1]);
  trie[root][0] = trie[root][1] = 0;
  tsz[root] = 0;
}
void cleanup() {
  nodes = 0;
  deleteall(0);
}
```

9.3 Z Algorithm

```
vector<int> Z(string& s) {
  int n = s.size();
  vector<int> z(n);
  int l = 0, r = 0;
  for (int i = 0; i < n; ++i) {
    if (i <= r) z[i] = min(z[i - l], r - i +
     1);
    while (i + z[i] < n && s[z[i]] == s[i + z[
     i]]) l = i, r = i + z[i]++;
  }
  return z;
}
```

9.4 Manacher

Description: For each position in a string, computes p[0][i] = half length of longest even palindrome around pos i, p[1][i] = longest odd (half rounded down). Time: O(N)

Usage:

```
auto p = manacher(s); if (is_palindrome(l, r, p))...
```

```
array<vector<int>, 2> manacher(const string& s
    ) {
  int n = s.size();
  array<vector<int>,2> p = {vector<int>(n+1),
      vector<int>(n)};
  for (int z=0; z<2; z++) for (int i=0,l=0,r
      =0; i < n; i++) {
    int t = r-i+!z;
    if (i<r) p[z][i] = min(t, p[z][l+t]);
    int L = i-p[z][i], R = i+p[z][i]-!z;
    while (L>=1 && R+1<n && s[L-1] == s[R+1])
      p[z][i]++, L--, R++;
    if (R>r) l=L, r=R;
  }
  return p;
}
```

9.5 KMP

Description: pi[x] computes the length of the longest prefix of s that ends at x, other than s[0...x] itself (abacaba -¿ 0010123). Can be used to find all occurrences of a string. Time: O(n)

```
vector<int> pi(const string& s) {
  int n = s.size();
  vector<int> p(n);
  for (int i = 1; i < n; ++i) {
    int g = p[i-1];
    while (g && s[i] != s[g]) g = p[g-1];
    p[i] = g + (s[i] == s[g]);
  }
  return p;
}
vector<int> match(const string& s, const
    string& pat) {
  vector<int> p = pi(pat + '\0' + s), res;
  int n = p.size(), ns = s.size(), np = pat.
     size();
  for (int i = n - ns; i < n; ++i)
    if (p[i] == np) res.push_back(i - 2 * np);
  return res;
}
```

9.6 Hashing

```
const ll a1 = 151, b1 = 127657753;
const ll a2 = 239, b2 = 987654319;
ll p1[maxN], p2[maxN];
void pre() {
  p1[0] = p2[0] = 1;
  for (int i = 1; i < maxN; i++) {
    p1[i] = (p1[i - 1] * a1) % b1;
    p2[i] = (p2[i - 1] * a2) % b2;
  }
}
struct Hash {
  vector<ll> h1, h2;
  int n;
  string s;
  Hash(string s) : s(s), n(s.size()) {
    h1.resize(n + 1, 0); h2.resize(n + 1, 0);
    for (int i = 0; i < n; i++) {
      h1[i + 1] = (h1[i] * a1 + s[i]) % b1;
      h2[i + 1] = (h2[i] * a2 + s[i]) % b2;
    }
  }
  array<int,2> get(int l, int r) { // [l, r]
     0-indexed
    ll r1 = (h1[r + 1] - h1[l] * p1[r - l +
     1]) % b1;
    ll r2 = (h2[r + 1] - h2[l] * p2[r - l +
     1]) % b2;
    return {r1 < 0 ? r1 + b1 : r1, r2 < 0 ? r2
      + b2 : r2};
  }
  // implement lcp, lcs, etc yourself
  // if s+s+s... (1 or more times) = t -> true
  bool repeat(string& s_in) {
    int m = s_in.size();
    if (m == 0 || n % m != 0 || n == 0) return
      false;
    Hash temp(s_in);
    if (get(0, m - 1) != temp.get(0, m - 1))
     return false;
    if (n > m && get(0, n - m - 1) != get(m, n
      - 1)) return false;
    return true;
  }
};
```

9.7 Suffix Array

Description: Builds suffix array for a string. sa[i] is the starting index of the suffix which is i'th in the sorted suffix array. The returned vector is of size $n+1$, and sa[0] = n. The lcp array contains longest common prefixes for neighbouring strings in the suffix array: lcp[i] = lcp(sa[i], sa[i-1]), lcp[0] = 0. The input string must not contain any nul chars. Time: O(nlogn)

```
struct SuffixArray {
  vector<int> sa, lcp;
  SuffixArray(string s, int lim = 256) {
    s.push_back(0); int n = s.size(), k = 0, a
     , b;
    vector<int> x(all(s)), y(n), ws(max(n, lim
     ));
    sa = lcp = y; iota(all(sa), 0);
    for (int j = 0, p = 0; p < n; j = max(1, j
     * 2), lim = p) {
      p = j; iota(all(y), n - j);
      for (int i = 0; i < n; i++) if (sa[i] >=
       j) y[p++] = sa[i] - j;
      fill(all(ws), 0);
      for (int i = 0; i < n; i++) ws[x[i]]++;
      for (int i = 1; i < lim; i++) ws[i] +=
      ws[i - 1];
      for (int i = n; i--;) sa[--ws[x[y[i]]]]
      = y[i];
      swap(x, y); p = 1; x[sa[0]] = 0;
      for (int i = 1; i < n; i++) a = sa[i -
      1], b = sa[i], x[b] =
        (y[a] == y[b] && y[a + j] == y[b + j])
       ? p - 1 : p++;
    }
    for (int i = 0, j; i < n - 1; lcp[x[i++]]
     = k)
      for (k && k--, j = sa[x[i] - 1]; s[i + k
     ] == s[j + k]; k++);
  }
};
```

9.8 Suffix Automaton

```
struct SuffixAutomaton {
  struct node { int len, link, fpos; map<char,
      int> nxt; };
  int sz = 1, last = 0;
  vector<node> t;
  vector<int> term;
  vector<long long> dp;
  vector<vector<int>> g;
  SuffixAutomaton(int n) : t(2 * n), term(2 *
     n, 0), dp(2 * n, -1), g(2 * n) {
    t[0].link = -1;
  }
  void extend(char c) {
    int p = last, q, clone, cur;
    if (t[p].nxt.count(c)) {
      q = t[p].nxt[c];
      if (t[q].len == t[p].len + 1) { last = q
     ; return; }
      clone = sz++; t[clone] = t[q]; t[clone].
     len = t[p].len + 1;
      t[q].link = last = clone;
      while (p != -1 && t[p].nxt[c] == q) t[p
     ].nxt[c] = clone, p = t[p].link;
      return;
    }
    cur = last = sz++;
    t[cur].len = t[p].len + 1; t[cur].fpos = t
     [cur].len;
    while (p != -1 && !t[p].nxt.count(c)) t[p
     ].nxt[c] = cur, p = t[p].link;
    if (p == -1) t[cur].link = 0;
    else {
      q = t[p].nxt[c];
      if (t[p].len + 1 == t[q].len) t[cur].
     link = q;
      else {
        clone = sz++; t[clone] = t[q]; t[clone
     ].len = t[p].len + 1;
        while (p != -1 && t[p].nxt[c] == q) t[
     p].nxt[c] = clone, p = t[p].link;
        t[q].link = t[cur].link = clone;
      }
    }
  }
  void build(const string &s) {
    for (char x : s) extend(x), term[last] =
     1;
    for (int i = 1; i < sz; i++) g[t[i].link].
     push_back(i);
  }
  long long cnt(int i) {
    if (dp[i] != -1) return dp[i];
    dp[i] = term[i];
    for (int x : g[i]) dp[i] += cnt(x);
    return dp[i];
  }
};
```

9.9 Aho Corasick

Description: Aho-Corasick automaton, used for multiple pattern matching. Initialize with AhoCorasick ac(patterns); the automaton start node will be at index 0. find(word) returns for each position the index of the longest word that ends there, or -1 if none. findAll(−, word) finds all words (up to $N\sqrt{N}$ many if no duplicate patterns) that start at each position (shortest first). Duplicate patterns are allowed; empty patterns are not. To find the longest words that start at each position, reverse all input. For large alphabets, split each symbol into chunks, with sentinel bits for symbol boundaries. Time: construction takes $O(26N)$, where N = sum of length of patterns. find(x) is $O(N)$, where N = length of x. findAll is $O(NM)$

```
struct AhoCorasick {
  enum {alpha = 26, first = 'A'}; // change
      this!
  struct Node {
    // (nmatches is optional)
    int back, next[alpha], start = -1, end =
      -1, nmatches = 0;
    Node(int v) { memset(next, v, sizeof(next)
      ); }
  };
  vector<Node> N;
  vector<int> backp;
  void insert(string& s, int j) {
    assert(!s.empty());
    int n = 0;
    for (char c : s) {
      int& m = N[n].next[c - first];
      if (m == -1) { n = m = N.size(); N.
      emplace_back(-1); }
      else n = m;
    }
    if (N[n].end == -1) N[n].start = j;
    backp.push_back(N[n].end);
    N[n].end = j;
    N[n].nmatches++;
  }
  AhoCorasick(vector<string>& pat) : N(1, -1)
      {
    for (int i = 0; i < (int)pat.size(); ++i)
      insert(pat[i], i);
    N[0].back = N.size();
    N.emplace_back(0);
    queue<int> q;
    for (q.push(0); !q.empty(); q.pop()) {
      int n = q.front(), prev = N[n].back;
      for (int i = 0; i < alpha; ++i) {
        int &ed = N[n].next[i], y = N[prev].
      next[i];
        if (ed == -1) ed = y;
        else {
          N[ed].back = y;
          (N[ed].end == -1 ? N[ed].end : backp
      [N[ed].start]) = N[y].end;
          N[ed].nmatches += N[y].nmatches;
          q.push(ed);
        }
      }
    }
  }
  vector<int> find(string word) {
    int n = 0;
    vector<int> res; // ll count = 0;
    for (char c : word) {
      n = N[n].next[c - first];
      res.push_back(N[n].end);
      // count += N[n].nmatches;
    }
    return res;
  }
  vector<vector<int>> findAll(vector<string>&
      pat, string word) {
    vector<int> r = find(word);
    int n = word.size();
    vector<vector<int>> res(n);
    for (int i = 0; i < n; ++i) {
      int ind = r[i];
      while (ind != -1) {
        res[i - pat[ind].size() + 1].push_back
      (ind);
        ind = backp[ind];
      }
    }
    return res;
  }
};
```

9.10 Palindromic Tree

```
struct EERTREE {
  struct node {
    int nxt[26], len, st, en, link, cnt, oc;
    int diff, slink; // For O(N log N) / O(N)
      Palindrome DP
  };
  string s;
  vector<node> t;
  int sz, last;
  void init(string _s) {
    s = _s;
    t.assign(s.size() + 9, {});
    sz = last = 2;
    t[1].len = -1; t[1].link = 1;
    t[2].len = 0;  t[2].link = 1;
  }
  int get_link(int u, int i) {
    while (i - 1 - t[u].len < 0 || s[i - 1 - t
      [u].len] != s[i]) u = t[u].link;
    return u;
  }
  bool extend(int i) {
    int c = s[i] - 'a', u = get_link(last, i);
    if (t[u].nxt[c]) return last = t[u].nxt[c
      ], t[last].oc++, 0;
    last = ++sz;
    t[sz].len = t[u].len + 2; t[sz].en = i; t[
      sz].st = i - t[sz].len + 1;
    t[sz].oc = 1; t[u].nxt[c] = sz;
    t[sz].link = (t[sz].len == 1) ? 2 : t[
      get_link(t[u].link, i)].nxt[c];
    t[sz].cnt = 1 + t[t[sz].link].cnt;
    // Series link setup for fast DP
    t[sz].diff = t[sz].len - t[t[sz].link].len
      ;
    t[sz].slink = (t[sz].diff == t[t[sz].link
      ].diff) ? t[t[sz].link].slink : t[sz].
      link;
    return 1;
  }
  void calc_oc() {
    for (int i = sz; i > 2; i--) t[t[i].link].
      oc += t[i].oc;
  }
  // --- Utility Methods ---
  void build() {
    for (int i = 0; i < (int)s.size(); i++)
      extend(i);
    calc_oc();
  }
  int count_distinct() { return sz - 2; }
  long long count_total() { // Requires
      calc_oc() / build() to be called first
    long long tot = 0;
    for (int i = 3; i <= sz; i++) tot += t[i].
      oc;
    return tot;
  }
  string get_palindrome(int u) {
    return s.substr(t[u].st, t[u].len);
  }
};
```

10 Flow and Matching

10.1 Dinic

```
const int64_t inf = 1LL << 60;
struct Dinic {
  struct Edge {
    int to, rev; ll c, oc;
    ll flow() { return max(oc - c, 0LL); } //
      flow on this edge
  };
  vector<int> lvl, ptr, q;
  vector<vector<Edge>> adj;
  Dinic(int n) : lvl(n+1), ptr(n+1), q(n+1),
      adj(n+1) {} // 1-indexed
  void add_edge(int a, int b, ll c, ll rcap =
      0) {
    adj[a].push_back({b, (int)adj[b].size(), c
      , c});
    adj[b].push_back({a, (int)adj[a].size() -
      1, rcap, rcap});
  }
  ll dfs(int v, int t, ll f) {
    if (v == t || !f) return f;
    for (int& i = ptr[v]; i < (int)adj[v].size
      (); i++) {
      Edge& e = adj[v][i];
      if (lvl[e.to] == lvl[v] + 1)
        if (ll p = dfs(e.to, t, min(f, e.c)))
      {
          e.c -= p; adj[e.to][e.rev].c += p;
          return p;
        }
    }
    return 0;
  }
  ll calc(int s, int t) {
    ll flow = 0; q[0] = s;
    for (int L = 0; L < 31; L++) do { //
      capacity scaling; L=30 may be faster for
      random data
      lvl = ptr = vector<int>(adj.size());
      int qi = 0, qe = lvl[s] = 1;
      while (qi < qe && !lvl[t]) {
        int v = q[qi++];
        for (Edge& e : adj[v])
          if (!lvl[e.to] && e.c >> (30 - L))
            q[qe++] = e.to, lvl[e.to] = lvl[v]
      + 1;
      }
      while (ll p = dfs(s, t, LLONG_MAX)) flow
      += p;
    } while (lvl[t]);
    return flow;
  }
  bool leftOfMinCut(int a) { return lvl[a] !=
      0; }
};
```

10.2 Min Cost Max Flow

Description: Build graph with add_edge(). min_cost_flow() returns {max_flow, min_cost}. Costs can be negative/zero. No negative cycles are allowed. flow(edge_id) returns flow through an edge, where id is 0-based index of edges as they were added.
Time: $O(VE + FE \log V)$ where F is max flow.

```
#define INFL (ll) 1e18
struct MCMF {
  struct Edge { int u, v; ll oc, cap, cost; };
  int s, t, n;
  vector<vector<int>> g;
  vector<Edge> edges;
  vector<ll> dist, p; vector<int> par;
  MCMF(int n, int s, int t) : s(s), t(t), n(n)
      { g.resize(n + 1); }
  void add_edge(int u, int v, ll cap, ll cost)
      {
    g[u].pb(edges.size()); g[v].pb(edges.size
      () + 1);
    edges.pb({u, v, cap, cap, cost}); edges.pb
      ({v, u, 0, 0, -cost});
  }
  void setpot() {
    p.assign(n + 1, INFL); p[s] = 0;
    for (int i = 0; i < n; i++) for (auto& e :
      edges)
      if (e.cap && p[e.u] != INFL) p[e.v] =
      min(p[e.v], p[e.u] + e.cost);
  }
  bool dij() {
    dist.assign(n + 1, INFL); dist[s] = 0;
    vector<bool> done(n + 1); par.assign(n +
      1, -1);
    priority_queue<pair<ll,int>> pq; pq.push
      ({0, s});
    while (!pq.empty()) {
      int u = pq.top().second; pq.pop();
      if (done[u]) continue;
      p[u] += dist[u]; done[u] = true;
      for (int id : g[u]) {
        auto& e = edges[id];
        ll d = e.cost + p[u] - p[e.v];
        if (!done[e.v] && e.cap && d < dist[e.
      v])
          par[e.v] = id, dist[e.v] = d, pq.
      push({-d, e.v});
      }
    }
    return dist[t] != INFL;
  }
  pair<ll,ll> min_cost_flow() {
    setpot();
    ll f = 0, mc = 0;
    while (dij()) {
      ll mn = INFL;
      for (int u = t; u != s; u = edges[par[u
      ]].u)
        mn = min(mn, edges[par[u]].cap);
      f += mn; mc += mn * p[t];
      for (int u = t; u != s; u = edges[par[u
      ]].u)
        edges[par[u]].cap -= mn, edges[par[u]
      ^ 1].cap += mn;
    }
    return {f, mc};
  }
  // flow on edge id (0-based)
  ll flow(int id) { return edges[id << 1].oc -
      edges[id << 1].cap; }
};
```

10.3 Bipartite Matching - Hopcroft Karp

```
int hopcroft(int n, int m, vector<vector<int
    >>& g, vector<int>& l) {
  vector<int> r(m + 1), d(n + 1), ptr(n + 1);
  auto dfs = [&](auto&& f, int u) -> bool {
    if (!u) return true;
    for (int& p = ptr[u]; p < (int)g[u].size()
      ; ++p) {
```

```
      int v = g[u][p];
      if (d[r[v]] == d[u] + 1 && f(f, r[v]))
        return 1[u] = v, r[v] = u, true;
    }
    return false;
  };
  int match = 0;
  while (true) {
    d.assign(n + 1, n + 2);
    queue<int> q;
    for (int u = 1; u <= n; ++u)
      if (!1[u]) q.push(u), d[u] = 0;
    while (!q.empty()) {
      int u = q.front(); q.pop();
      for (int v : g[u]) {
        if (d[r[v]] != n + 2) continue;
        if (r[v]) q.push(r[v]);
        d[r[v]] = d[u] + 1;
      }
    }
    if (d[0] == n + 2) return match;
    ptr.assign(n + 1, 0);
    for (int u = 1; u <= n; ++u)
      if (!1[u]) match += dfs(dfs, u);
  }
}
```

11 Geometry

11.1 Geometry Primitives

```
#define EPS 1e-12
#define pi 3.14159265358979311599796346\
854418516159057617187 5L
bool feq(const long double& a, const long
     double& b) {
  long double del = fabsl(a - b);
  if (del <= EPS) return true;
  return del <= EPS * max(fabsl(a), fabsl(b));
}
```

11.2 Point 2D

Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

```
template <class T>
int sgn(T x) {
  return (x > 0) - (x < 0);
  // return (x > EPS) - (x < -EPS); // for
     float
}
template <class T>
struct Point {
  typedef Point P;
  T x, y;
  explicit Point(T x = 0, T y = 0) : x(x), y(y
     ) {}
  bool operator<(P p) const { return tie(x, y)
     < tie(p.x, p.y); }
  bool operator==(P p) const { return tie(x, y
     ) == tie(p.x, p.y); } // Use feq for
     float
  P operator+(P p) const { return P(x + p.x, y
     + p.y); }
  P operator-(P p) const { return P(x - p.x, y
     - p.y); }
  P operator*(T d) const { return P(x * d, y *
     d); }
  P operator/(T d) const { return P(x / d, y /
     d); }
  T dot(P p) const { return x * p.x + y * p.y;
     }
  T cross(P p) const { return x * p.y - y * p.
     x; }
  T cross(P a, P b) const { return (a - *this)
     .cross(b - *this); }
  T dist2() const { return x * x + y * y; }
  T dist2(P p) const { return (*this - p).
     dist2(); }
  long double dist() const { return sqrt((long
      double)dist2()); }
  long double dist(P p) const { return (*this
     - p).dist(); }
  // angle to x-axis in interval [-pi, pi]
  double angle() const { return atan2(y, x); }
  long double angle(P p) const {
    return atan2((long double)this->cross(p),
     (long double)this->dot(p));
  }
  P unit() const { return *this / dist(); }
     // makes dist()=1
  P perp() const { return P(-y, x); }
     // rotates +90 degrees
  P normal() const { return perp().unit(); }
  // returns point rotated 'a' radians ccw
     around the origin
  P rotate(long double a) const {
    return P(x * cos(a) - y * sin(a), x * sin(
      a) + y * cos(a));
  }
  P project(P p) const { return *this * ((
     double)dot(p) / dist2()); }
  double scalar_project(P p) const { return (
     double)dot(p) / dist(); }
  P reflect(P p) const { return project(p) * 2
      - p; }
  // Orientation test: >0 for left turn, <0
     for right turn, 0 if collinear
  T side(P p1, P p2) const { return (p1 - *
     this).cross(p2 - *this); }
  friend ostream& operator<<(ostream& os, P p)
      {
    return os << "(" << p.x << "," << p.y << "
     )";
  }
  // Same direction, length r (returns *this
     if degenerate)
  P truncate(double r) const { double k = dist
     (); return !sgn(k) ? *this : *this * (r
     / k); }
};
```

11.3 Polar Sort

```
bool half(P p) {
  return p.y < 0 || (p.y == 0 && p.x < 0);
}
void polar_sort(vector<P>& v, P o = P(0, 0)) {
  sort(v.begin(), v.end(), [&](P a, P b) {
    P u = a - o, w = b - o;
    return make_tuple(half(u), 0.0, u.dist2())
      < make_tuple(half(w), u.cross(w), w.
     dist2());
  });
}
```

11.4 Line Intersection

Description:

Usage:

```
 auto res = lineInter(s1,e1,s2,e2); if (res.first ==
1) cout << "intersection point at " << res.second <<
endl;
```

```
template <class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2)
      {
  auto d = (e1 - s1).cross(e2 - s2);
  if (d == 0)  // if parallel
    return {-(s1.cross(e1, s2) == 0), P(0, 0)
     };
  auto p = s2.cross(e1, e2), q = s2.cross(e2,
     s1);
  return {1, (s1 * p + e1 * q) / d};
}
```

11.5 Line Equation

```
array<T, 3> line_equation(const Point<T>&b,
     const Point<T>&c){
 array<T, 3>res;
 res[0] = b.y - c.y, res[1] = -(b.x - c.x),
     res[2] = b.x*c.y - b.y*c.x;
 if(res[0] < 0)res[0] = -res[0], res[1] = -res
     [1], res[2] = -res[2];
 else if(res[0] == 0 and res[1] < 0)res[1] = -
     res[1], res[2] = -res[2]; // for double
     use feq(res[0], 0)
 return res;
}
```

11.6 Segment Distance

Description:

Usage:

```
 Point<double> a, b(2,2), p(1,1); bool onSegment =
segDist(a,b,p) < 1e-10;
```

```
typedef Point<double> P;
double segDist(P& s, P& e, P& p) {
  if (s == e) return (p - s).dist();
  auto d = (e - s).dist2(), t = min(d, max(.0,
     (p - s).dot(e - s)));
  return ((p - s) * d - (e - s) * t).dist() /
     d;
}
```

11.7 Segment Intersection

Description:

Usage:

```
vector<P> inter = segInter(s1,e1,s2,e2); if
(sz(inter)==1) cout << "segments intersect at " <<
inter[0] << endl;
```

```
template <class P>
vector<P> segInter(P a, P b, P c, P d) {
  auto oa = c.cross(d, a), ob = c.cross(d, b),
       oc = a.cross(b, c),
       od = a.cross(b, d);
  // Checks if intersection is single non-
     endpoint point.
  if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(
     od) < 0)
    return {(a * ob - b * oa) / (ob - oa)};
  set<P> s;
  if (onSegment(c, d, a)) s.insert(a);
  if (onSegment(c, d, b)) s.insert(b);
  if (onSegment(a, b, c)) s.insert(c);
  if (onSegment(a, b, d)) s.insert(d);
  return {all(s)};
}
```

11.8 Side Of

Description:

Usage:

```
bool left = sideOf(p1,p2,q)==1;
```

```
template <class P>
int sideOf(P& s, P& e, P& p) {
  return sgn(s.cross(e, p));
}
template <class P>
int sideOf(const P& s, const P& e, const P& p,
      double eps) {
  auto a = (e - s).cross(p - s);
  double l = (e - s).dist() * eps;
  return (a > l) - (a < -l);
}
```

11.9 On Segment

Description: Returns true iff p lies on the line segment from s to e.

Usage:

```
\texttt {(segDist(s,e,p)<=epsilon)} instead when using
Point<double>.
```

```
template <class P>
bool onSegment(P s, P e, P p) {
  double d = (e - s).dist();
  if (d < 1e-9) return (p - s).dist() <= 1e-9;
  return abs((e - s).cross(p - s)) / d <= 1e-9
      && (s - p).dot(e - p) <= 1e-9;
}
```

11.10 Linear Transformation

Description: Apply the linear transformation (translation, rotation and scaling) which takes line p0-p1 to line q0-q1 to point r.

```
typedef Point<double> P;
P linearTransformation(const P& p0, const P&
     p1, const P& q0, const P& q1,
                       const P& r) {
  P dp = p1 - p0, dq = q1 - q0, num(dp.cross(
     dq), dp.dot(dq));
  return q0 + P((r - p0).cross(num), (r - p0).
     dot(num)) / dp.dist2();
}
```

11.11 Circle Intersection

```
typedef Point<double> P;
bool circleInter(P a, P b, double r1, double
     r2, pair<P, P>* out) {
  if (a == b) {
    assert(r1 != r2);
    return false;
  }
  P vec = b - a;
  double d2 = vec.dist2(), sum = r1 + r2, dif
     = r1 - r2,
         p = (d2 + r1 * r1 - r2 * r2) / (d2 *
     2), h2 = r1 * r1 - p * p * d2;
  if (sum * sum < d2 || dif * dif > d2) return
      false;
  P mid = a + vec * p, per = vec.perp() * sqrt
     (fmax(0, h2) / d2);
  *out = {mid + per, mid - per};
  return true;
}
```

11.12 Circle Polygon Intersection

```
typedef Point<double> P;
#define arg(p, q) atan2(p.cross(q), p.dot(q))
double circlePoly(P c, double r, vector<P> ps)
      {
  auto tri = [&](P p, P q) {
    auto r2 = r * r / 2;
    P d = q - p;
    auto a = d.dot(p) / d.dist2(), b = (p.
     dist2() - r * r) / d.dist2();
    auto det = a * a - b;
    if (det <= 0) return arg(p, q) * r2;
    auto s = max(0., -a - sqrt(det)), t = min
     (1., -a + sqrt(det));
    if (t < 0 || 1 <= s) return arg(p, q) * r2
     ;
    P u = p + d * s, v = q + d * (t - 1);
```

```
    return arg(p, u) * r2 + u.cross(v) / 2 +
      arg(v, q) * r2;
  };
  auto sum = 0.0;
  rep(i, 0, sz(ps)) sum += tri(ps[i] - c, ps[(
      i + 1) % sz(ps)] - c);
  return sum;
}
```

11.13 Circle Line Intersection

```
template <class P>
vector<P> circleLine(P c, double r, P a, P b)
    {
  P ab = b - a, p = a + ab * (c - a).dot(ab) /
      ab.dist2();
  double s = a.cross(b, c), h2 = r * r - s * s
      / ab.dist2();
  if (h2 < 0) return {};
  if (h2 == 0) return {p};
  P h = ab.unit() * sqrt(h2);
  return {p - h, p + h};
}
```

11.14 Circle Segment Intersection

```
template <class P>
vector<P> circleSegmentIntersection(P c,
    double r, P a, P b) {
  P ab = b - a;
  double l2 = ab.dist2();
  if (l2 < 1e-12) return (a - c).dist() <= r +
      1e-9 ? vector<P>{a} : vector<P>{};
  P p = a + ab * (c - a).dot(ab) / l2;
  double h2 = r * r - pow((c - a).cross(ab),
      2) / l2;
  if (h2 < -1e-9) return {};
  P h = ab.unit() * sqrt(max(0.0, h2));
  vector<P> lpts = abs(h2) < 1e-9 ? vector<P>{
      p} : vector<P>{p - h, p + h}, res;
  for (auto &pt : lpts) {
    if ((a - pt).dot(b - pt) <= 1e-9) res.
      push_back(pt);
  }
  if (res.empty() && (a - c).dist() <= r + 1e
      -9 && (b - c).dist() <= r + 1e-9) {
    res = {a, b};
  }
  return res;
}
```

11.15 Circle Tangents

Description: Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0. $O(n)$

```
template <class P>
vector<pair<P, P>> tangents(P c1, double r1, P
     c2, double r2) {
  P d = c2 - c1;
  double dr = r1 - r2, d2 = d.dist2(), h2 = d2
      - dr * dr;
  if (d2 == 0 || h2 < 0) return {};
  vector<pair<P, P>> out;
  for (double sign : {-1, 1}) {
    P v = (d * dr + d.perp() * sqrt(h2) * sign
      ) / d2;
    out.push_back({c1 + v * r1, c2 + v * r2});
  }
  if (h2 == 0) out.pop_back();
  return out;
}
```

11.16 Minimum Enclosing Circle

Description: Computes the minimum circle that encloses a set of points. $O(n)$

```
bool inside(const P& p, const P& c, double r)
    {
  return (p - c).dist() <= r + 1e-9;
}
// For generalized, udpate calc with gaussian
    elimination
pair<double, P> calc(const vector<P>& R) {
  if (R.empty()) return {0, P()};
  if (R.size() == 1) return {0, R[0]};
  P c = R.size() == 2 ? (R[0] + R[1]) / 2 :
      ccCenter(R[0], R[1], R[2]);
  return {(c - R[0]).dist(), c};
}
pair<double, P> welzl(vector<P>& p, vector<P>&
     R, int n, int D) {
  if (!n || R.size() == D + 1) return calc(R);
  auto res = welzl(p, R, n - 1, D);
  if (!inside(p[n - 1], res.second, res.first)
      ) {
    R.push_back(p[n - 1]);
    res = welzl(p, R, n - 1, D);
    R.pop_back();
  }
  return res;
}
pair<double, P> mec(vector<P>& p, int D = 2) {
  shuffle(p.begin(), p.end(), mt19937(1337));
  vector<P> R;
  return welzl(p, R, p.size(), D);
}
```

11.17 Inside Polygon

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow. $O(n)$

Usage:

```
vector<P> v = {P{4,4}, P{1,2}, P{2,1}}; bool in =
inPolygon(v, P{3, 3}, false);
```

```
bool is_point_on_polygon(vector<P> &p, P& z) {
    int n = p.size();
    for (int i = 0; i < n; i++) {
      if (onSegment(p[i], p[(i + 1) % n], z))
      return 1;
    }
    return 0;
}
int winding_number(vector<P>& p, P& z) { // O(
    n)
  if (is_point_on_polygon(p, z)) return 1e9;
  int n = p.size(), ans = 0;
  for (int i = 0; i < n; ++i) {
    int j = (i + 1) % n;
    bool below = p[i].y < z.y;
    if (below != (p[j].y < z.y)) {
      auto orient = z.cross(p[j], p[i]);
      if (abs(orient) < 1e-9) return 0;
      if (below == (orient > 0)) ans += below
      ? 1 : -1;
    }
  }
  return ans;
}
// -1 if strictly inside, 0 if on the polygon,
     1 if strictly outside
int is_point_in_polygon(vector<P>& p,  P& z) {
      // O(n)
  int k = winding_number(p, z);
  return k == 1e9 ? 0 : k == 0 ? 1 : -1;
}
```

11.18 Polygon Area

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

```
template <class T>
T polygonArea2(vector<Point<T>>& v) {
  T a = v.back().cross(v[0]);
  for(int i = 0; i < (int) v.size() - 1; ++i)
    a += v[i].cross(v[i + 1]);
  return a;
}
```

11.19 Polygon Center

Description: Returns the center of mass for a polygon. Time: $O(n)$

```
typedef Point<double> P;
P polygonCenter(const vector<P>& v) {
  P res(0, 0);
  double A = 0;
  for (int i = 0, j = sz(v) - 1; i < sz(v); j
      = i++) {
    res = res + (v[i] + v[j]) * v[j].cross(v[i
      ]);
    A += v[j].cross(v[i]);
  }
  return res / A / 3;
}
```

11.20 Polygon Cut

```
vector<P> polygonCut(const vector<P>& poly, P
    s, P e) {
  vector<P> res;
  for (int i = 0; i < (int)poly.size(); ++i) {
    P cur = poly[i], prev = i ? poly[i - 1] :
      poly.back();
    auto a = s.cross(e, cur), b = s.cross(e,
      prev);
    if ((a < 0) != (b < 0)) res.push_back(cur
      + (prev - cur) * (a / (a - b)));
    if (a < 0) res.push_back(cur);
  }
  return res;
}
```

11.21 Polygon Union

```
double rat(P a, P b) { return sgn(b.x) ? a.x /
     b.x : a.y / b.y; }
double polyUnion(vector<vector<P>>& poly) {
  double ret = 0;
  for (int i = 0; i < (int)poly.size(); ++i)
    for (int v = 0; v < (int)poly[i].size();
      ++v) {
    P A = poly[i][v], B = poly[i][(v + 1) % (
      int)poly[i].size()];
    vector<pair<double, int>> segs = {{0, 0},
      {1, 0}};
    for (int j = 0; j < (int)poly.size(); ++j)
      if (i != j) {
      for (int u = 0; u < (int)poly[j].size();
      ++u) {
        P C = poly[j][u], D = poly[j][(u + 1)
      % (int)poly[j].size()];
        int sc = sideOf(A, B, C), sd = sideOf(
      A, B, D);
        if (sc != sd) {
          double sa = C.cross(D, A), sb = C.
      cross(D, B);
          if (min(sc, sd) < 0) segs.
      emplace_back(sa / (sa - sb), sgn(sc - sd
      ));
        } else if (!sc && !sd && j < i && sgn
      ((B - A).dot(D - C)) > 0) {
          segs.emplace_back(rat(C - A, B - A),
       1);
          segs.emplace_back(rat(D - A, B - A),
       -1);
        }
      }
    }
    sort(segs.begin(), segs.end());
    for (auto& s : segs) s.first = min(max(s.
      first, 0.0), 1.0);
    double sum = 0;
    int cnt = segs[0].second;
    for (int j = 1; j < (int)segs.size(); ++j)
      {
      if (!cnt) sum += segs[j].first - segs[j
      - 1].first;
      cnt += segs[j].second;
    }
    ret += A.cross(B) * sum;
  }
  return ret / 2;
}
```

11.22 Extreme Vertex

```
// id of the vertex having maximum dot product
      with z
// polygon must be convex
// top - upper right vertex
// for minimum dot product negate z and return
      -p[id].dot(z)
int extreme_vertex(const vector<P>& p, const P
     & z, const int top) { // O(log n)
  int n = p.size();
  if (n == 1) return 0;
  auto ans = p[0].dot(z);
  int id = 0;
  if (p[top].dot(z) > ans) ans = p[top].dot(z)
      , id = top;
  int l = 1, r = top - 1;
  while (l < r) {
    int mid = (l + r) / 2;
    if (p[mid + 1].dot(z) >= p[mid].dot(z)) l
      = mid + 1;
    else r = mid;
  }
  if (p[l].dot(z) > ans) ans = p[l].dot(z), id
      = l;
  l = top + 1, r = n - 1;
  while (l < r) {
    int mid = (l + r) / 2;
    if (p[(mid + 1) % n].dot(z) >= p[mid].dot(
      z)) l = mid + 1;
    else r = mid;
  }
  l %= n;
  if (p[l].dot(z) > ans) ans = p[l].dot(z), id
      = l;
  return id;
}
```

11.23 Manhattan MST

```
vector<array<int, 3>> manhattanMST(vector<P>
     ps) {
  vector<int> id(ps.size());
  iota(id.begin(), id.end(), 0);
  vector<array<int, 3>> edges;
```

```
  for (int k = 0; k < 4; ++k) {
    sort(id.begin(), id.end(),
         [&](int i, int j) { return (ps[i] -
      ps[j]).x < (ps[j] - ps[i]).y; });
    map<int, int> sweep;
    for (int i : id) {
      for (auto it = sweep.lower_bound(-ps[i].
     y); it != sweep.end();
           sweep.erase(it++)) {
        int j = it->second;
        P d = ps[i] - ps[j];
        if (d.y > d.x) break;
        edges.push_back({d.y + d.x, i, j});
      }
      sweep[-ps[i].y] = i;
    }
    for (P& p : ps)
      if (k & 1)
        p.x = -p.x;
      else
        swap(p.x, p.y);
  }
  return edges;
}
```

11.24 Convex Hull

Description: Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull. $O(nlogn)$

```
typedef Point<ll> P;
vector<P> convexHull(vector<P> pts) {
  int n = pts.size();
  if (n <= 1) return pts;
  sort(all(pts));
  vector<P> h(n + 1);
  int s = 0, t = 0;
  for (int it = 2; it--; s = --t, reverse(all(
      pts)))
    for (P p : pts) {
      while (t >= s + 2 && h[t - 2].cross(h[t
     - 1], p) <= 0) t--;
      h[t++] = p;
    }
  return {h.begin(), h.begin() + t - (t == 2
      && h[0] == h[1])};
}
```

11.25 Hull Diameter

Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points). $O(n)$

```
typedef Point<ll> P;
array<P, 2> hullDiameter(vector<P> S) {
  int n = sz(S), j = n < 2 ? 0 : 1;
  pair<ll, array<P, 2>> res({0, {S[0], S[0]}})
      ;
  rep(i, 0, j) for (;; j = (j + 1) % n) {
    res = max(res, {(S[i] - S[j]).dist2(), {S[
      i], S[j]}});
    if ((S[(j + 1) % n] - S[j]).cross(S[i + 1]
      - S[i]) >= 0) break;
  }
  return res.second;
}
```

11.26 Point Inside Hull

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included. $O(logN)$

```
typedef Point<ll> P;
bool inHull(vector<P>& l, P p, bool strict =
    true) {
  int n = l.size(), a = 1, b = n - 1, r = !
      strict;
  if (n < 3) return r && onSegment(l[0], l.
      back(), p);
  if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b)
      ;
  if (sideOf(l[0], l[a], p) >= r || sideOf(l
      [0], l[b], p) <= -r) return false;
  while (abs(a - b) > 1) {
    int c = (a + b) / 2;
    (sideOf(l[0], l[c], p) > 0 ? b : a) = c;
  }
  return sgn(l[a].cross(l[b], p)) < r;
}
```

11.27 Line Hull Intersection

Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon:

- $(-1, -1)$ if no collision,
- $(i, -1)$ if touching the corner i,
- (i, i) if along side $(i, i+1)$,
- (i, j) if crossing sides $(i, i+1)$ and $(j, j+1)$.

In the last case, if a corner i is crossed, this is treated as happening on side $(i, i+1)$. The points are returned in the same order as the line hits the polygon. extrVertex returns the point of a hull with the max projection onto a line. $O(logN)$

```
#define cmp(i, j) sgn(dir.perp().cross(poly[(i
    ) % n] - poly[(j) % n]))
#define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i
    - 1 + n) < 0
template <class P>
int extrVertex(vector<P>& poly, P dir) {
  int n = sz(poly), lo = 0, hi = n;
  if (extr(0)) return 0;
  while (lo + 1 < hi) {
    int m = (lo + hi) / 2;
    if (extr(m)) return m;
    int ls = cmp(lo + 1, lo), ms = cmp(m + 1,
     m);
    (ls < ms || (ls == ms && ls == cmp(lo, m))
      ? hi : lo) = m;
  }
  return lo;
}
#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vector<P>&
    poly) {
  int endA = extrVertex(poly, (a - b).perp());
  int endB = extrVertex(poly, (b - a).perp());
  if (cmpL(endA) < 0 || cmpL(endB) > 0) return
      {-1, -1};
  array<int, 2> res;
  rep(i, 0, 2) {
    int lo = endB, hi = endA, n = sz(poly);
    while ((lo + 1) % n != hi) {
      int m = ((lo + hi + (lo < hi ? 0 : n)) /
      2) % n;
      (cmpL(m) == cmpL(endB) ? lo : hi) = m;
    }
    res[i] = (lo + !cmpL(hi)) % n;
    swap(endA, endB);
  }
  if (res[0] == res[1]) return {res[0], -1};
  if (!cmpL(res[0]) && !cmpL(res[1]))
    switch ((res[0] - res[1] + sz(poly) + 1) %
      sz(poly)) {
      case 0:
        return {res[0], res[0]};
      case 2:
        return {res[1], res[1]};
    }
  return res;
}
```

11.28 Half Plane Intersection

```
typedef Point<double> P;
using Line = pair<P, P>;
bool bad(Line a, Line b, Line c) {
  auto res = lineInter(a.first, a.second, b.
      first, b.second);
  return res.first > 0 && sideOf(c.first, c.
      second, res.second, EPS) < 0;
}
vector<P> halfPlaneIntersection(vector<Line>&
    L) {
  auto ang = [](Line l) { return (l.second-l.
      first).angle(); };
  sort(all(L), [&](Line a,Line b) { return ang
      (a)<ang(b); });
  vector<Line> lines;
  for (int i = 0; i < L.size(); i++) {
    if (i && abs(ang(L[i])-ang(L[i-1]))<EPS) {
      if (sideOf(L[i].first,L[i].second,lines.
      back().first,EPS) < 0) {
        lines.pop_back(); lines.push_back(L[i
      ]);
      }
    } else lines.push_back(L[i]);
  }
  int n = lines.size(), qf = 0, qb = 0;
  vector<Line> q(n);
  for (int i = 0; i < n; i++) {
    while (qb-qf >= 2 && bad(q[qb-1], q[qb-2],
      lines[i])) qb--;
    while (qb-qf >= 2 && bad(q[qf], q[qf+1],
      lines[i])) qf++;
    q[qb++] = lines[i];
  }
  while (qb-qf >= 3 && bad(q[qb-1], q[qb-2], q
      [qf])) qb--;
  while (qb-qf >= 3 && bad(q[qf], q[qf+1], q[
      qb-1])) qf++;
  if(qb-qf < 3) return {};
  vector<P> poly;
  for (int i = qf; i < qb; i++) {
    int nxt = (i+1 == qb) ? qf : i + 1;
    poly.push_back(lineInter(q[i].first, q[i].
      second, q[nxt].first, q[nxt].second).
      second);
  }
  return poly;
}
```

11.29 Minkowski Sum & Dif

```
using P = Point<ld>;
void reorder(vector<P>& poly) {
  int pos = 0;
  for (int i = 1; i < (int)poly.size(); ++i)
    if (poly[i] < poly[pos]) pos = i;
  rotate(poly.begin(), poly.begin() + pos,
      poly.end());
}
vector<P> minkowski(vector<P> a, vector<P> b)
    {
  reorder(a), reorder(b);
  int n = a.size(), m = b.size(), ai = 0, bi =
      0;
  a.push_back(a[0]), a.push_back(a[1]);
  b.push_back(b[0]), b.push_back(b[1]);
  vector<P> c;
  while (ai < n || bi < m) {
    c.push_back(a[ai] + b[bi]);
    if (ai < n && bi < m) {
      auto ori = (a[ai + 1] - a[ai]).cross(b[
      bi + 1] - b[bi]);
      if (ori > 1e-9) ai++;
      else if (ori < -1e-9) bi++;
      else ai++, bi++;
    } else if (ai < n) ai++;
    else bi++;
  }
  return c;
}
vector<P> minkowski_dif(vector<P> a, vector<P>
    b) {
  for (auto& p : b) p.x = -p.x, p.y = -p.y;
  return minkowski(a, b);
}
```

11.30 Maximum Inscribed Circle

```
ld maxInscribedCircle(vector<P>& poly) {
  ld lo = 0, hi = 1e6, ans = 0, L = 1e9;
  int n = poly.size();
  while (hi - lo >= 1e-12) {
    ld mid = lo + (hi - lo) / 2.0;
    vector<Line> l = {{{-L, -L}, {L, -L}}, {{L
      , -L}, {L, L}}, {{L, L}, {-L, L}}, {{-L,
       L}, {-L, -L}}};
    for (int i = 0; i < n; ++i) {
      auto a = poly[i], b = poly[(i + 1) % n];
      auto c = (b - a).unit().perp();
      l.push_back({a + c * mid, b + c * mid});
    }
    auto cur = halfPlaneIntersection(l);
    if (!cur.empty()) ans = lo = mid;
    else hi = mid;
  }
  return ans;
}
```

11.31 Rotating Calipers

```
// Outer loop advances edge (i -> i+1), inner
     loop advances antipodal pointer j
for (int i = 0, j = 1; i < n; ++i) {
  while ((p[(i + 1) % n] - p[i]).cross(p[(j +
      1) % n] - p[j]) > 0) {
    j = (j + 1) % n;
  }
  // Process antipodal pair: p[i] (or edge i
     -> i+1) and p[j]
}
```

11.32 Closest Pair

Description: Finds the closest pair of points. O(nlogn)

```
typedef Point<ll> P;
pair<P, P> closest(vector<P> v) {
  assert(sz(v) > 1);
  set<P> S;
  sort(all(v), [](P a, P b) { return a.y < b.y
      ; });
  pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P
      ()}};
```

```
  int j = 0;
  for (P p : v) {
    P d{1 + (ll)sqrt(ret.first), 0};
    while (v[j].y <= p.y - d.x) S.erase(v[j
      ++]);
    auto lo = S.lower_bound(p - d), hi = S.
      upper_bound(p + d);
    for (; lo != hi; ++lo) ret = min(ret, {(*
      lo - p).dist2(), {*lo, p}});
    S.insert(p);
  }
  return ret.second;
}
```

[Picks theorem states that $A = I + \frac{B}{2} - 1$, Area A can be calculated using polyArea().]

12 Geometry 3D

12.1 Point 3D

```
using P2 = Point<double>;
struct P3 {
  double x, y, z;
  explicit P3(double x = 0, double y = 0,
      double z = 0) : x(x), y(y), z(z) {}
  bool operator<(P3 p) const { return tie(x, y
      , z) < tie(p.x, p.y, p.z); }
  bool operator==(P3 p) const { return tie(x,
      y, z) == tie(p.x, p.y, p.z); }
  P3 operator+(P3 p) const { return P3(x + p.x
      , y + p.y, z + p.z); }
  P3 operator-(P3 p) const { return P3(x - p.x
      , y - p.y, z - p.z); }
  P3 operator*(double d) const { return P3(x *
       d, y * d, z * d); }
  P3 operator/(double d) const { return P3(x /
       d, y / d, z / d); }
  double dot(P3 p) const { return x * p.x + y
      * p.y + z * p.z; }
  P3 cross(P3 p) const { return P3(y * p.z - z
       * p.y, z * p.x - x * p.z, x * p.y - y *
       p.x); }
  double dist2() const { return x * x + y * y
      + z * z; }
  double dist2(P3 p) const { return (*this - p
      ).dist2(); }
  double dist() const { return sqrt(dist2());
      }
  double dist(P3 p) const { return (*this - p)
      .dist(); }
  P3 unit() const { return *this / dist(); }
  double angle(P3 p) const { return atan2(
      cross(p).dist(), dot(p)); }
  // (this*a).b
  double triple(P3 a, P3 b) const { return
      cross(a).dot(b); }
  // Smallest angle between this and p,
      ignoring direction, in [0, PI/2]
  double small_angle(P3 p) const { return acos
      (min(fabs(dot(p)) / dist() / p.dist(),
      1.0)); }
  // Same direction, length r
  P3 truncate(double r) const { double k =
      dist(); return !sgn(k) ? *this : *this *
       (r / k); }
  // Rotate vector around unit axis 'u' by
      angle 'a' (Rodrigues' rotation formula)
  P3 rotate(P3 u, double a) const {
    u = u.unit();
    return *this * cos(a) + u.cross(*this) *
      sin(a) + u * u.dot(*this) * (1 - cos(a))
      ;
  }
  // Returns an arbitrary orthogonal unit
      vector to this
  P3 perp() const { return fabs(x) > EPS ||
      fabs(y) > EPS ? P3(-y, x, 0).unit() : P3
      (0, -z, y).unit(); }
};
 // orient' is >0 if s lies on lies on the
      same side of plane pqr as (q-p) x (r-p),
      <0 opposite, 0 on it.
 // orient_by_normal: 2D-like orientation of p
      ->q->r on the plane with normal n.
double orient(P3 p, P3 q, P3 r, P3 s) { return
       (q - p).cross(r - p).dot(s - p); }
double orient_by_normal(P3 p, P3 q, P3 r, P3 n
      ) { return (q - p).cross(r - p).dot(n);
      }
```

12.2 Plane

```
// 3D Plane: normal n, offset d (n.dot(p) = d)
struct Plane {
  P3 n; double d;
  Plane(P3 n, double d) : n(n), d(d) {}
  Plane(P3 n, P3 p) : n(n), d(n.dot(p)) {}
  Plane(P3 p, P3 q, P3 r) : Plane((q - p).
      cross(r - p), p) {}
  double side(P3 p) const { return n.dot(p) -
      d; }
  double dist(P3 p) const { return fabs(side(p
      )) / n.dist(); }
  P3 proj(P3 p) const { return p - n * (side(p
      ) / n.dist2()); }
  P3 reflect(P3 p) const { return p - n * (2 *
       side(p) / n.dist2()); }
  // [-PI, PI]
  double angle(Plane p) const { return n.angle
      (p.n); }
  // [0, PI/2]
  double small_angle(Plane p) const { return n
      .small_angle(p.n); }
  bool is_para(Plane p) const { return n.cross
      (p.n).dist() < EPS; }
  bool is_perp(Plane p) const { return sgn(n.
      dot(p.n)) == 0; }
  // Translate the plane by vector t
  Plane translate(P3 t) const { return {n, d +
       n.dot(t)}; }
  // Shift the plane along n by distance dis
  Plane shift_up(double dis) const { return {n
      , d + dis * n.dist()}; }
  // Two distinct points lying on the plane (n
       must be non-zero)
  pair<P3, P3> two_pts() const {
    int sx = sgn(n.x), sy = sgn(n.y), sz = sgn
      (n.z);
    assert(sx || sy || sz);
    if (!sx && !sy) return {P3(1, 0, d / n.z),
      P3(0, 1, d / n.z)};
    if (!sy && !sz) return {P3(d / n.x, 1, 0),
      P3(d / n.x, 0, 1)};
    if (!sz && !sx) return {P3(1, d / n.y, 0),
      P3(0, d / n.y, 1)};
    if (!sx) return {P3(1, d / n.y, 0), P3(0,
      0, d / n.z)};
    if (!sy) return {P3(0, 1, d / n.z), P3(d /
      n.x, 0, 0)};
    if (!sz) return {P3(d / n.x, 0, 1), P3(0,
      d / n.y, 0)};
    if (sgn(d)) return {P3(d / n.x, 0, 0), P3
      (0, d / n.y, 0)};
    return {P3(n.y, -n.x, 0), P3(-n.y, n.x, 0)
      };
  }
};
// Intersection line of two planes 3-point
    pairs.
pair<P3, P3> plane_inter(P3 a1, P3 b1, P3 c1,
    P3 a2, P3 b2, P3 c2) {
  auto n1 = (b1 - a1).cross(c1 - a1), n2 = (b2
     - a2).cross(c2 - a2);
  auto d = n1.cross(n2);
  if (d.dist2() == 0) return {P3(), P3()};
  auto o = (n2 * n1.dot(a1) - n1 * n2.dot(a2))
    .cross(d) / d.dist2();
  return {o, o + d};
}
```

12.3 Line 3D

```
// 3d line parametric form: 'o + k * d'.
struct Line3D {
  P3 d, o;
  Line3D() {}
  Line3D(P3 p, P3 q) : d(q - p), o(p) {}
  Line3D(Plane p1, Plane p2) {
    d = p1.n.cross(p2.n);
    o = (p2.n * p1.d - p1.n * p2.d).cross(d) /
      d.dist2();
  }
  double dist2(P3 p) const { return d.cross(p
      - o).dist2() / d.dist2(); }
  double dist(P3 p) const { return sqrt(dist2(
      p)); }
  P3 proj(P3 p) const { return o + d * (d.dot(
      p - o) / d.dist2()); }
  P3 inter(Plane p) const { return o - d * (p.
      side(o) / d.dot(p.n)); }
  // Orthogonal reflection of point p across
      the line
  P3 reflect(P3 p) const { return proj(p) * 2
      - p; }
  // Compare points by their projection on the
       line (use as a sort comparator)
  bool cmp_proj(P3 p, P3 q) const { return d.
      dot(p) < d.dot(q); }
  // Small angle between the direction vectors
       of two lines
  double angle(Line3D l) const { return d.
      small_angle(l.d); }
  bool is_para(Line3D l) const { return d.
      cross(l.d).dist() < EPS; }
  bool is_perp(Line3D l) const { return sgn(d.
      dot(l.d)) == 0; }
  // Distance from 3D segment [p, q] to point
      x
  static double seg_dist(P3 p, P3 q, P3 x) {
    if ((q - p).dot(x - p) < 0) return x.dist(
      p);
    if ((p - q).dot(x - q) < 0) return x.dist(
      q);
    return (q - p).unit().cross(x - p).dist();
  }
  // Check if point p lies on segment defined
      by [o, o + d]
  bool on_segment(P3 p) const {
    return dist(p) < EPS && (o - p).dot(o + d
      - p) <= EPS;
  }
};
// Line-line relations: min dist, closest
    point
double line_dist(Line3D l1, Line3D l2) {
  auto n = l1.d.cross(l2.d);
  if (n.dist2() == 0) return l1.dist(l2.o);
  return fabs((l2.o - l1.o).dot(n)) / n.dist()
      ;
}
P3 line_closest(Line3D l1, Line3D l2) {
  auto n2 = l2.d.cross(l1.d.cross(l2.d));
  return l1.o + l1.d * ((l2.o - l1.o).dot(n2)
      / l1.d.dot(n2));
}
```

12.4 Plane Line

```
// Plane-line angles, relations, and
    perpendiculars through a point.
double angle(Plane p, Line3D l) { return PI /
    2 - p.n.small_angle(l.d); }
bool is_para(Plane p, Line3D l) { return sgn(p
    .n.dot(l.d)) == 0; }
bool is_perp(Plane p, Line3D l) { return p.n.
    cross(l.d).dist() < EPS; }
Line3D perp_through(Plane p, P3 o) { return
    Line3D(o, o + p.n); }
Plane perp_through(Line3D l, P3 o) { return
    Plane(l.d, o); }
```

12.5 Convex Hull 3d

```
// O(N^2) Incremental 3D Convex Hull builder (
    outputs flattened triangle vector)
struct CH3D {
  struct face { int a, b, c; bool ok; };
  int n, num;
  vector<P3> P;
  vector<face> F;
  vector<vector<int>> g;
  void init(vector<P3> p) {
    P = p; n = p.size(); num = 0;
    F.resize(8 * n + 1); g.assign(n + 1,
      vector<int>(n + 1, 0));
  }
  double volume(P3 a, P3 b, P3 c, P3 d) {
    return ((b - a).cross(c - a)).dot(d - a)
    ; }
  double dblcmp(P3 &p, face &f) { return ((P[f
    .b] - P[f.a]).cross(P[f.c] - P[f.a])).
    dot(p - P[f.a]); }
  void deal(int p, int a, int b) {
    int f = g[a][b]; face add;
    if (F[f].ok) {
      if (dblcmp(P[p], F[f]) > EPS) dfs(p, f);
      else {
        add = {b, a, p, true};
        g[p][b] = g[a][p] = g[b][a] = num;
        F[num++] = add;
      }
    }
  }
  void dfs(int p, int now) {
    F[now].ok = false;
    deal(p, F[now].b, F[now].a);
    deal(p, F[now].c, F[now].b);
    deal(p, F[now].a, F[now].c);
  }
  void build() {
    if (n < 4) return;
    bool ok = false;
    for (int i = 1; i < n; i++) if (P[0].dist(
      P[i]) > EPS) { swap(P[1], P[i]); ok =
      true; break; }
    if (!ok) return; ok = false;
    for (int i = 2; i < n; i++) if ((P[0] - P
      [1]).cross(P[1] - P[i]).dist() > EPS) {
      swap(P[2], P[i]); ok = true; break; }
    if (!ok) return; ok = false;
    for (int i = 3; i < n; i++) if (fabs(((P
      [0] - P[1]).cross(P[1] - P[2])).dot(P[0]
      - P[i])) > EPS) { swap(P[3], P[i]); ok
      = true; break; }
    if (!ok) return;
    for (int i = 0; i < 4; i++) {
      face add = {(i + 1) % 4, (i + 2) % 4, (i
      + 3) % 4, true};
```

```
      if (dblcmp(P[i], add) > 0) swap(add.b,
     add.c);
      g[add.a][add.b] = g[add.b][add.c] = g[
     add.c][add.a] = num;
      F[num++] = add;
    }
    for (int i = 4; i < n; i++) {
      for (int j = 0; j < num; j++) if (F[j].
     ok && dblcmp(P[i], F[j]) > EPS) { dfs(i,
      j); break; }
    }
    int tmp = num; num = 0;
    for (int i = 0; i < tmp; i++) if (F[i].ok)
      F[num++] = F[i];
  }
  // center of mass
  P3 centroid() {
    P3 ans(0, 0, 0), o(0, 0, 0); double all =
     0;
    for (int i = 0; i < num; i++) {
      double vol = volume(o, P[F[i].a], P[F[i
     ].b], P[F[i].c]);
      ans = ans + (o + P[F[i].a] + P[F[i].b] +
      P[F[i].c]) / 4.0 * vol;
      all += vol;
    }
    return ans / all;
  }
  // Total surface area of the 3D Convex Hull
  double area() {
    if (n == 3) return (P[1] - P[0]).cross(P
     [2] - P[0]).dist() * 0.5;
    double res = 0;
    for (int i = 0; i < num; i++) {
      res += (P[F[i].b] - P[F[i].a]).cross(P[F
     [i].c] - P[F[i].a]).dist();
    }
    return res * 0.5;
  }
  // Total volume enclosed by the 3D Convex
     Hull
  double volume() {
    double res = 0; P3 o(0, 0, 0);
    for (int i = 0; i < num; i++) res +=
     volume(o, P[F[i].a], P[F[i].b], P[F[i].c
     ]);
    return fabs(res / 6.0);
  }
  // Are the triangular faces s and t coplanar
     ?
  bool same(int s, int t) {
    P3 a = P[F[s].a], b = P[F[s].b], c = P[F[s
     ].c];
    return fabs(volume(a, b, c, P[F[t].a])) <
     EPS && fabs(volume(a, b, c, P[F[t].b]))
     < EPS &&
           fabs(volume(a, b, c, P[F[t].c])) <
     EPS;
  }
  int num_tri() { return num; } // number of
     surface triangles
  int num_poly() {              // number of
     surface polygons, O(num^2)
    int res = 0;
    for (int i = 0; i < num; i++) {
      bool ok = true;
      for (int j = 0; j < i; j++) if (same(i,
     j)) { ok = false; break; }
      res += ok;
    }
    return res;
  }
  // Distance from point p to the plane of
     face i
  double pt_face_dist(P3 p, int i) {
    return fabs(volume(P[F[i].a], P[F[i].b], P
     [F[i].c], p)) /
           (P[F[i].b] - P[F[i].a]).cross(P[F[i
     ].c] - P[F[i].a]).dist();
  }
  // Check if point p is inside or on the 3D
     Convex Hull
  bool contains(P3 p) {
    for (int i = 0; i < num; i++) {
      if (dblcmp(p, F[i]) > EPS) return false;
    }
    return true;
  }
};
```

12.6 Sphere

```
//  Spherical coordinates, distances, and
     sphere-sphere intersections.
P3 sph_pt(double r, double lat, double lon) {
  lat *= PI / 180; lon *= PI / 180;
  return P3(r * cos(lat) * cos(lon), r * cos(
     lat) * sin(lon), r * sin(lat));
}
double great_circle_dist(P3 o, double r, P3 a,
      P3 b) { return r * (a - o).angle(b - o)
     ; }
struct Sphere {
  P3 c; double r;
  double volume() const { return 4.0 / 3.0 *
     PI * r * r * r; }
  double surface_area() const { return 4 * PI
     * r * r; }
  // Line-Sphere Intersection Points
  vector<P3> inter(Line3D l) const {
    P3 proj = l.proj(c);
    double d2 = c.dist2(proj);
    if (d2 > r * r + EPS) return {};
    double h = sqrt(max(0.0, r * r - d2));
    P3 dir = l.d.unit();
    if (h < EPS) return {proj};
    return {proj - dir * h, proj + dir * h};
  }
  // if this point is inside the sphere
  bool contains(P3 p) const { return sgn(p.
     dist(c) - r) <= 0; }
  // Length of the tangent segment from an
     external point p (-1 if p is inside)
  double tangent_len(P3 p) const {
    double d = p.dist(c);
    return sgn(d - r) < 0 ? -1 : sqrt(max(0.0,
      d * d - r * r));
  }
  // Polar angle of the cap of tangency points
      from p (-1 if inside, -2 if on the
     surface)
  double tangent_angle(P3 p) const {
    double d = p.dist(c);
    if (sgn(d - r) < 0) return -1;
    if (sgn(d - r) == 0) return -2;
    return acos(r / d);
  }
};
// sphere-sphere intersection
pair<double, double> sphere_inter(Sphere s1,
     Sphere s2) {
  double d = s1.c.dist(s2.c);
  if (d >= s1.r + s2.r) return {0, 0};
  double R = max(s1.r, s2.r), r = min(s1.r, s2
     .r);
  double x = (R * R - r * r + d * d) / (2 * d)
     ;
  double h1 = R - x, h2 = (R + r - d) - h1;
  double vol = PI * h1 * h1 * (3 * R - h1) /
     3.0 + PI * h2 * h2 * (3 * r - h2) / 3.0;
  double area = 2 * PI * R * h1 + 2 * PI * r *
      h2;
  return {area, vol};
}
// Radius of the circle in which two
     intersecting spheres meet
double sphere_inter_radius(Sphere s1, Sphere
     s2) {
  double d = s1.c.dist(s2.c), R = max(s1.r, s2
     .r), r = min(s1.r, s2.r);
  if (sgn(d - R - r) >= 0) return 0;
  double w = d * d - r * r + R * R;
  return sqrt(max(0.0, 4 * d * d * R * R - w *
      w)) / (2 * d);
}
// Sphere through four non-coplanar points
Sphere circum_sphere(P3 a, P3 b, P3 c, P3 d) {
  assert(sgn(Plane(a, b, c).side(d)) != 0);
  Plane u(a - b, (a + b) / 2), v(b - c, (b + c
     ) / 2), w(c - d, (c + d) / 2);
  Line3D l1(u, v), l2(v, w);
  P3 o = line_closest(l1, l2);
  return {o, o.dist(a)};
}
// Center of the circle through three non-
     collinear coplanar points.
P3 circle_center(P3 a, P3 b, P3 c) {
  P3 v1 = b - a, v2 = c - a;
  double v11 = v1.dot(v1), v22 = v2.dot(v2),
     v12 = v1.dot(v2);
  double base = 0.5 / (v11 * v22 - v12 * v12);
  return a + v1 * (base * v22 * (v11 - v12)) +
      v2 * (base * v11 * (v22 - v12));
}
// approximate min enclosing sphere
Sphere min_sphere(vector<P3> p) {
  P3 c(0, 0, 0); int n = p.size(), pos = 0;
  for (auto &i : p) c = c + i;
  c = c / n;
  double ratio = 0.1;
  for (int it = 0; it < 100000; it++) {
    pos = 0;
    for (int i = 1; i < n; i++) if (c.dist2(p[
     i]) > c.dist2(p[pos])) pos = i;
    c = c + (p[pos] - c) * ratio;
    ratio *= 0.998;
  }
  return {c, c.dist(p[pos])};
}
```

12.7 TetraHedron Volume

```
// Tetrahedron (A, B, C, D)
double vol_tetrahedron(p3 a, p3 b, p3 c, p3 d)
     {
  return fabs((b - a).cross(c - a).dot(d - a))
      / 6.0;
}
```

13 Miscellaneous

13.1 Interval Container

Description: Add and remove intervals from a set of disjoint intervals. Will merge the added interval with any overlapping intervals in the set when adding. Intervals are [inclusive, exclusive). Time: O(logN)

```
set<pair<int, int>>::iterator addInterval(set<
     pair<int, int>>& is, int L, int R) {
  if (L == R) return is.end();
  auto it = is.lower_bound({L, R}), before =
     it;
  while (it != is.end() && it->first <= R) {
    R = max(R, it->second);
    before = it = is.erase(it);
  }
  if (it != is.begin() && (--it)->second >= L)
     {
    L = min(L, it->first);
    R = max(R, it->second);
    is.erase(it);
  }
  return is.insert(before, {L,R});
}
void removeInterval(set<pair<int, int>>& is,
     int L, int R) {
  if (L == R) return;
  auto it = addInterval(is, L, R);
  auto r2 = it->second;
  if (it->first == L) is.erase(it);
  else (int&)it->second = L;
  if (R != r2) is.emplace(R, r2);
}
```

13.2 Efficient Mask Traversal

Description: Traverses the mask with minimal changes in 2^n as opposed to the traditional $n2^n$ approach. If you don't care about time complexity, you can do a greedy brute force with 4^n time complexity.

```
long long mask = 0;
for (int i = 1; i < (1LL << n); ++i) {
  int bit = __builtin_ctz(i);
  mask ^= (1LL << bit);
}
```

13.3 Generationg Permutations

```
VI a;
//...
sort(all(a));
do {
  // Process permutation...
} while (next_permutation(all(a)));
```

13.4 Iterating over Submasks

```
int n = 17;
for (int mask = 0; mask < (1 << n); ++mask) {
  for (int sub = mask; sub; sub = (sub - 1) &
     mask) {
    // Process submask...
  }
}
```

13.5 Random

```
mt19937 rng(chrono::steady_clock::now().
     time_since_epoch().count());
int randint(int l, int r) {
  uniform_int_distribution<int> dist(l, r);
  return dist(rng);
}
```

13.6 pbds

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
template <typename T>
using ordered_set = tree<T, null_type, less<T
     >, rb_tree_tag,
     tree_order_statistics_node_update>;
ordered_set<int> st;
// ordered multiset
ordered_set<pair<int, int>> multi;
// get index/ number of smaller elements than
     x
```

```
st.order_of_key(x);
// access by index
st.find_by_order(idx);
```

13.7 Bitset

```
bitset<N> b; // All 0s; b[i] access; b.test(i)
      bounds check
bitset<N> b(string("110")); // b[0]=0, b[1]=1,
      b[2]=1
b.set(i, v=1); b.reset(i); b.flip(i); //
     Single bit modify
b.set(); b.reset(); b.flip(); // All bits
     modify
b.count(); b.any(); b.all(); // popcount; if
     1+ bits set; if all bits set
b >>= k; b <<= k; b &= b2; b |= b2; b ^= b2; b
      = ~b; // O(N/64) bitwise
// GCC Specific (The good stuff):
b._Find_first(); // Index of first '1' (
     returns N if none)
b._Find_next(i); // Index of next '1' after i
     (returns N if none)
```

13.8 Primes

```
int BASES[] = {31, 37, 41, 43, 47, 53, 59, 61,
      67, 71, 137, 139, 149, 151, 157, 257,
     263, 269, 271, 277, 311, 313, 317, 331,
     347};
int MOD_32[] = {998244353, 999999937,
     1000000007, 1000000009, 1000000021,
     1000000103, 1000003931, 1000004357,
     2147483647};
ll MOD_1E12[] = {1000000000039,
     1000000000063,1000000000091,
     1000000000121, 1000000000169,
     1000000000213, 1000000000223};
ll MOD_64[] = {1000000000000000003,
1000000000000000009,1000000000000000033,
4611686018427387847,9223372036854775783};
pair<int, int> ntt_primes = {{(119 << 23) + 1,
      3}, {(5 << 25) + 1, 3}, {(7 << 26) + 1,
      3},{(479 << 21) + 1, 3}, {(483 << 21) +
      5, 5}};
```

13.9 Digit Range Sum

Description: Gives the digit sum of the numbers in the range [l,r]. I have encountered this problem twice.

```
auto qrsm = [](ll n) -> ll {
  ll ans = 0;
  ll po = 1;
  ll nm = n;
  while (n) {
    ll tm = n / 10;
    ll rem = n % 10;
    ll s1 = rem * (rem - 1) / 2;
    ll past = (nm % po) + 1ll;
    ans += (tm * 45ll) * po + (s1) * (po) + (
      rem * past);
    po *= 10;
    n /= 10;
  }
  return ans;
};
auto rsm = [&](ll l, ll r) -> ll { return qrsm
     (r) - qrsm(l - 1); };
```

13.10 Custom Hash

```
// Usage: unordered_set<pair<int, int>,
     pointhash> st;
struct pointhash {
  size_t operator()(const pair<int, int>& p)
     const {
    return hash<int>()(p.first) ^ (hash<int>()
     (p.second) << 1);
  }
};
struct custom_hash {
  static uint64_t splitmix64(uint64_t x) {
    x += 0x9e3779b97f4a7c15;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
    x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
    return x ^ (x >> 31);
  }
  size_t operator()(uint64_t x) const {
    static const uint64_t FIXED_RANDOM =
        chrono::steady_clock::now().
     time_since_epoch().count();
    return splitmix64(x + FIXED_RANDOM);
  }
};
unordered_map<long long, int, custom_hash>
     safe_map;
```

13.11 Pragma

```
#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt
     ")
```

14 Formulae

14.1 Combinatorics

14.1.1 Combinatorial Identities

1. $\sum_{0\le k\le n}\binom{n-k}{k} = Fib_{n+1}$
2. $\binom{n}{k} = \binom{n}{n-k}$
3. $\binom{n}{k}+\binom{n}{k+1} = \binom{n+1}{k+1}$
4. $k\binom{n}{k} = n\binom{n-1}{k-1}$
5. $\binom{n}{k} = \frac{n}{k}\binom{n-1}{k-1}$
6. $\sum_{i=0}^{n}\binom{n}{i} = 2^n$
7. $\sum_{i\ge 0}\binom{n}{2i} = 2^{n-1}$
8. $\sum_{i\ge 0}\binom{n}{2i+1} = 2^{n-1}$
9. $\sum_{i=0}^{k}(-1)^i\binom{n}{i} = (-1)^k\binom{n-1}{k}$
10. $\sum_{i=0}^{k}\binom{n+i}{i} = \sum_{i=0}^{k}\binom{n+i}{n} = \binom{n+k+1}{k}$
11. $1\binom{n}{1}+2\binom{n}{2}+3\binom{n}{3}+\cdots+n\binom{n}{n} = n2^{n-1}$
12. $1^2\binom{n}{1}+2^2\binom{n}{2}+3^2\binom{n}{3}+\cdots+n^2\binom{n}{n} = (n+n^2)2^{n-2}$
13. **Vandermonde's Identify:** $\sum_{k=0}^{r}\binom{m}{k}\binom{n}{r-k} = \binom{m+n}{r}$
14. **Hockey-Stick Identify:** $n,r\in N, n>r, \sum_{i=r}^{n}\binom{i}{r} = \binom{n+1}{r+1}$
15. $\sum_{i=0}^{k}\binom{k}{i}^2 = \binom{2k}{k}$
16. $\sum_{k=0}^{n}\binom{n}{k}\binom{n}{n-k} = \binom{2n}{n}$
17. $\sum_{k=q}^{n}\binom{n}{k}\binom{k}{q} = 2^{n-q}\binom{n}{q}$
18. $\sum_{i=0}^{n}k^i\binom{n}{i} = (k+1)^n$
19. $\sum_{i=0}^{n}\binom{2n}{i} = 2^{2n-1}+\frac{1}{2}\binom{2n}{n}$
20. $\sum_{i=1}^{n}\binom{n}{i}\binom{n-1}{i-1} = \binom{2n-1}{n-1}$
21. $\sum_{i=0}^{n}\binom{2n}{i}^2 = \frac{1}{2}\left(\binom{4n}{2n}+\binom{2n}{n}^2\right)$
22. **Highest Power of 2 that divides $^{2n}C_n$:** Let x be the number of 1s in the binary representation. Then the number of odd terms will be 2^x. Let it form a sequence. The n-th value in the sequence (starting from $n=0$) gives the highest power of 2 that divides $^{2n}C_n$.
23. **Pascal Triangle**
    (a) In a row p where p is a prime number, all the terms in that row except the 1s are multiples of p.
    (b) Parity: To count odd terms in row n, convert n to binary. Let x be the number of 1s in the binary representation. Then the number of odd terms will be 2^x.
    (c) Every entry in row $2^n-1, n\ge 0$, is odd.
24. An integer $n\ge 2$ is prime if and only if all the intermediate binomial coefficients $\binom{n}{1},\binom{n}{2},\ldots,\binom{n}{n-1}$ are divisible by n.
25. **Kummer's Theorem:** For given integers $n\ge m\ge 0$ and a prime number p, the largest power of p dividing $\binom{n}{m}$ is equal to the number of carries when m is added to n-m in base p. For implementation take inspiration from lucas theorem.
26. Number of different binary sequences of length n such that no two 0's are adjacent=Fib_{n+1}
27. **Combination with repetition:** Let's say we choose k elements from an n-element set, the order doesn't matter and each element can be chosen more than once. In that case, the number of different combinations is: $\binom{n+k-1}{k}$
28. Number of ways to divide n persons in $\frac{n}{k}$ equal groups i.e. each having size k is $\frac{n!}{k!^{\frac{n}{k}}(\frac{n}{k})!} = \prod_{n\ge k}^{n-=k}\binom{n-1}{k-1}$
29. The number non-negative solution of the equation: $x_1+x_2+x_3+\cdots+x_k = n$ is $\binom{n+k-1}{n}$
30. Number of ways to choose n ids from 1 to b such that every id has distance at least k $= \binom{b-(n-1)(k-1)}{n}$
31. $\sum_{i=1,3,5,\ldots}^{i\le n}\binom{n}{i}a^{n-i}b^i = \frac{1}{2}((a+b)^n-(a-b)^n)$
32. $\sum_{i=0}^{n}\frac{\binom{k}{i}}{\binom{n}{i}} = \frac{\binom{n+1}{n-k+1}}{\binom{n}{k}}$
33. **Derangement:** a permutation of the elements of a set, such that no element appears in its original position. Let $d(n)$ be the number of derangements of the identity permutation fo size n.
    $d(n) = (n-1)\big(d(n-1)+d(n-2)\big)$,
    where $d(0)=1,\ d(1)=0$.
34. **Involutions:** permutations such that $p^2=$ identity permutation. $a_0=a_1=1$ and $a_n=a_{n-1}+(n-1)a_{n-2}$ for $n>1$.
35. Let $T(n,k)$ be the number of permutations of size n for which all cycles have length $\le k$.
$$T(n,k)=\begin{cases} n! & ;n\le k \\ n\cdot T(n-1,k) & \\ -F(n-1,k) & \\ \cdot T(n-k-1,k) & ;n>k\end{cases}$$
Here $F(n,k)=n\cdot(n-1)\cdot\cdots\cdot(n-k+1)$
36. **Lucas Theorem**
    (a) If p is prime, then $\binom{p^a}{k} \equiv 0 \pmod p$
    (b) For non-negative integers m and n and a prime p, the following congruence relation holds: $\binom{m}{n} \equiv \prod_{i=0}^{k}\binom{m_i}{n_i} \pmod p$, where, $m = m_kp^k+m_{k-1}p^{k-1}+\cdots+m_1p+m_0$, and $n=n_kp^k+n_{k-1}p^{k-1}+\cdots+n_1p+n_0$ are the base p expansions of m and n respectively. This uses the convention that $\binom{m}{n}=0$,when $m<n$.
37. $\sum_{i=0}^{n}\binom{n}{i}\cdot i^k$
$= \sum_{i=0}^{n}\binom{n}{i}\cdot\sum_{j=0}^{k}\left\{{k\atop j}\right\}\cdot i^{\underline{j}}$
$= \sum_{i=0}^{n}\binom{n}{i}\cdot\sum_{j=0}^{k}\left\{{k\atop j}\right\}\cdot j!\binom{n}{i}$
$= \sum_{i=0}^{n}\frac{n!}{(n-i)!}\cdot\sum_{j=0}^{k}\left\{{k\atop j}\right\}\cdot\frac{1}{(i-j)!}$
$= \sum_{i=0}^{n}\sum_{j=0}^{k}\frac{n!}{(n-i)!}\cdot\left\{{k\atop j}\right\}\cdot\frac{1}{(i-j)!}$

$$= n! \sum_{i=0}^{n} \sum_{j=0}^{k} \left\{ {k \atop j} \right\} \cdot \frac{1}{(n-i)!} \cdot \frac{1}{(i-j)!}$$
$$= n! \sum_{i=0}^{n} \sum_{j=0}^{k} \left\{ {k \atop j} \right\} \cdot \binom{n-j}{n-i} \cdot \frac{1}{(n-j)!}$$
$$= n! \sum_{j=0}^{k} \left\{ {k \atop j} \right\} \cdot \frac{1}{(n-j)!} \sum_{i=0}^{n} \cdot \binom{n-j}{n-i}$$
$$= \sum_{j=0}^{k} \left\{ {k \atop j} \right\} \cdot n^{\underline{j}} \cdot 2^{n-j}$$

Here $n^{\underline{j}} = P(n,j) = \frac{n!}{(n-j)!}$ and $\left\{ {k \atop j} \right\}$ is stirling number of the second kind.

So, instead of $O(n)$, now you can calculate the original equation in $O(k^2)$ or even in $O(k \log^2 n)$ using NTT.

38. $\sum_{i=0}^{n-1} \binom{i}{j} x^i = x^j (1-x)^{-j-1} \cdot \left(1 - x^n \sum_{i=0}^{j} \binom{n}{i} x^{j-i} (1-x)^i\right)$

39. $x_0, x_1, x_2, x_3, \ldots, x_n$
$x_0 + x_1, x_1 + x_2, x_2 + x_3, \ldots x_n$
$\ldots$
If we continuously do this n times then the polynomial of the first column of the n-th row will be
$$p(n) = \sum_{k=0}^{n} \binom{n}{k} \cdot x(k)$$

40. If $P(n) = \sum_{k=0}^{n} \binom{n}{k} \cdot Q(k)$, then,
$$Q(n) = \sum_{k=0}^{n} (-1)^{n-k} \binom{n}{k} \cdot P(k)$$

41. If $P(n) = \sum_{k=0}^{n} (-1)^k \binom{n}{k} \cdot Q(k)$, then,
$$Q(n) = \sum_{k=0}^{n} (-1)^k \binom{n}{k} \cdot P(k)$$

14.1.2 More useful identities added later

1. ways to arrange i pairs of brackets into j valid non-empty blocks $dp[i][j] = \left(j \cdot \binom{2i-j}{i-j} \cdot (2i-j)^{-1}\right) \pmod M$
2. $\sum_{k=1}^{n} k^p = \frac{1}{p+1} \sum_{r=0}^{p} \binom{p+1}{r} B_r^+ n^{p+1-r}$
3. Prefix sum of nCr: $2B(l,r,n) = \binom{n}{l} + \binom{n}{r} - \binom{n+1}{l} + B(l,r,n+1)$
4. $(q+p)^k - (q-p)^k = 2 \cdot \sum_{j \text{ odd}} \binom{k}{j} p^j q^{k-j}$
5. $\sum_{i=1}^{n} i^{\underline{j}} = \frac{(n+1)^{\underline{j+1}}}{j+1}$
6. $\sum_{i=0}^{n} i \binom{n}{i} = n 2^{n-1}$
7. $\binom{n}{r} \pmod 2 \neq 0 \iff (n \,\&\, r) == r$

14.1.3 Inclusion-Exclusion Principle

Let S_j be the sum of sizes of all intersections of exactly j properties:
$$S_j = \sum_{1 \le i_1 < \cdots < i_j \le n} |A_{i_1} \cap \cdots \cap A_{i_j}|$$

- **Union (At least 1):** $|A_1 \cup \cdots \cup A_n| = \sum_{j=1}^{n} (-1)^{j-1} S_j$
- **Exactly k properties:** $E_k = \sum_{j=k}^{n} (-1)^{j-k} \binom{j}{k} S_j$
- **At least k properties:** $L_k = \sum_{j=k}^{n} (-1)^{j-k} \binom{j-1}{k-1} S_j$

Implementation Note:
If intersections depend only on the number of sets j, precompute $S_j = \binom{n}{j} \times$ intersect(j) in $O(n)$. Otherwise, use bitmask PIE in $O(2^n)$.

14.1.4 Labeled unrooted trees

8. n vertices: n^{n-2}
9. k trees of size n_i: $(\prod n_i) n^{k-2}$
10. With degrees d_i: $\frac{(n-2)!}{\prod (d_i - 1)!}$

14.1.5 Catalan Numbers

11. $C_n = \frac{1}{n+1} \binom{2n}{n}$
12. $C_0 = 1, C_1 = 1$ and $C_n = \sum_{k=0}^{n-1} C_k C_{n-1-k}$
13. Number of correct bracket sequence consisting of n opening and n closing brackets.
14. The number of ways to completely parenthesize n+1 factors.
15. The number of triangulations of a convex polygon with n+2 sides (i.e. the number of partitions of polygon into disjoint triangles by using the diagonals).
16. The number of ways to connect the $2n$ points on a circle to form n disjoint i.e. non-intersecting chords.
17. The number of monotonic lattice paths from point $(0,0)$ to point (n,n) in a square lattice of size $n \times n$, which do not pass above the main diagonal (i.e. connecting $(0,0)$ to (n,n)).
18. The number of rooted full binary trees with n+1 leaves (vertices are not numbered). A rooted binary tree is full if every vertex has either two children or no children.
19. The number of ordered trees with $n+1$ vertices.
20. Number of permutations of $\{1, \ldots, n\}$ that avoid the pattern 123 (or any of the other patterns of length 3); that is, the number of permutations with no three-term increasing subsequence. For $n = 3$, these permutations are $132, 213, 231, 312$ and 321. For $n = 4$, they are $1432, 2143, 2413, 2431, 3142, 3214$, $3241, 3412, 3421, 4132, 4213, 4231, 4312$ and 4321.
21. **Balanced Parentheses count with prefix:** The count of balanced parentheses sequences consisting of $n + k$ pairs of parentheses where the first k symbols are open brackets. Let the number be $C_n^{(k)}$, then
$$C_n^{(k)} = \frac{k+1}{n+k+1} \binom{2n+k}{n}$$

14.1.6 Narayana numbers

22. $N(n,k) = \frac{1}{n} \left(\frac{n}{k}\right) \left(\frac{n}{k-1}\right)$
23. The number of expressions containing n pairs of parentheses, which are correctly matched and which contain k distinct nestings. For instance, $N(4,2) = 6$ as with four pairs of parentheses six sequences can be created which each contain two times the sub-pattern '()'.

14.1.7 Stirling numbers of the first kind

24. The Stirling numbers of the first kind count permutations according to their number of cycles (counting fixed points as cycles of length one).
25. $S(n,k)$ counts the number of permutations of n elements with k disjoint cycles.
26. $S(n,k) = (n-1) \cdot S(n-1,k) + S(n-1,k-1)$, *where*, $S(0,0) = 1, S(n,0) = S(0,n) = 0$
27. $\sum_{k=0}^{n} S(n,k) = n!$
28. The unsigned Stirling numbers may also be defined algebraically, as the coefficient of the rising factorial:
$$x^{\bar{n}} = x(x+1)\ldots(x+n-1) = \sum_{k=0}^{n} S(n,k) x^k$$
29. Lets $[n,k]$ be the stirling number of the first kind, then
$$\left[{n \atop k} \right] = \sum_{0 \le i_1 < i_2 < i_k < n} i_1 i_2 \cdots k \,.$$

14.1.8 Stirling numbers of the second kind

30. Stirling number of the second kind is the number of ways to partition a set of n objects into k non-empty subsets.
31. $S(n,k) = \frac{1}{k!} \sum_{j=0}^{k} (-1)^{k-j} \binom{k}{j} j^n$
32. $S(n,k) = k \cdot S(n-1,k) + S(n-1,k-1)$, *where* $S(0,0) = 1, S(n,0) = S(0,n) = 0$
33. $S(n,2) = 2^{n-1} - 1$
34. $S(n,k) \cdot k!$ = number of ways to color n nodes using colors from 1 to k such that each color is used at least once.
35. An r-associated Stirling number of the second kind is the number of ways to partition a set of n objects into k subsets, with each subset containing at least r elements. It is denoted by $S_r(n,k)$ and obeys the recurrence relation. $S_r(n+1,k) = kS_r(n,k) + \binom{n}{r-1} S_r(n-r+1, k-1)$
36. Denote the n objects to partition by the integers $1, 2, \ldots, n$. Define the reduced Stirling numbers of the second kind, denoted $S^d(n,k)$, to be the number of ways to partition the integers $1, 2, \ldots, n$ into k nonempty subsets such that all elements in each subset have pairwise distance at least d. That is, for any integers i and j in a given subset, it is required that $|i - j| \ge d$. It has been shown that these numbers satisfy, $S^d(n,k) = S(n-d+1, k-d+1), n \ge k \ge d$

14.1.9 Bell Number

37. Counts the number of partitions of a set.
38. $B_{n+1} = \sum_{k=0}^{n} \left(\frac{n}{k}\right) \cdot B_k$
39. $B_n = \sum_{k=0}^{n} S(n,k)$,where $S(n,k)$ is stirling number of second kind.

14.2 Math

40. $\sum_{i=0}^{n} i \cdot i! = (n+1)! - 1.$
41. $a^k - b^k = (a-b) \cdot (a^{k-1}b^0 + a^{k-2}b^1 + \cdots + a^0 b^{k-1})$
42. $\min(a+b, c) = a + \min(b, c-a)$
43. $|a-b| + |b-c| + |c-a| = 2(\max(a,b,c) - \min(a,b,c))$
44. $a \cdot b \le c \to a \le \left\lfloor \frac{c}{b} \right\rfloor$ is correct
45. $a \cdot b < c \to a < \left\lfloor \frac{c}{b} \right\rfloor$ is incorrect
46. $a \cdot b \ge c \to a \ge \left\lfloor \frac{c}{b} \right\rfloor$ is correct
47. $a \cdot b > c \to a > \left\lfloor \frac{c}{b} \right\rfloor$ is correct
48. For positive integer n, and arbitrary real numbers m, x, $\left\lfloor \frac{\lfloor x/m \rfloor}{n} \right\rfloor = \left\lfloor \frac{x}{mn} \right\rfloor$ $\left\lceil \frac{\lceil x/m \rceil}{n} \right\rceil = \left\lceil \frac{x}{mn} \right\rceil$
49. Lagrange's identity:
$$\left(\sum_{k=1}^{n} a_k^2\right)\left(\sum_{k=1}^{n} b_k^2\right) - \left(\sum_{k=1}^{n} a_k b_k\right)^2$$
$$= \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} (a_i b_j - a_j b_i)^2$$
$$= \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1, j \neq i}^{n} (a_i b_j - a_j b_i)^2$$
50. $\sum_{i=1}^{n} i a^i = \frac{a(na^{n+1} - (n+1)a^n + 1)}{(a-1)^2}$
51. **Vieta's formulas:** Any general polynomial of degree n
$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$$
(with the coefficients being real or complex numbers and $a_n \neq 0$) is known by the fundamental theorem of algebra to have

n (not necessarily distinct) complex roots $r_1, r_2, \ldots, r_n$.

$$\begin{cases} r_1 + r_2 + \cdots + r_{n-1} + r_n = -\frac{a_{n-1}}{a_n} \\ (r_1 r_2 + r_1 r_3 + \cdots + r_1 r_n) \\ +(r_2 r_3 + r_2 r_4 + \cdots + r_2 r_n) + \ldots \\ +r_{n-1} r_n = \frac{a_{n-2}}{a_n} \\ \vdots \\ r_1 r_2 \ldots r_n = (-1)^n \frac{a_0}{a_n}. \end{cases}$$

Vieta's formulas can equivalently be written as

$$\sum_{1 \le i_1 < i_2 < \cdots < i_k \le n} \left(\prod_{j=1}^{k} r_{i_j} \right) = (-1)^k \frac{a_{n-k}}{a_n},$$

52. We are given n numbers $a_1, a_2, \ldots, a_n$ and our task is to find a value x that minimizes the sum,

$$|a_1 - x| + |a_2 - x| + \cdots + |a_n - x|$$

optimal x = median of the array. if n is even x = [left median,right median] i.e. every number in this range will work. For minimizing

$$(a_1 - x)^2 + (a_2 - x)^2 + \cdots + (a_n - x)^2$$

optimal $x = \dfrac{(a_1 + a_2 + \cdots + a_n)}{n}$

53. Given an array a of n non-negative integers. The task is to find the sum of the product of elements of all the possible subsets. It is equal to the product of $(a_i + 1)$ for all a_i

54. **Pentagonal number theorem:** In mathematics, the pentagonal number theorem states that

$\prod_{n=1}^{\infty}(1 - x^n) = \sum_{k=-\infty}^{\infty}(-1)^k x^{\frac{k(3k-1)}{2}}$
$= 1 + \sum_{k=1}^{\infty}(-1)^k \left(x^{\frac{k(3k+1)}{2}} + x^{\frac{k(3k-1)}{2}} \right)$.

In other words,

$$\begin{aligned} &(1-x)(1-x^2)(1-x^3)\cdots \\ &= 1 - x - x^2 + x^5 + x^7 - x^{12} \\ &-x^{15} + x^{22} + x^{26} - \cdots . \end{aligned}$$

The exponents $1, 2, 5, 7, 12, \cdots$ on the right hand side are given by the formula $g_k = \frac{k(3k-1)}{2}$ for $k = 1, -1, 2, -2, 3, \cdots$ and are called (generalized) pentagonal numbers. It is useful to find the partition number in $O(n\sqrt{n})$

14.2.1 Fibonacci Numbers

55. $F_0 = 0, F_1 = 1$ and $F_n = F_{n-1} + F_{n-2}$

56. $F_n = \sum_{k=0}^{\lfloor \frac{n-1}{2} \rfloor} \binom{n-k-1}{k}$

57. $F_n = \frac{1}{\sqrt{5}}(\frac{1+\sqrt{5}}{2})^n - \frac{1}{\sqrt{5}}(\frac{1-\sqrt{5}}{2})^n$

58. $\sum_{i=1}^{n} F_i = F_{n+2} - 1$

59. $\sum_{i=0}^{n-1} F_{2i+1} = F_{2n}$

60. $\sum_{i=1}^{n} F_{2i} = F_{2n+1} - 1$

61. $\sum_{i=1}^{n} F_i^2 = F_n F_{n+1}$

62. $F_m F_{n+1} - F_{m-1} F_n = (-1)^n F_{m-n}$ $F_{2n} = F_{n+1}^2 - F_{n-1}^2 = F_n(F_{n+1} + F_{n-1})$

63. $F_m F_n + F_{m-1} F_{n-1} = F_{m+n-1}$ $F_m F_{n+1} + F_{m-1} F_n = F_{m+n}$

64. A number is Fibonacci if and only if one or both of $(5 \cdot n^2 + 4)$ or $(5 \cdot n^2 - 4)$ is a perfect square

65. Every third number of the sequence is even and more generally, every k^{th} number of the sequence is a multiple of F_k

66. $gcd(F_m, F_n) = F_{gcd(m,n)}$

67. Any three consecutive Fibonacci numbers are pairwise coprime, which means that, for every n, $gcd(F_n, F_{n+1}) = gcd(F_n, F_{n+2}), gcd(F_{n+1}, F_{n+2}) = 1$

68. If the members of the Fibonacci sequence are taken $mod\ n$, the resulting sequence is periodic with period at most $6n$.

14.2.2 Pythagorean Triples

69. A Pythagorean triple consists of three positive integers a, b, and C, such that $a^2 + b^2 = c^2$. Such a triple is commonly written (a, b, c)

70. Euclid's formula is a fundamental formula for generating Pythagorean triples given an arbitrary pair of integers m and n with $m > n > 0$. The formula states that the integers

$$a = m^2 - n^2, b = 2mn, c = m^2 + n^2$$

form a Pythagorean triple. The triple generated by Euclid's formula is primitive if and only if m and n are coprime and not both odd. When both m and n are odd, then a, b, and c will be even, and the triple will not be primitive; however, dividing a, b, and c by 2 will yield a primitive triple when m and n are coprime and both odd.

71. The following will generate all Pythagorean triples uniquely:

$$\begin{aligned} &a = k\,(m^2 - n^2), \\ &b = k\,(2mn), c = k\,(m^2 + n^2). \end{aligned}$$

where m, n, and k are positive integers with $m > n$, and with m and n coprime and not both odd.

72. **Theorem:** The number of Pythagorean triples a,b,n with $maxa, b, n = n$ is given by

$$\frac{1}{2}\left(\prod_{p^\alpha || n} (2\alpha + 1) - 1 \right)$$

where the product is over all prime divisors p of the form $4k + 1$. The notation $p^\alpha || n$ stands for the highest exponent α for which p^α divides n **Example:** For $n = 2 \cdot 3^2 \cdot 5^3 \cdot 7^4 \cdot 11^5 \cdot 13^6$, the number of Pythagorean triples with hypotenuse n is $\frac{1}{2}(7.13 - 1) = 45$. To obtain a formula for the number of Pythagorean triples with hypotenuse less than a specific positive integer N, we may add the numbers corresponding to each $n < N$ given by the Theorem. There is no simple way to compute this as a function of N.

14.2.3 Sum of Squares Function

73. The function is defined as

$$r_k(n) = \left| (a_1, a_2, \ldots, a_k) \in \mathbf{Z}^k : n = a_1^2 + a_2^2 + \cdots + a_k^2 \right|$$

74. The number of ways to write a natural number as sum of two squares is given by $r_2(n)$. It is given explicitly by $r_2(n) = 4(d_1(n) - d_3(n))$ where d1(n) is the number of divisors of n which are congruent with 1 modulo 4 and d3(n) is the number of divisors of n which are congruent with 3 modulo 4. The prime factorization $n = 2^g p_1^{f_1} p_2^{f_2} \ldots q_1^{h_1} q_2^{h_2} \ldots$, where p_i are the prime factors of the form $p_i \equiv 1 \pmod 4$, and q_i are the prime factors of the form $q_i \equiv 3 \pmod 4$ gives another formula $r_2(n) = 4(f_1 + 1)(f_2 + 1)\ldots$, if all exponents h_1,h_2,... are even. If one or more h_i are odd, then $r_2(n) = 0$.

75. The number of ways to represent n as the sum of four squares is eight times the sum of all its divisors which are not divisible by 4, i.e. $r_4(n) = 8 \sum_{d|n;4\nmid d} d$ $r_8(n) = 16 \sum_{d|n} (-1)^{n+d} d^3$

14.3 Number Theory

76. $ab \mod ac = a(b \mod c)$

77. for $i > j$, $\gcd(i, j) = \gcd(i - j, j) \le (i - j)$

78. $\sum_{x=1}^{n} \left[d | x^k \right] = \left\lfloor \frac{n}{\prod_{i=0} p_i^{\left\lceil \frac{e_i}{k} \right\rceil}} \right\rfloor$, where $d = \prod_{i=0} p_i^{e_i}$. Here, $[a|b]$ means if a divides b then it is 1, otherwise it is 0.

79. The number of lattice points on segment (x_1, y_1) to (x_2, y_2) is $\gcd(abs(x_1 - x_2), abs(y_1 - y_2)) + 1$

80. $(n-1)! \mod n = n - 1$ if n is prime, 2 if $n = 4$, 0 otherwise.

81. A number has odd number of divisors if it is perfect square

82. The sum of all divisors of a natural number n is odd if and only if $n = 2^r \cdot k^2$ where r is non-negative and k is positive integer.

83. Let a and b be coprime positive integers, and find integers $a\prime$ and $b\prime$ such that $aa\prime \equiv 1 \mod b$ and $bb\prime \equiv 1 \mod a$. Then the number of representations of a positive integers (n) as a non negative linear combination of a and b is

$$\frac{n}{ab} - \left\{ \frac{b\prime n}{a} \right\} - \left\{ \frac{a\prime n}{b} \right\} + 1$$

Here, x denotes the fractional part of x.

$$\begin{aligned} &\sum_{i=1}^{a} \sum_{j=1}^{b} \sum_{k=1}^{c} d(i \cdot j \cdot k) \\ &= \sum_{\gcd(i,j)=\gcd(j,k)=\gcd(k,i)=1} \left\lfloor \frac{a}{i} \right\rfloor \left\lfloor \frac{b}{j} \right\rfloor \left\lfloor \frac{c}{k} \right\rfloor \end{aligned}$$

Here, $d(x)$ = number of divisors of x.

84. **Gauss's generalization of Wilson's theorem:**, Gauss proved that,

$$\prod_{\substack{k=1 \\ \gcd(k,m)=1}}^{m} k \equiv \begin{cases} -1 \pmod m & \text{if } m = 4,\ p^\alpha,\ 2p^\alpha \\ 1 \pmod m & \text{otherwise} \end{cases}$$

where p represents an odd prime and α a positive integer. The values of m for which the product is -1 are precisely the ones where there is a primitive root modulo m.

14.3.1 Extgcd and Diophantine

Linear Diophantine equation solves the equation $ax + by = c$ where $gcd(a, b) \mid c$ Extended gcd calculates $ax + by = 1$ where $gcd(a, b) = 1$. Suppose, $(x, y) = (x0, y0)$. Now we can write diophantine equation as $\frac{a}{g}x + \frac{b}{g}y = \frac{c}{g}$ where $g = gcd(a, b)$. This equation can be put into extended gcd and we can obtain $(X, Y) = (\frac{c}{g}x0, \frac{c}{g}y0)$. Thus the general solution is obtained as:

$$x = X + \frac{b}{g}t \quad y = Y - \frac{a}{g}t \quad t \in \mathbb{Z}$$

Typically these type of problem requires a range of x,y ($x \in [minx, maxx], y \in [miny, maxy]$).

14.3.2 Divisor Function

85. $\sigma_x(n) = \sum_{d|n} d^x$

86. It is multiplicative i.e if $\gcd(a, b) = 1 \rightarrow \sigma_x(ab) = \sigma_x(a)\sigma_x(b)$.

87. $$\sigma_x(n) = \prod_{i=1}^{\tau} \frac{p_i^{(a_i+1)x} - 1}{p_i^x - 1}$$

88. **Divisor Summatory Function**

(a) Let $\sigma_0(k)$ be the number of divisors of k.

(b) $D(x) = \sum_{n \le x} \sigma_0(n)$

(c) $D(x) = \sum_{k=1}^{x} \lfloor \frac{x}{k} \rfloor = 2 \sum_{k=1}^{u} \lfloor \frac{x}{k} \rfloor - u^2$, where $u = \sqrt{x}$

(d) $D(n)$ =Number of increasing arithmetic progressions where $n + 1$ is the second or later term. (i.e. The last term, starting term can be any positive integer $\le n$. For example, $D(3) = 5$ and there are 5 such arithmetic progressions: $(1, 2, 3, 4); (2, 3, 4); (1, 4); (2, 4); (3, 4)$.

89. Let $\sigma_1(k)$ be the sum of divisors of k. Then, $\sum_{k=1}^{n} \sigma_1(k) = \sum_{k=1}^{n} k \left\lfloor \frac{n}{k} \right\rfloor$

90. $\prod_{d|n} d = n^{\frac{\sigma_0}{2}}$ if n is not a perfect square, and $= \sqrt{n} \cdot n^{\frac{\sigma_0 - 1}{2}}$ if n is a perfect square.

14.3.3 Euler's Totient Function

91. The function is multiplicative. This means that if $\gcd(m,n)=1$, $\phi(m\cdot n)=\phi(m)\cdot\phi(n)$.

92. $\phi(n)=n\prod_{p|n}(1-\frac{1}{p})$

93. If p is prime and $(k\geq 1)$, then: $\phi(p^k)=p^{k-1}(p-1)=p^k(1-\frac{1}{p})$

94. $J_k(n)$, the Jordan totient function, is the number of k-tuples of positive integers all less than or equal to n that form a coprime $(k+1)$-tuple together with n. It is a generalization of Euler's totient, $\phi(n)=J_1(n)$. $J_k(n)=n^k\prod_{p|n}(1-\frac{1}{p^k})$

95. $\sum_{d|n}J_k(d)=n^k$

96. $\sum_{d|n}\phi(d)=n$

97. $\phi(n)=\sum_{d|n}\mu(d)\cdot\frac{n}{d}=n\sum_{d|n}\frac{\mu(d)}{d}$

98. $\phi(n)=\sum_{d|n}d\cdot\mu(\frac{n}{d})$

99. $a|b\rightarrow\varphi(a)|\varphi(b)$

100. $n|\varphi(a^n-1)$ for $a,n>1$

101. $\varphi(mn)=\varphi(m)\varphi(n)\cdot\frac{d}{\varphi(d)}$ where $d=gcd(m,n)$ Note the special cases

$$\varphi(2m)=\begin{cases}2\varphi(m) & ;if\ m\ is\ even\\ \varphi(m) & ;if\ m\ is\ odd\end{cases}$$

$$\varphi(n^m)=n^{m-1}\varphi(n)$$

102. $\varphi(lcm(m,n))\cdot\varphi(gcd(m,n))=\varphi(m)\cdot\varphi(n)$ Compare this to the formula $lcm(m,n)\cdot gcd(m,n)=m\cdot n$

103. $\varphi(n)$ is even for $n\geq 3$. Moreover, if if n has r distinct odd prime factors, $2^r|\varphi(n)$

104. $\sum_{d|n}\frac{\mu^2(d)}{\varphi(d)}=\frac{n}{\varphi(n)}$

105. $\sum_{1\leq k\leq n,\gcd(k,n)=1}k=\frac{1}{2}n\varphi(n)$ for $n>1$

106. $\frac{\varphi(n)}{n}=\frac{\varphi(rad(n))}{rad(n)}$ where $rad(n)=\prod_{p|n,p\ prime}p$

107. $\phi(m)\geq\log_2 m$

108. $\phi(\phi(m))\leq\frac{m}{2}$

109. **If m is prime:**

$$n^x \bmod m = n^{x \bmod (m-1)} \bmod m$$

110. When $x\geq\log_2 m$:

$$n^x \bmod m = n^{\phi(m)+x \bmod \phi(m)} \bmod m$$

111. $\sum_{1\leq k\leq n,\gcd(k,n)=1}\gcd(k-1,n)=\varphi(n)d(n)$ where $d(n)$ is number of divisors. Same equation for $\gcd(a\cdot k-1,n)$ where a and n are coprime.

112. For every n there is at least one other integer $m\neq n$ such that $\varphi(m)=\varphi(n)$.

113. $\sum_{i=1}^{n}\varphi(i)\cdot\lfloor\frac{n}{i}\rfloor=\frac{n*(n+1)}{2}$

114. $\sum_{i=1,i\%2\neq 0}^{n}\varphi(i)\cdot\lfloor\frac{n}{i}\rfloor=\sum_{k\geq 1}[\frac{n}{2^k}]^2$. Note that [ ] is used here to denote round operator not floor or ceil

115. $$\sum_{i=1}^{n}\sum_{j=1}^{n}ij[\gcd(i,j)=1]=\sum_{i=1}^{n}\varphi(i)i^2$$

116. Average of coprimes of n which are less than n is $\frac{n}{2}$.

14.3.4 Mobius Function and Inversion

117. It is a multiplicative function.

118. $$\sum_{d|n}\mu(d)=\begin{cases}1 & ;n=1\\ 0 & ;n>0\end{cases}$$

119. $\sum_{n=1}^{N}\mu^2(n)=\sum_{n=1}^{\sqrt{N}}\mu(k)\cdot\left\lfloor\frac{N}{k^2}\right\rfloor$ This is also the number of square-free numbers $\leq n$

120. **Mobius inversion theorem:** The classic version states that if g and f are arithmetic functions satisfying $g(n)=\sum_{d|n}f(d)$ for every integer $n\geq 1$ then $g(n)=\sum_{d|n}\mu(d)g\left(\frac{n}{d}\right)$ for every integer $n\geq 1$

121. If $F(n)=\prod_{d|n}f(d)$, then $F(n)=\prod_{d|n}F\left(\frac{n}{d}\right)^{\mu(d)}$

122. $\sum_{d|n}\mu(d)\phi(d)=\prod_{j=1}^{K}(2-P_j)$ where p_j is the primes factorization of d

123. If $F(n)$ is multiplicative, $F\not\equiv 0$, then $\sum_{d|n}\mu(d)f(d)=\prod_{i=1}(1-f(P_i))\cdot$ where p_i are primes of n.

14.3.5 GCD and LCM

124. $\gcd(a,0)=a$

125. $\gcd(a,b)=\gcd(b,a \mod b)$

126. Every common divisor of a and b is a divisor of $\gcd(a,b)$.

127. if m is any integer, then $\gcd(a+m\cdot b,b)=\gcd(a,b)$

128. The gcd is a multiplicative function in the following sense: if a_1 and a_2 are relatively prime, then $\gcd(a_1\cdot a_2,b)=\gcd(a_1,b)\cdot\gcd(a_2,b)$.

129. $\gcd(a,b)\cdot\operatorname{lcm}(a,b)=|a\cdot b|$

130. $\gcd(a,\operatorname{lcm}(b,c))=\operatorname{lcm}(\gcd(a,b),\gcd(a,c))$.

131. $\operatorname{lcm}(a,\gcd(b,c))=\gcd(\operatorname{lcm}(a,b),\operatorname{lcm}(a,c))$.

132. For non-negative integers a and b, where a and b are not both zero, $\gcd(n^a-1,n^b-1)=n^{\gcd(a,b)}-1$

133. $\gcd(a,b)=\sum_{k|a\text{ and }k|b}\phi(k)$

134. $\sum_{i=1}^{n}[\gcd(i,n)=k]=\phi\left(\frac{n}{k}\right)$

135. $\sum_{k=1}^{n}\gcd(k,n)=\sum_{d|n}d\cdot\phi\left(\frac{n}{d}\right)$

136. $\sum_{k=1}^{n}x^{\gcd(k,n)}=\sum_{d|n}x^d\cdot\phi\left(\frac{n}{d}\right)$

137. $\sum_{k=1}^{n}\frac{1}{\gcd(k,n)}=\sum_{d|n}\frac{1}{d}\cdot\phi\left(\frac{n}{d}\right)=\frac{1}{n}\sum_{d|n}d\cdot\phi(d)$

138. $\sum_{k=1}^{n}\frac{k}{\gcd(k,n)}=\frac{n}{2}\cdot\sum_{d|n}\frac{1}{d}\cdot\phi\left(\frac{n}{d}\right)=\frac{n}{2}\cdot\frac{1}{n}\cdot\sum_{d|n}d\cdot\phi(d)$

139. $\sum_{k=1}^{n}\frac{n}{\gcd(k,n)}=2*\sum_{k=1}^{n}\frac{k}{\gcd(k,n)}-1$, for $n>1$

140. $\sum_{i=1}^{n}\sum_{j=1}^{n}[\gcd(i,j)=1]=\sum_{d=1}^{n}\mu(d)\lfloor\frac{n}{d}\rfloor^2$

141. $\sum_{i=1}^{n}\sum_{j=1}^{n}\gcd(i,j)=\sum_{d=1}^{n}\phi(d)\lfloor\frac{n}{d}\rfloor^2$

142. $\sum_{i=1}^{n}\sum_{j=1}^{n}i\cdot j[\gcd(i,j)=1]=\sum_{i=1}^{n}\phi(i)i^2$

143. $F(n)=\sum_{i=1}^{n}\sum_{j=1}^{n}\operatorname{lcm}(i,j)=\sum_{l=1}^{n}\left(\frac{(1+\lfloor\frac{n}{l}\rfloor)(\lfloor\frac{n}{l}\rfloor)}{2}\right)^2\sum_{d|l}\mu(d)ld$

144. $\gcd(\operatorname{lcm}(a,b),\operatorname{lcm}(b,c),\operatorname{lcm}(a,c))=\operatorname{lcm}(\gcd(a,b),\gcd(b,c),\gcd(a,c))$

145. $\gcd(A_L,A_{L+1},\ldots,A_R)=\gcd(A_L,A_{L+1}-A_L,\ldots,A_R-A_{R-1})$.

146. Given n, If $SUM=LCM(1,n)+LCM(2,n)+\cdots+LCM(n,n)$ then SUM $=\frac{n}{2}(\sum_{d|n}(\phi(d)\times d)+1$

14.3.6 Legendre Symbol

147. $\left(\frac{a}{p}\right)\equiv a^{\frac{p-1}{2}}\pmod p$ and $\left(\frac{a}{p}\right)\in\{-1,0,1\}$.

148. The Legendre symbol is periodic in its first (or top) argument: if $a\equiv b\pmod p$, then $\left(\frac{a}{p}\right)=\left(\frac{b}{p}\right)$.

149. The Legendre symbol is a completely multiplicative function of its top argument: $\left(\frac{ab}{p}\right)=\left(\frac{a}{p}\right)\left(\frac{b}{p}\right)$

150. The Fibonacci numbers $1,1,2,3,5,8,13\ldots$ are defined by the recurrence $F_1=F_2=1, F_{n+1}=F_n+F_{n-1}$. If p is a prime number then $F_{p-\left(\frac{p}{5}\right)}\equiv 0\pmod p$, $F_p\equiv\left(\frac{p}{5}\right)\pmod p$.

151. Continuing from previous point, $\left(\frac{p}{5}\right)=$ infinite concatenation of the sequence $(1,-1,-1,1,0)$ from $p\geq 1$.

152. If $n=k^2$ is perfect square then $\left(\frac{n}{p}\right)=1$ for every odd prime except $\left(\frac{n}{k}\right)=0$ if k is an odd prime.

14.4 Geometry

14.4.1 Triangle

- **Sine Rule:** $\frac{a}{\sin A}=\frac{b}{\sin B}=\frac{c}{\sin C}=2R$
- **Cosine Rule:** $c^2=a^2+b^2-2ab\cos C$
- **Area:** $\Delta=\sqrt{s(s-a)(s-b)(s-c)}$ $=\frac{1}{2}bc\sin A$
- **Circumradius (R):** $R=\frac{abc}{4\Delta}=\frac{a}{2\sin A}$
- **Inradius (r):** $r=\frac{\Delta}{s}$
- **Ex-radii (r_a,r_b,r_c):** $r_a=\frac{\Delta}{s-a}$
- **Centroid (G):** $\left(\frac{\sum x}{3},\frac{\sum y}{3}\right)$
- **Incenter (I):** $\left(\frac{ax_1+bx_2+cx_3}{a+b+c},\frac{ay_1+by_2+cy_3}{a+b+c}\right)$
- **Excenter (I_A):** Replace a with $-a$ in Incenter formula.
- **Euler Line:** O,G,H are collinear; $HG=2GO$.
- **Median (m_a):** $4m_a^2=2b^2+2c^2-a^2$ (Apollonius)
- **Angle Bisector (l_a):** $l_a=\frac{2bc}{b+c}\cos(A/2)$
- **Stewart's Theorem:** Cevian(a line from A to a) length d dividing a into m,n:

$$b^2m+c^2n=a(d^2+mn)$$

- **Centroid (G):** Intersection of *medians* (vertex to midpoint).
- **Incenter (I):** Intersection of *angle bisectors*.
- **Circumcenter (O):** Intersection of *perpendicular bisectors* of the sides.
- **Orthocenter (H):** Intersection of *altitudes* (vertex perpendicular to opposite side).

14.4.2 Circle

- **General Eq:** $x^2 + y^2 + 2gx + 2fy + c = 0$. Center $(-g, -f)$, $r = \sqrt{g^2 + f^2 - c}$.
- **Tangent at $P(x_1, y_1)$:** $xx_1 + yy_1 + g(x + x_1) + f(y + y_1) + c = 0$
- **Slope of tangent from $P(x, y)$:** $y + f = m(x + g) \pm r\sqrt{1 + m^2}$
- **Tangency Condition ($y = mx + c$):** $c^2 = r^2(1 + m^2)$
- **Chord Length:** $L = 2\sqrt{r^2 - d^2}$ (d = dist to center)
- **Power of Point P:** $d^2 - r^2$ (d = dist P to center) (Useful for determining if a point is inside or outside)
- **Secant Theorem:** $PA \cdot PB = PC \cdot PD$ (AB and CD are 2 lines drawn from point P over the circle)
- **Sector Area:** $\frac{1}{2}r^2\theta$ (θ in rads)
- **Segment Area (Cut by chord):** $\frac{1}{2}r^2(\theta - \sin\theta)$

14.4.3 Vector

- **Direction Cosines:** $l = \cos\alpha, m = \cos\beta, n = \cos\gamma$. $l^2 + m^2 + n^2 = 1$.
- **Dot Product:** $\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}||\mathbf{b}| \cos\theta$
- **Projection of a on b:** $(\mathbf{a} \cdot \hat{b})\hat{b}$
- **Cross Product:** Area $\triangle = \frac{1}{2}|\mathbf{a} \times \mathbf{b}|$.
- **Scalar Triple (Vol):** $[\mathbf{abc}] = \mathbf{a} \cdot (\mathbf{b} \times \mathbf{c})$. Tet Vol $= \frac{1}{6}|[\dots]|$.
- **Vector Triple:** $\mathbf{a} \times (\mathbf{b} \times \mathbf{c}) = (\mathbf{a} \cdot \mathbf{c})\mathbf{b} - (\mathbf{a} \cdot \mathbf{b})\mathbf{c}$

14.4.4 Plane

Through $\mathbf{a}$, normal $\mathbf{n}$. Eq: $(\mathbf{r} - \mathbf{a}) \cdot \mathbf{n} = 0$.

- **Cartesian:** $Ax + By + Cz + D = 0$. Normal (A, B, C).
- **Dist Point P to Plane:** $D = \frac{|Ax_1 + By_1 + Cz_1 + D|}{\sqrt{A^2 + B^2 + C^2}}$
- **Angle Between Planes:** $\cos\theta = \frac{|\mathbf{n}_1 \cdot \mathbf{n}_2|}{|\mathbf{n}_1||\mathbf{n}_2|}$
- **Angle Between Line & Plane:** $\sin\phi = \frac{|\mathbf{b} \cdot \mathbf{n}|}{|\mathbf{b}||\mathbf{n}|}$
- **Intersection (Line of two planes):** Direction $\mathbf{v} = \mathbf{n}_1 \times \mathbf{n}_2$.
- **Family of Planes:** $P_1 + \lambda P_2 = 0$ (Planes through intersection line).
- **Foot of Perp from P:** $\frac{x - x_1}{A} = \frac{y - y_1}{B} = \frac{z - z_1}{C} = \frac{-(Ax_1 + By_1 + Cz_1 + D)}{A^2 + B^2 + C^2}$
- **Image of Point in Plane:** Use -2 instead of -1 above.
- **Plane Bisectors:** $\frac{A_1x + B_1y + C_1z + D_1}{\sqrt{A_1^2 + B_1^2 + C_1^2}} = \pm\frac{A_2x + B_2y + C_2z + D_2}{\sqrt{A_2^2 + B_2^2 + C_2^2}}$

14.4.5 Sphere

- **Standard:** $(x - u)^2 + (y - v)^2 + (z - w)^2 = r^2$.
- **Diameter Form:** $(x - x_1)(x - x_2) + (y - y_1)(y - y_2) + (z - z_1)(z - z_2) = 0$.
- **Tangent Plane at P:** $(\mathbf{r} - \mathbf{c}) \cdot (\mathbf{p} - \mathbf{c}) = r^2$.
- **Circle of Intersection:** Radius $R = \sqrt{r^2 - d^2}$.
- **Spherical Cosine Rule:** $\cos a = \cos b \cos c + \sin b \sin c \cos A$.

14.4.6 Solid

- **Cone:** $V = \frac{1}{3}\pi r^2 h$, $SA = \pi r(r + l)$
- **Sphere:** $V = \frac{4}{3}\pi r^3$, $SA = 4\pi r^2$
- **Spherical Cap:** $SA = 2\pi rh$. $V = \frac{1}{3}\pi h^2(3r - h)$.
- **Frustum (Cone):** $V = \frac{1}{3}\pi h(R^2 + r^2 + Rr)$. $LSA = \pi(R + r)l$.
- **Pyramid:** $V = \frac{1}{3}$(Base Area) $\times\ h$. $LSA = \frac{1}{2}$(Perimeter) $\times\ l$.
- **Regular Tetrahedron:** Side a. $H = a\sqrt{2/3}$. $V = \frac{a^3}{6\sqrt{2}}$. Face Angle $\cos\theta = 1/3$.

14.4.7 Trigonometry

- $\sin(A \pm B) = \sin A \cos B \pm \cos A \sin B$
- $\cos(A \pm B) = \cos A \cos B \mp \sin A \sin B$
- $\cos 2A = \cos^2 A - \sin^2 A = 2\cos^2 A - 1$
- $\tan^{-1} x \pm \tan^{-1} y = \tan^{-1}\left(\frac{x \pm y}{1 \mp xy}\right)$ *(Note: For $+$, if $xy > 1$, add π to the result.)*
- $2\tan^{-1} x = \tan^{-1}\left(\frac{2x}{1 - x^2}\right) = \sin^{-1}\left(\frac{2x}{1 + x^2}\right) = \cos^{-1}\left(\frac{1 - x^2}{1 + x^2}\right)$
- $\tan^{-1} x + \cot^{-1} x = \frac{\pi}{2}$

14.4.8 Complex Numbers

1. $|z| = \sqrt{a^2 + b^2} = \sqrt{\mathrm{Re}(z)^2 + \mathrm{Im}(z)^2}$
2. $|z|^2 = z \cdot \bar{z}$
3. $\frac{1}{z} = \frac{\bar{z}}{|z|^2} \quad (z \neq 0)$
4. $|z| = 1 \iff \bar{z} = \frac{1}{z}$
5. $\left|\frac{z}{|z|}\right| = 1$ (Projection onto unit circle)
6. $|z| = |\bar{z}| = |-z| = |-\bar{z}| = |iz|$
7. $|z_1 \cdot z_2| = |z_1| \cdot |z_2|$
8. $\left|\frac{z_1}{z_2}\right| = \frac{|z_1|}{|z_2|}$
9. $|z^n| = |z|^n$
10. $|z_1 \pm z_2|^2 = |z_1|^2 + |z_2|^2 \pm 2\mathrm{Re}(z_1\bar{z}_2)$
11. $\mathrm{Re}(z) = \frac{z + \bar{z}}{2}, \quad \mathrm{Im}(z) = \frac{z - \bar{z}}{2i}$
12. $|z_1 + z_2| \leq |z_1| + |z_2|$ (equality iff $\arg(z_1) = \arg(z_2)$)
13. $\big||z_1| - |z_2|\big| \leq |z_1 - z_2|$ (Reverse Triangle Inequality)
14. $|z_1 + z_2|^2 + |z_1 - z_2|^2 = 2(|z_1|^2 + |z_2|^2)$ (Parallelogram Law)

14.5 Miscellaneous

15. $a + b = a \oplus b + 2(a \& b)$.
16. $a + b = a \mid b + a \& b$
17. $a \oplus b = a \mid b - a \& b$
18. k_{th} bit is set in x iff $x \mod 2^{k-1} \geq 2^k$. It comes handy when you need to look at the bits of the numbers which are pair sums or subset sums etc.
19. k_{th} bit is set in x iff $x \mod 2^{k-1} - x \mod 2^k \neq 0$ ($= 2^k$ to be exact). It comes handy when you need to look at the bits of the numbers which are pair sums or subset sums etc.
20. $n \mod 2^i = n \& (2^i - 1)$
21. $1 \oplus 2 \oplus 3 \oplus \cdots \oplus (4k - 1) = 0$ for any $k \geq 0$
22. **Erdos Gallai Theorem:** The degree sequence of an undirected graph is the non-increasing sequence of its vertex degrees A sequence of non-negative integers $d_1 \geq d_2 \geq \cdots \geq d_n$ can be represented as the degree sequence of finite simple graph on n vertices if and only if $d_1 + d_2 + \cdots + d_n$ is even and
$$\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k)$$
holds for every k in $1 \leq k \leq n$.
23. $f_n = \sum_{i=0}^{n} \binom{n}{i} g_i \Rightarrow g_n = \sum_{i=0}^{n} (-1)^{n-i} \binom{n}{i} f_i$